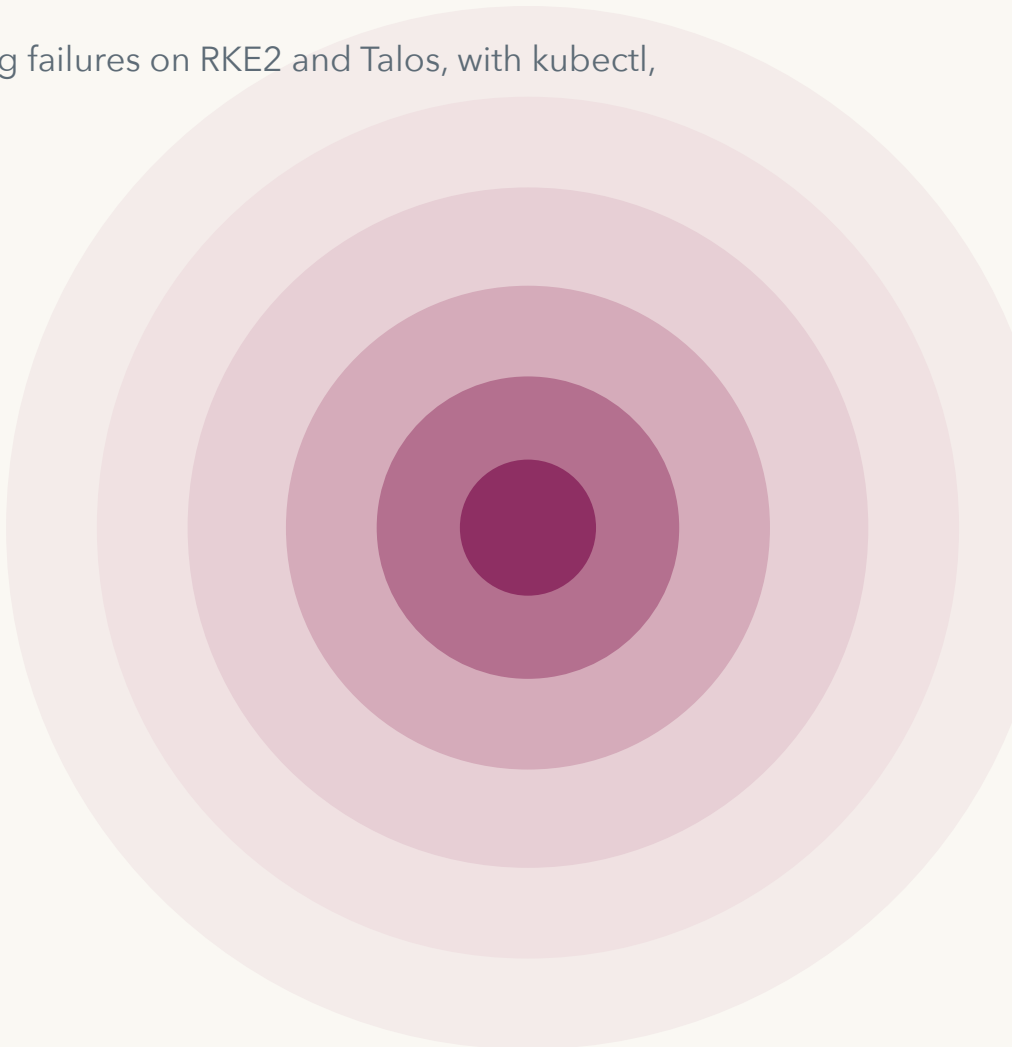
HANDS-ON GUIDE

Debugging Kubernetes

Finding and fixing failures on RKE2 and Talos, with kubectl, k9s and talosctl



Thomas Zachmann

As of November 9, 2026

Contents

How to Use This Book 6
Why Two Distributions 6
First, Clone the Repo 6
Where You Run a Command 7
The Boxes 7
How a Scenario Works 8
What Was Tested and What Was Not 8
What You Need 9
1 The Method 12
1.1 Guessing or Narrowing Down 12
1.2 The Map 13
1.3 Phase, State and Reason 14
1.4 Exit Codes 15
1.5 The Basic Tools 16
1.6 An Example from Start to Finish 18
2 The Lab 22
2.1 What You Build 22
2.2 Tools on the Mac 23
2.3 RKE2: The Configuration 24
2.4 RKE2: The Server 25
2.5 RKE2: The Agent 26
2.6 The RKE2 Kubeconfig 27
2.7 Talos: Booting 27
2.8 Talos: The Patches 28
2.9 Talos: Generating the Configuration 29
2.10 Talos: Applying and Bootstrapping 30
2.11 Storage: local-path-provisioner 31
2.12 Setting Up k9s 33
2.13 Wrap-Up: Both Clusters at a Glance 33
3 k9s as a Cockpit 36
3.1 Why a Second Tool 36
3.2 Starting and Finding Your Way 37

3.3 Resources, Namespaces and Filters 38
3.4 A Pod in Detail 39
3.5 Reading Logs 40
3.6 Events, XRay and Pulses 40
3.7 Port Forward 41
3.8 Contexts and Read-Only 41
3.9 This Book's Plugins 42
3.10 Where kubectl Is Better 44
4 Pods Stay Pending 46
4.1 What Breaks in This Chapter 46
4.2 The Scheduler Explains Itself 46
4.3 Requests Too Large 47
4.4 nodeSelector and Taint 49
4.5 PVC Without StorageClass 52
4.6 ResourceQuota: The Pod That Does Not Exist 54
5 Images and Container Start 58
5.1 What Breaks in This Chapter 58
5.2 Three Pull Errors, Three Messages 58
5.3 Looking on the Node 61
5.4 Rate Limit and Registry Mirror 62
5.5 CreateContainerConfigError 64
5.6 exec format error 66
6 Crashes and Restarts 70
6.1 What Breaks in This Chapter 70
6.2 CrashLoopBackOff: Command and Application 70
6.3 OOMKilled 73
6.4 Probes: Too Strict and Aimed Wrong 74
6.5 readOnlyRootFilesystem 77
6.6 Distroless: Debugging Without a Shell 78
7 Services, DNS and NetworkPolicies 84
7.1 What Breaks in This Chapter 84
7.2 The Service Finds No Pods 84
7.3 Endpoints Exist, Connection Still Refused 87
7.4 Names, Namespaces and ndots 88

7.5 Default Deny Blocks DNS 90
7.6 The Policy That Does Nothing 93
8 Ingress, Gateway and TLS 96
8.1 What Breaks in This Chapter 96
8.2 Traefik on Both Clusters 96
8.3 404: The Controller Does Not Feel Responsible 99
8.4 404 Despite the Right Class 100
8.5 503: The Route Exists, but Nobody Is Ready 102
8.6 TLS: The Wrong Certificate 103
8.7 The Dashboard and the Gateway API 106
9 Storage 108
9.1 What Breaks in This Chapter 108
9.2 The PVC Cannot Find Its StorageClass 109
9.3 Talos: The Helper Pod Is Not Allowed to Start 110
9.4 Two Replicas, One Volume, Two Nodes 113
9.5 Permission Denied on the Volume 115
9.6 When Space Runs Out 117
10 RBAC and Pod Security 120
10.1 What Breaks in This Chapter 120
10.2 Reading a Forbidden Message 120
10.3 RoleBinding in the Wrong Namespace 121
10.4 Role Instead of ClusterRole 124
10.5 Pod Security: What the Distributions Set 125
10.6 hostPath: Runs on RKE2, Not on Talos 126
10.7 restricted: Deployment Yes, Pods No 128
11 Failures Caused by Kyverno 132
11.1 What Breaks in This Chapter 132
11.2 Installing Kyverno 132
11.3 Policy Rejects the Deployment 133
11.4 Policy Rejects Only the Pods 135
11.5 Mutation Changes the Pod Unexpectedly 137
11.6 Webhook Unreachable 140
11.7 PolicyException Without Effect 142
11.8 Test Policies Before They Get in the Way 144

12 Falco: Alerts and Disruptions 148
12.1 What Breaks in This Chapter 148
12.2 Installing Falco 148
12.3 Falco Does Not Start on Talos 149
12.4 Alert After `kubectl exec` 151
12.5 False Positive From a Legitimate Job 153
12.6 No Kubernetes Metadata 155
13 Nodes 160
13.1 What Breaks in This Chapter 160
13.2 The Toolbox for Nodes 160
13.3 Node Becomes NotReady 161
13.4 Pods Hang After a Node Failure 165
13.5 Pods Get Evicted 167
13.6 Drain Hangs on a PodDisruptionBudget 170
13.7 Clean Up 172
14 Control Plane, etcd and Webhooks 174
14.1 What Breaks in This Chapter 174
14.2 Is the API Healthy? 174
14.3 Before Every Intervention: Check and Back Up etcd 175
14.4 The API Server Does Not Start 177
14.5 Expired Client Certificate 179
14.6 A Webhook Blocks the Namespace 182
14.7 The Namespace Stays in Terminating 184
15 Rollouts, Autoscaling and Helm 188
15.1 What Breaks in This Chapter 188
15.2 The Rollout Hangs 188
15.3 The HPA Does Not Measure 191
15.4 The Helm Release Hangs 194
15.5 The Job Gives Up 196
15.6 The ConfigMap Change Does Not Arrive 197
15.7 Clean Up 198
16 Appendix 200
16.1 Cheat Sheet: From the Message to the First Command 200
16.2 RKE2 and Talos: Same Question, Two Commands 202

16.3 k9s: The Most Important Keys 203
16.4 Versions 204
16.5 The Repo at a Glance 204
16.6 Official Documentation 205

How to Use This Book

A Kubernetes cluster rarely admits what is wrong with it. It shows you `Pending`, `CrashLoopBackOff` or a terse `Forbidden` and leaves the rest to you. If you have ever stared at a pod for an hour that simply would not start, you know the feeling. Most of the time the answer was sitting in an event that nobody read.

This book turns troubleshooting into a routine. You build two small clusters and break them on purpose, again and again and in many ways. For every failure you see how it shows up, how you investigate it with `kubectl` and k9s, what the actual cause is and how you fix it. After a few chapters you will notice that the questions repeat. That is exactly the goal.

Why Two Distributions

Kubernetes is the same everywhere, the platform underneath is not. Troubleshooting often does not end at the pod but on the node: at the kubelet, at containerd, at etcd or at a setting that comes with the distribution. That is why you work with two very different representatives:

	RKE2	Talos Linux
Operating system	Ubuntu 24.04, RKE2 as a systemd service	Talos, an immutable Linux just for Kubernetes
Node access	SSH, `journalctl`, `crictl`	no shell, only the API with `talosctl`
Configuration	`/etc/rancher/rke2/config.yaml`	machine config, applied with `talosctl`
Network	Canal, enforces NetworkPolicies	Flannel, ignores NetworkPolicies
Bundled	Traefik, metrics-server, CoreDNS	CoreDNS, otherwise deliberately little

RKE2 feels like a classic Linux server. On Talos many familiar moves do not work because there is no shell. Where troubleshooting differs, a box shows both paths side by side:

RKE2
This is how you proceed on RKE2, usually via SSH on the node.

TALOS
This is the path through `talosctl`, without a shell on the node.

First, Clone the Repo

Every file this book refers to lives in a public repository: github.com/thomaszachmann/kubernetes-debuggen. Each failure scenario has its own folder under `szenarien/` with one file that creates the failure and one that fixes it. The repo is shared with the German edition, so its file and

folder names are German: `szenarien` means scenarios, `kaputt.yaml` means broken, and `fix.yaml` is the fix.

MAC Step 1

```
git clone https://github.com/thomaszachmann/kubernetes-debuggen.git
cd kubernetes-debuggen
```

Downloads the repo and switches to its root directory. All commands labeled **Mac** run from here. Paths like `szenarien/04-pending/` are always relative to this folder.

Where You Run a Command

Every command carries a colored label:

Label	Meaning
MAC	Your terminal on the laptop, in the root directory of the repo. From here you control both clusters with `kubectl --context rke2` or `--context talos`, and the Talos nodes with `talosctl`.
K9S	Keystrokes in k9s. `⏎` stands for the Enter key.
RKE2-CP	An SSH session on the RKE2 server `rke2-cp` (`172.16.0.160`).
RKE2-W1	An SSH session on the RKE2 agent `rke2-w1` (`172.16.0.161`).
POD	A shell in a container, opened with `kubectl exec` or `kubectl debug`.

There is no separate label for Talos. You never log in to a Talos node, `talosctl` always runs on the Mac.

WHY IT MATTERS

With two clusters, the most common mistake is the wrong context. That is why every command names `--context rke2` or `--context talos` explicitly. This book never uses the active default context of `kubectl`. Make this a habit in your daily work too: a fix in the wrong cluster is worse than no fix at all.

The Boxes

SYMPTOM

What you see when the failure occurs. Every investigation starts here.

CAUSE

What is really behind it, and why Kubernetes behaves this way.

WHAT YOU SHOULD SEE

The expected result. If your output differs, do not move on. Clear up the difference first.

WHY IT MATTERS

The background that helps you with the next, similar problem.

PITFALL

What typically goes wrong and how you recognize it.

TRY IT YOURSELF

This is your turn. Think it through yourself first before you look things up.

How a Scenario Works

From chapter 4 on, every scenario follows the same pattern. You apply a file that contains a failure. Then you see the symptom, investigate it step by step and find the cause. Only then do you fix the failure and check the result. Do not read the cause ahead of time. You learn while searching, not while reading the solution.

Every scenario runs in its own namespace, for example `dbg-04` for chapter 4. Whatever you break there stays there. At the end of a chapter you clean up with a single command:

MAC — Step 2

```
kubectl --context rke2 delete namespace dbg-04
kubectl --context talos delete namespace dbg-04
```

Deletes the namespace and every object in it, on both clusters.

The chapters build on each other loosely. Read chapters 1 to 3 in order. After that you can jump around, for example straight to Kyverno in chapter 11 if that is what you are dealing with right now.

What Was Tested and What Was Not

A book full of commands is only as good as its testing. So here is an honest account of how far that testing went.

Tested on November 9, 2026:

- All versions come from the official release pages and Helm repositories: RKE2, Talos, k9s, Kyverno, Falco, Traefik, metrics-server and local-path-provisioner.
- All Kubernetes manifests in the repo were checked with `kubeconform` against Kubernetes 1.36.
- All Kyverno policies pass `kyverno test` with Kyverno CLI 1.19.1 against the test resources in the repo.
- All Helm values were rendered with `helm template` against the chart versions given, and the shell scripts pass `shellcheck`.
- The Talos patches were merged into a complete configuration and checked with `talosctl validate`, though with `talosctl` 1.12. Commands that only Talos 1.14 knows were looked up in the 1.14.5 documentation and source code.
- Error messages from Kubernetes, Kyverno, Traefik and Helm were read in the source code of the versions given. Commands, flags and paths for RKE2, k9s and `kubectl` come from the official documentation and the CLIs themselves.

Not tested:

- The book was **not** run through on real clusters. All output in the boxes and code blocks are examples that follow the format of the real tools. Pod names, times and IDs will look different on your machine.
- The wording of error messages can change slightly between versions. The core, meaning the keyword such as `Insufficient cpu` or `violates PodSecurity`, usually stays the same.

So expect something to differ at some point. For this book that is a gift: it is a real failure, and you are holding the right tool to find it.

What You Need

- Four VMs with fixed IP addresses on the same network:

Role	Hostname	IP	Size
RKE2 server	`rke2-cp`	`172.16.0.160`	4 CPUs, 8 GB RAM, 60 GB disk, Ubuntu 24.04
RKE2 agent	`rke2-w1`	`172.16.0.161`	2 CPUs, 4 GB RAM, 40 GB disk, Ubuntu 24.04
Talos control plane	`talos-cp`	`172.16.0.170`	4 CPUs, 8 GB RAM, 60 GB disk, Talos ISO
Talos worker	`talos-w1`	`172.16.0.171`	2 CPUs, 4 GB RAM, 40 GB disk, Talos ISO

- On the laptop: `kubectl`, `talosctl`, `helm`, `k9s`, `jq`, `curl`, `openssl` and `git`. For chapter 11 also the Kyverno CLI.
- SSH access to the two Ubuntu VMs with a user who can run `sudo`.

- Basic Kubernetes knowledge: you know what pods, deployments, services and namespaces are. The book explains everything else.

DIFFERENT IP ADDRESSES?

The four addresses appear in the configurations under `labor/` and in many commands. If yours are different, replace them everywhere before you start chapter 2. A `grep -rn "172.16.0.1[67]" .` shows you every occurrence.

CHAPTER 1

The Method

You assign every symptom to a layer of the cluster, know the basic `kubectl` tools for each layer and read pod status, container state and exit code correctly. A small example shows the process from start to finish.

Time: 45 minutes **Requires:** Basic Kubernetes knowledge; for the example, the lab from Chapter 2

1.1 Guessing or Narrowing Down

When you face a pod in `CrashLoopBackOff` for the first time, it is tempting to just try things: rebuild the image, delete the pod, reapply the deployment. Sometimes that even helps. But you have not understood anything, and next time you start from scratch again.

This book works differently. Every failure goes through the same five steps:

Step	What you do
Reproduce	Trigger the failure on purpose. If you cannot trigger it, you cannot reliably declare it fixed either.
Investigate	Commands in a fixed order, from the outside in. You collect facts, not guesses.
Understand	Name the cause in two or three sentences, including the mechanism behind it.
Fix	The smallest change that removes the cause. Not the symptom.
Verify	Show that it works again, using the same signal where you saw the failure.

The most important step is the second one. Good troubleshooters are not faster at guessing. They ask better questions in a better order. Every question cuts the space of possible causes in half. After three or four questions, usually only one is left.

WHY IT MATTERS

While you search, jot down what you checked and what came out of it. It sounds tedious, but it keeps you from running the same command three times. And if you hand the case over to a colleague, half the analysis is already done.

1.2 The Map

A Kubernetes cluster consists of layers that build on each other. Almost every symptom belongs to one of them. If you know the layer, you also know the first commands.

Layer	Typical symptom	First question	Chapter
Scheduling	Pod stays `Pending`	Why does the scheduler not find a node?	4
Image and runtime	`ImagePullBackOff`, `CreateContainerConfigError`	Can the node pull the image and create the container?	5
Workload	`CrashLoopBackOff`, `OOMKilled`, not ready	What does the process itself say, and how does it end?	6
Network	Timeout, `connection refused`, DNS error	Does the packet arrive, and does anyone answer?	7, 8
Storage	PVC `Pending`, `Permission denied`	Is there a volume, and may the process write to it?	9
Policy and admission	`Forbidden`, `denied the request`	Who rejected it: RBAC, Pod Security or a webhook?	10, 11
Runtime	Falco alert, process behaves unexpectedly	Is this an attack or a legitimate workflow?	12
Node	`NotReady`, evictions, `Unknown`	Is the kubelet alive, and does the node have enough resources?	13
Control plane	`kubectl` hangs, changes do not arrive	Are the API server, etcd and webhooks healthy?	14
Rollout	Deployment stuck, HPA shows `<unknown>`	Is the controller waiting for something that never happens?	15

The layers have a direction. A failure further down often shows up further up. A node with a full disk leads to evictions, which you first see as pods that disappeared. A hanging webhook in the control plane makes every `kubectl apply` fail, even though your manifests are fine. So you start at the top with the symptom, but you are ready to go one layer deeper as soon as the facts point there.

From the Outside In

The investigation follows a fixed order. It is chosen so that every step is cheap and prepares the next one:

1. `kubectl get`: What is the state, and since when?
2. `kubectl describe`: What does Kubernetes know about this object, and which events are attached to it?
3. `kubectl events`: What happened around it, including objects you have not looked at yet?
4. `kubectl logs`: What does the process say, including the previous run?
5. `kubectl debug`: What does it look like from the inside, in the network and file system of the pod?
6. Node: kubelet, containerd, kernel. On RKE2 via SSH, on Talos with `talosctl`.
7. Control plane: API server, etcd, webhooks.

Most failures are explained after step 4. You rarely need steps 6 and 7, but when you do, you need them urgently. That is why you practice them in chapters 13 and 14 on a healthy cluster, before things get serious.

1.3 Phase, State and Reason

The `STATUS` column of `kubectl get pods` is a summary that `kubectl` builds from several fields. If you read it as a single field, you draw the wrong conclusions. There are three levels behind it:

Level	Field	Values
Pod phase	`status.phase`	`Pending`, `Running`, `Succeeded`, `Failed`, `Unknown`. Only these five.
Container state	`status.containerStatuses[].state`	Exactly one of `waiting`, `running`, `terminated`
Reason	`state.waiting.reason`, `state.terminated.reason`	`ContainerCreating`, `ImagePullBackOff`, `CrashLoopBackOff`, `CreateContainerConfigError`, `Completed`, `Error`, `OOMKilled` and others

A pod in `CrashLoopBackOff` has the phase `Running`. That surprises many people. The phase only says that the pod is assigned to a node and at least one container has started or is restarting right now. The container itself is in `waiting` with the reason `CrashLoopBackOff`, and `kubectl` shows that reason in the `STATUS` column. `Terminating` is not a phase either. `kubectl` shows it as soon as the pod has a `deletionTimestamp`.

For the cause, `lastState` often matters more than the current state. It tells you how the previous run of the container ended:

MAC Step 1

```
kubectl --context rke2 get pod <pod> -n dbg-01 \
  -o jsonpath='{.status.containerStatuses[0].lastState.terminated}'
```

Shows reason, exit code, start and end time of the last terminated run. `describe` shows the same under `Last State`, but here you get it in machine-readable form, for example for a script.

1.4 Exit Codes

When a process in the container ends, the runtime reports its exit code to the kubelet. If the process was killed by a signal, the code is 128 plus the signal number, following the Unix convention. You should know these seven codes by heart:

Code	Meaning	Typical cause
`0`	Exited normally	Correct for a job. For a deployment it is a failure: the main process must not end, otherwise Kubernetes restarts it.
`1`	General error	The application aborted on its own. The answer is almost always in the log, often in the previous one.
`126`	Not executable	File found, but no execute permission, or a script without a matching interpreter.
`127`	Command not found	Typo in the `command`, program missing from the image. The shell in the container reports `not found`.
`137`	SIGKILL (128 + 9)	With reason `OOMKilled`: memory limit exceeded. Without it: killed hard, for example because the process did not react to SIGTERM after a failed liveness probe.
`139`	SIGSEGV (128 + 11)	Segmentation fault, usually a bug in native code, for example in a C library the application uses.
`143`	SIGTERM (128 + 15)	Shut down in an orderly way, for example during a rollout or by `kubectl delete`. Rarely a failure.

PITFALL

A container whose program cannot even be started, for example because the path in `command` does not exist and there is no shell in between, does not end with 127. containerd then reports the reason `StartError` with exit code `128` and a message like `executable file not found in $PATH`. Only when a shell looks up the command, as with `sh -c`, do you get the 127.

1.5 The Basic Tools

All commands in this section work the same against both clusters. The examples show `--context rke2`. For Talos you only swap the context.

get

MAC — Step 2

```
kubectl --context rke2 get pods -n dbg-01 -o wide
```

`-o wide` adds the pod IP and the node. This shows you right away whether all broken pods are on the same node. If they are, the cause is more likely the node than the application.

In the output, watch `READY` (how many containers are ready), `RESTARTS` (with the time of the last restart) and `AGE`. A pod with 40 restarts in two hours has a different problem than one that crashed for the first time five minutes ago. With `-w` you watch changes live, with `-o yaml` you see the complete object including `status`.

describe

MAC — Step 3

```
kubectl --context rke2 describe pod <pod> -n dbg-01
```

Summarizes the spec, the status of all containers, the conditions and the events of the pod. Read from bottom to top: first the events, then `State` and `Last State` of the containers.

`describe` only shows events that are attached directly to this object. If, for example, Pod Security rejects a pod, the pod is never created. The event is then attached to the ReplicaSet, and a `describe` on the pod is impossible because it does not exist. That is why the next command always belongs in the routine.

events

MAC — Step 4

```
kubectl --context rke2 events -n dbg-01
kubectl --context rke2 events -n dbg-01 --for pod/<pod>
kubectl --context rke2 events -A --types=Warning
```

`kubectl events` lists events sorted by time, newest at the bottom. `--for` filters on one object, `--types=Warning` hides the normal messages. `-A` searches all namespaces.

The older `kubectl get events` does not sort by time. If you still need it, for example for a particular output format, add `--sort-by=.lastTimestamp`.

PITFALL

Events are short-lived. By default the API server deletes them after one hour. A failure from last night has no events left this morning. Then all you have is `status`, the logs and, if you have one, a central logging system.

logs

MAC — Step 5

```
kubectl --context rke2 logs <pod> -n dbg-01
kubectl --context rke2 logs <pod> -n dbg-01 --previous
kubectl --context rke2 logs <pod> -n dbg-01 -c <container>
kubectl --context rke2 logs <pod> -n dbg-01 --all-containers --since=10m
kubectl --context rke2 logs deploy/app -n dbg-01 --tail=50
```

The first command shows the log of the running container. If the container keeps restarting, that log is often empty or only shows the beginning. The cause is then in the previous run: `--previous`. With several containers you pick one with `-c` or use `--all-containers`. `deploy/app` picks one pod of the deployment.

The kubelet keeps exactly one log from the previous run. If the pod is deleted and recreated, for example by a rollout, that one is gone too. This applies to `kubectl` and k9s alike.

debug

Many images contain no shell and no tools. `kubectl exec` does not help then. `kubectl debug` attaches an additional container with your tool image:

MAC — Step 6

```
kubectl --context rke2 debug -it <pod> -n dbg-01 \
  --image=nicolaka/netshoot:v0.15 --profile=baseline --target=app
```

Starts an ephemeral container with netshoot in the running pod. It shares the pod's network. With `--target` it also sees the processes of the container `app`, for example with `ps`.

Variant	What for
`kubectl debug <pod> --image=...`	Ephemeral container in the running pod. Shares the network and, with `--target`, the processes too.
`kubectl debug <pod> --copy-to=<new> ...`	Copy of the pod, optionally with a different image (`--set-image`) or a different command. Useful when the original pod crashes immediately.
`kubectl debug node/<node> ...`	Pod with host namespaces on a node, the node's file system is under `/host`.

PITFALL

Without `--profile`, `kubectl debug` uses the `general` profile. It adds the capability `SYS_PTRACE` to the debug container. Pod Security at the `baseline` level does not allow this capability, and Talos enforces `baseline` in every namespace except `kube-system`. There the API server rejects the debug container. With `--profile=baseline` it works on both clusters. `kubectl debug node/`, on the other hand, needs host namespaces and fails on Talos in a normal namespace. Chapter 13 shows how to deal with that.

explain, auth can-i and the API Server

Three commands round out the toolkit. You need them less often, but they answer questions that otherwise take a long time:

MAC — Step 7

```
kubectl --context rke2 explain pod.spec.containers.startupProbe
```

Shows the documentation of a field straight from the API server's schema, matching the cluster's version. Faster than any web search, and it applies to exactly this cluster, CRDs included.

MAC — Step 8

```
kubectl --context rke2 auth can-i list pods -n dbg-01 \
  --as=system:serviceaccount:dbg-01:default
```

Asks whether a specific user or ServiceAccount may perform an action. The answer is `yes` or `no`. With `--list` you see all permissions. More on this in chapter 10.

MAC — Step 9

```
kubectl --context rke2 get --raw '/readyz?verbose'
```

Asks the API server for its internal checks. Every line starts with `[+]` (OK) or `[-]` (failed), and at the end it says `readyz check passed`. If etcd or an informer is stuck here, no amount of searching in your pods will help.

1.6 An Example from Start to Finish

A small case shows the process. An application expects an environment variable `DB_HOST`. If it is missing, the application aborts with a message. The lab from chapter 2 has to be running for this. Read the example now anyway, and replay it as soon as the clusters are up.

FILE szenarien/01-methode/kaputt.yaml (excerpt)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
```

```
  namespace: dbg-01
spec:
  # ...
      containers:
        - name: app
          image: busybox:1.37
          command:
            - sh
            - -c
            - |
              if [ -z "$DB_HOST" ]; then
                echo "FEHLER: DB_HOST ist nicht gesetzt" >&2
                exit 1
              fi
              echo "verbinde mit $DB_HOST"
              sleep 3600
```

The file first creates the namespace `dbg-01`. Pod and container run without root and without capabilities. That way the pod meets the `restricted` level, and Talos does not print warnings when you apply it. The messages are German, as in the whole repo: `FEHLER: DB_HOST ist nicht gesetzt` means “ERROR: DB_HOST is not set”, and `verbinde mit` means “connecting to”.

MAC Step 10

```
kubectl --context rke2 apply -f szenarien/01-methode/kaputt.yaml
```

Creates the namespace and the deployment.

SYMPTOM

```
NAME                   READY   STATUS             RESTARTS      AGE
app-6b8f9c7d54-x7kqp   0/1     CrashLoopBackOff   3 (21s ago)   88s
```

The pod is never ready, the restarts go up, and the intervals between them get longer.

The status `CrashLoopBackOff` only says that the container ends repeatedly and the kubelet is waiting before the next start. Why it ends is written somewhere else. First question: how did the last run end?

MAC Step 11

```
kubectl --context rke2 describe pod -n dbg-01 -l app=app
```

`-l` selects the pod by its label, so you do not have to type the name.

```
    State:          Waiting
      Reason:       CrashLoopBackOff
    Last State:     Terminated
      Reason:       Error
      Exit Code:    1
...
Events:
  Type     Reason   Age                From     Message
  ----     ------   ----               ----     -------
```

```
  Warning  BackOff  12s (x6 over 85s)  kubelet  Back-off restarting failed container
app in pod ...
```

Exit code `1`, reason `Error`, no `OOMKilled`. The image was pulled, the container started. That rules out scheduling, image and runtime. The process itself gave up. Next question: what did it say?

MAC Step 12

```
kubectl --context rke2 logs -n dbg-01 deploy/app --previous
```

The log of the previous run that crashed.

```
FEHLER: DB_HOST ist nicht gesetzt
```

CAUSE

The application checks at startup whether `DB_HOST` is set and otherwise exits with exit code 1. The deployment does not set the variable. Because a deployment has `restartPolicy: Always`, the kubelet starts the container again and again and stretches the pause between attempts up to five minutes.

The fix is the missing variable, not a restart of the pod. The file `fix.yaml` contains the same deployment with an `env` block:

FILE `szenarien/01-methode/fix.yaml (excerpt)`

```
          env:
            - name: DB_HOST
              value: db.dbg-01.svc.cluster.local
```

MAC Step 13

```
kubectl --context rke2 apply -f szenarien/01-methode/fix.yaml
kubectl --context rke2 rollout status deploy/app -n dbg-01
```

The change to the pod template triggers a rollout. `rollout status` waits until the new pod is ready.

WHAT YOU SHOULD SEE

- `rollout status` reports `deployment "app" successfully rolled out.`
- `get pods -n dbg-01` shows a new pod: `1/1`, `Running`, `0` restarts.
- `logs -n dbg-01 deploy/app` prints `verbinde mit db.dbg-01.svc.cluster.local.`

On Talos the example runs the same way. Repeat it with `--context talos` if you like. Then clean up:

MAC Step 14

```
kubectl --context rke2 delete ns dbg-01
kubectl --context talos delete ns dbg-01 --ignore-not-found
```

Deletes the namespace together with the deployment and pods.

The investigation took three commands. Each one answered a question and ruled out a layer. That is exactly how all scenarios in this book work, except that the questions get harder.

TRY IT YOURSELF

In a copy of `kaputt.yaml`, change the command so that the line `slep 3600` runs instead of `exit 1`, and apply it. Which exit code do you expect before you look? Check your guess with `describe` and `logs --previous`. Hint: the shell reports `sh: slep: not found` and ends with 127. Delete the namespace again afterwards.

WHAT YOU LEARNED

- Every failure goes through five steps: reproduce, investigate, understand, fix, verify.
- The map assigns a symptom to a layer: scheduling, image and runtime, workload, network, storage, policy, node, control plane.
- The investigation goes from the outside in: `get`, `describe`, `events`, `logs`, `debug`, then node and control plane.
- `STATUS` is a summary. Pod phase, container state and reason are three different levels, and `lastState` explains the previous run.
- Exit codes above 128 mean a signal. 137 with `OOMKilled` is the memory limit, 137 without it is a hard kill.
- On Talos, `kubectl debug` needs `--profile=baseline`, because the default profile adds a capability that Pod Security `baseline` forbids.

CHAPTER 2

The Lab

Four empty VMs become two clusters: RKE2 on Ubuntu and Talos Linux. Your Mac controls them through the contexts `rke2` and `talos`, both have a StorageClass `local-path`, and k9s is set up with this book's plugins.

Time: 120 minutes **Requires:** Four VMs (two with Ubuntu 24.04 and SSH, two empty), a Mac with Homebrew, the cloned repo

2.1 What You Build

Every failure in this book runs on two clusters. Both use the same Kubernetes version but differ in how they are operated: RKE2 is a package on an ordinary Linux that you log in to via SSH. Talos is an operating system of its own, without a shell and without SSH, that you control only through an API. These differences are exactly what makes debugging interesting.

Node	IP	Context	System	Role
`rke2-cp`	`172.16.0.160`	`rke2`	Ubuntu 24.04	Server: control plane, etcd, also runs workloads
`rke2-w1`	`172.16.0.161`	`rke2`	Ubuntu 24.04	Agent
`talos-cp`	`172.16.0.170`	`talos`	Talos v1.14.5	Control plane, scheduling allowed
`talos-w1`	`172.16.0.171`	`talos`	Talos v1.14.5	Worker

The control plane nodes need 4 CPUs, 8 GB RAM and 60 GB disk, the other two 2 CPUs, 4 GB and 40 GB. Give every VM a fixed IP or a DHCP reservation. The addresses end up in configurations and certificates and must not change later. If you have different addresses, replace them in the files under `labor/` (German for lab) and in the commands.

Both clusters run Kubernetes 1.36.6. What they bring along differs a lot:

	RKE2 v1.36.6+rke2r1	Talos v1.14.5
CNI	Canal (Flannel plus Calico for NetworkPolicies)	Flannel, does **not** enforce NetworkPolicies
Ingress	Traefik (`rke2-traefik`)	none
metrics-server	yes	no

	RKE2 v1.36.6+rke2r1	Talos v1.14.5
StorageClass	none	none
Pod Security	no default	cluster-wide `enforce: baseline`, `warn` and `audit` at `restricted`
Node access	SSH, `journalctl`, `crictl`	only `talosctl` over port 50000

Later chapters add the missing pieces when they are needed. Only the StorageClass is created here.

WHY IT MATTERS

Take a snapshot of all four VMs as soon as you finish this chapter. You will damage the clusters on purpose in the coming chapters. Most of the time you fix the failure yourself afterwards, but if an experiment goes wrong, the snapshot gets you back to the starting point in a minute.

2.2 Tools on the Mac

MAC Step 1

```
brew install kubernetes-cli helm siderolabs/tap/talosctl derailed/k9s/k9s
```

Installs `kubectl`, `helm`, `talosctl` and k9s. Homebrew installs the latest version of each. The book describes kubectl 1.36, talosctl 1.14.5, helm 4 and k9s 0.51.0.

MAC Step 2

```
kubectl version --client
talosctl version --client
k9s version
```

Shows the installed versions.

PITFALL

`talosctl` should match the Talos version of the nodes. A much newer `talosctl` uses `gen config` to create configurations for its own Talos version and puts in the matching installer image. If your version is not 1.14, download the binary for v1.14.5 from the project's release page on GitHub.

MAC Step 3

```
git clone https://github.com/thomaszachmann/kubernetes-debuggen.git
cd kubernetes-debuggen
```

All paths in the book are relative to this directory. You run commands labeled **Mac** here.

2.3 RKE2: The Configuration

At startup RKE2 reads the file `/etc/rancher/rke2/config.yaml`. Every key corresponds to an option of the command `rke2 server` or `rke2 agent`. The files for both nodes are in the repo:

FILE labor/rke2/config-cp.yaml

```yaml
node-name: rke2-cp
node-ip: 172.16.0.160
token: TOKEN-HIER-EINSETZEN
tls-san:
  - 172.16.0.160
  - rke2-cp
write-kubeconfig-mode: "0600"
cni: canal
ingress-controller: traefik
```

`token` is the shared secret the agent uses to identify itself to the server. The placeholder `TOKEN-HIER-EINSETZEN` means "insert token here". `tls-san` adds the IP and name to the API server's certificate, otherwise `kubectl` on the Mac rejects the connection with an `x509` error. `write-kubeconfig-mode` makes the admin kubeconfig readable only by root. `cni` and `ingress-controller` match the defaults of RKE2 1.36. They are listed anyway so the file shows what the lab works with.

FILE labor/rke2/config-w1.yaml

```yaml
server: https://172.16.0.160:9345
node-name: rke2-w1
node-ip: 172.16.0.161
token: TOKEN-HIER-EINSETZEN
```

The agent registers with the server through the supervisor port 9345, not through the API port 6443. Over this connection it gets its certificates and the address of the API server.

The repo is public, so there is no real token in it. You create one now and insert it while copying:

MAC — Step 4

```
RKE2_TOKEN=$(openssl rand -hex 32)
sed "s/TOKEN-HIER-EINSETZEN/$RKE2_TOKEN/" labor/rke2/config-cp.yaml \
  | ssh <user>@172.16.0.160 'umask 077; cat > ~/rke2-config.yaml'
sed "s/TOKEN-HIER-EINSETZEN/$RKE2_TOKEN/" labor/rke2/config-w1.yaml \
  | ssh <user>@172.16.0.161 'umask 077; cat > ~/rke2-config.yaml'
```

Creates a random token and puts each node's configuration into the home directory, readable only by your user. Replace `<user>` with your SSH user. You do not need the token again later. The server also stores it under `/var/lib/rancher/rke2/server/node-token`.

2.4 RKE2: The Server

MAC Step 5

```
ssh <user>@172.16.0.160
```

Opens the session on the server. All steps labeled **rke2-cp** run here.

RKE2-CP Step 6

```
sudo hostnamectl set-hostname rke2-cp
sudo install -D -m 600 -o root -g root \
  ~/rke2-config.yaml /etc/rancher/rke2/config.yaml
rm ~/rke2-config.yaml
```

Sets the hostname to match the `node-name` and puts the configuration in place. It has to be there **before** RKE2 starts for the first time. `-D` creates the directory.

RKE2-CP Step 7

```
curl -sfL https://get.rke2.io | \
  sudo INSTALL_RKE2_VERSION=v1.36.6+rke2r1 sh -
sudo systemctl enable --now rke2-server.service
```

The script installs exactly the version used in this book. Without `INSTALL_RKE2_VERSION` you would get the latest version from the `stable` channel. The first start takes a few minutes because RKE2 pulls the control plane images.

RKE2-CP Step 8

```
sudo journalctl -u rke2-server -f
```

Follows the service log. Stop with `Ctrl-C`. Errors in `config.yaml` show up right at the beginning.

For work directly on the node, later chapters always need the same environment. RKE2 puts `kubectl`, `crictl` and `ctr` in a directory of its own and writes its kubeconfig to a location of its own:

RKE2-CP Step 9

```
sudo -i
export KUBECONFIG=/etc/rancher/rke2/rke2.yaml
export PATH=$PATH:/var/lib/rancher/rke2/bin
export CRI_CONFIG_FILE=/var/lib/rancher/rke2/agent/etc/crictl.yaml
kubectl get pods -n kube-system
```

Switches to root and sets the three variables. Without `CRI_CONFIG_FILE`, `crictl` does not find the containerd socket. Whenever later chapters work on an RKE2 node, they assume exactly these four lines.

WHAT YOU SHOULD SEE

In `kube-system` you see `etcd-rke2-cp`, `kube-apiserver-rke2-cp` and the other static pods of the control plane, plus `rke2-canal`, `rke2-coredns`, `rke2-metrics-server` and `rke2-traefik`. Several `helm-install-rke2-…` pods are `Completed`. That is correct: they installed the bundled charts and are done.

The admin kubeconfig belongs to root. For the Mac you create a copy that your user is allowed to fetch:

RKE2-CP Step 10

```
install -m 600 -o <user> /etc/rancher/rke2/rke2.yaml /tmp/rke2.yaml
exit
exit
```

Still runs as root from the `sudo -i` session. The first `exit` ends the root shell, the second ends the SSH session.

2.5 RKE2: The Agent

MAC Step 11

```
ssh <user>@172.16.0.161
```

Opens the session on the agent.

RKE2-W1 Step 12

```
sudo hostnamectl set-hostname rke2-w1
sudo install -D -m 600 -o root -g root \
  ~/rke2-config.yaml /etc/rancher/rke2/config.yaml
rm ~/rke2-config.yaml
curl -sfL https://get.rke2.io | \
  sudo INSTALL_RKE2_VERSION=v1.36.6+rke2r1 INSTALL_RKE2_TYPE=agent sh -
sudo systemctl enable --now rke2-agent.service
```

Same as on the server, but with `INSTALL_RKE2_TYPE=agent`. The service is now called `rke2-agent`, and you read its log with `sudo journalctl -u rke2-agent -f`.

PITFALL

If the agent hangs and the log keeps reporting connection errors to `172.16.0.160:9345`, a firewall is usually blocking it. On Ubuntu Server `ufw` is inactive by default, and `sudo ufw status` shows it. If it is active, RKE2 needs at least TCP 6443 and 9345 to the server, TCP 10250 between all nodes, and for Canal UDP 8472 (VXLAN) and TCP 9099. Keep port 22 open, or you lock yourself out.

2.6 The RKE2 Kubeconfig

The file from the server points to `127.0.0.1` and calls the cluster, user and context `default`. You change both before it goes into your `~/.kube/config`.

MAC — Step 13

```
mkdir -p ~/.kube
[ -f ~/.kube/config ] && cp -p ~/.kube/config ~/.kube/config.bak
scp <user>@172.16.0.160:/tmp/rke2.yaml ~/.kube/rke2.raw
ssh <user>@172.16.0.160 'rm /tmp/rke2.yaml'
```

Backs up an existing kubeconfig, fetches the file from the server and deletes the copy there. It contains full admin rights.

MAC — Step 14

```
sed -e 's/127\.0\.0\.1/172.16.0.160/' -e 's/: default$/: rke2/' \
  ~/.kube/rke2.raw > ~/.kube/rke2.yaml
KUBECONFIG=~/.kube/config:~/.kube/rke2.yaml \
  kubectl config view --flatten > ~/.kube/config.neu
chmod 600 ~/.kube/config.neu
mv ~/.kube/config.neu ~/.kube/config
rm ~/.kube/rke2.raw ~/.kube/rke2.yaml
```

`sed` puts in the IP and renames everything to `rke2`. `kubectl` merges several files listed in `KUBECONFIG`, and `--flatten` writes the certificates directly into the output. If a context `rke2` already exists, remove it first with `kubectl config delete-context rke2`, otherwise the old entry wins.

MAC — Step 15

```
kubectl --context rke2 get nodes
```

First access from the Mac.

WHAT YOU SHOULD SEE

Two nodes, `rke2-cp` and `rke2-w1`, both `Ready`, version `v1.36.6+rke2r1`. If `rke2-w1` is missing, check the agent's log.

2.7 Talos: Booting

You do not install Talos as a package. You boot it from an ISO. Get the ISO from the Image Factory at `factory.talos.dev`: version v1.14.5, platform bare metal, architecture amd64. You do not need any extensions for the lab. If your hypervisor runs a guest agent, for example Proxmox with `qemu-guest-agent`, you can add the matching extension.

Boot both Talos VMs from this ISO. The console shows the Talos dashboard with the IP address. The node is now in **maintenance mode**: it runs in RAM, has no configuration yet and is waiting

for one. In this state the maintenance API accepts configurations without authentication. That is by design, which is why the lab belongs on a network only you can reach.

MAC Step 16

```
talosctl get disks --insecure --nodes 172.16.0.170
talosctl get disks --insecure --nodes 172.16.0.171
```

Lists the disks of the nodes. `--insecure` talks to the maintenance API. Note the name of the system disk, for example `sda` or `vda`.

Then remove the ISO from the virtual drives. The node keeps running because Talos lives entirely in RAM. This way Talos cannot accidentally boot from the ISO again after the installation.

2.8 Talos: The Patches

A Talos configuration is a long YAML document that `talosctl gen config` creates. You do not edit it by hand. Instead you describe your changes in small patches. There are three in the repo:

FILE labor/talos/patch-cp.yaml

```yaml
machine:
  install:
    disk: /dev/sda
cluster:
  allowSchedulingOnControlPlanes: true
---
apiVersion: v1alpha1
kind: HostnameConfig
hostname: talos-cp
auto: off
```

`machine.install.disk` is the disk Talos installs itself on. If `get disks` showed a different name, adjust it here and in `patch-w1.yaml`. `allowSchedulingOnControlPlanes` removes the taint Talos uses to keep normal pods off the control plane. The second document sets the hostname. `auto: off` turns off the automatically generated name like `talos-2ld-dvw`.

FILE labor/talos/patch-w1.yaml

```yaml
machine:
  install:
    disk: /dev/sda
---
apiVersion: v1alpha1
kind: HostnameConfig
hostname: talos-w1
auto: off
```

FILE `labor/talos/patch-local-path.yaml`

```
apiVersion: v1alpha1
kind: UserVolumeConfig
name: local-path-provisioner
volumeType: directory
```

A **user volume** of type `directory` creates the directory `/var/mnt/local-path-provisioner` on the `EPHEMERAL` partition and passes it through into the kubelet's container. That is where the local-path-provisioner stores the volumes later. A dedicated directory is necessary because the default directory `/opt/local-path-provisioner` is read-only on Talos and the kubelet does not see every path under `/var`.

2.9 Talos: Generating the Configuration

MAC Step 17

```
talosctl gen secrets -o labor/talos/secrets.yaml
```

Creates the certificate authorities and keys of the cluster. With this file you can regenerate the configuration at any time without the certificates changing. Like all generated files, it is listed in `.gitignore`.

MAC Step 18

```
talosctl gen config talos https://172.16.0.170:6443 \
  --with-secrets labor/talos/secrets.yaml \
  --kubernetes-version 1.36.6 \
  --config-patch @labor/talos/patch-local-path.yaml \
  --config-patch-control-plane @labor/talos/patch-cp.yaml \
  --config-patch-worker @labor/talos/patch-w1.yaml \
  --output labor/talos/
```

`talos` is the name of the cluster, the URL is the API server's endpoint. `--config-patch` applies to all nodes, the other two only to the control plane or the worker. Without `--kubernetes-version`, `talosctl` uses its own default, which does not necessarily match the RKE2 cluster.

MAC Step 19

```
talosctl validate --config labor/talos/controlplane.yaml --mode metal
talosctl validate --config labor/talos/worker.yaml --mode metal
```

Checks both files before they go to a node.

WHAT YOU SHOULD SEE

The directory `labor/talos/` now contains `controlplane.yaml`, `worker.yaml` and `talosconfig`. `validate` reports `is valid for metal mode` for both files.

2.10 Talos: Applying and Bootstrapping

MAC Step 20

```
talosctl apply-config --insecure --nodes 172.16.0.170 \
  --file labor/talos/controlplane.yaml
talosctl apply-config --insecure --nodes 172.16.0.171 \
  --file labor/talos/worker.yaml
```

Sends each node its configuration. Talos installs itself to disk and reboots. From now on the node is only reachable with the certificates from `talosconfig`, and `--insecure` no longer works.

MAC Step 21

```
export TALOSCONFIG=$PWD/labor/talos/talosconfig
talosctl config endpoint 172.16.0.170
echo "export TALOSCONFIG=$PWD/labor/talos/talosconfig" >> ~/.zshrc
```

`TALOSCONFIG` tells `talosctl` where the certificates and context are. The endpoint is the node `talosctl` sends all requests through. The last line adds the variable to your shell permanently. All later chapters assume it.

MAC Step 22

```
talosctl --nodes 172.16.0.170 bootstrap
```

Starts etcd on the control plane. You run this command exactly **once** and only on one node. If the node is still waiting for the reboot, repeat it after a minute.

MAC Step 23

```
talosctl --nodes 172.16.0.170 health
```

Checks etcd, the kubelet, the control plane and whether all nodes have joined the cluster. The command waits up to 20 minutes and reports every step.

PITFALL

If you run `bootstrap` a second time on another control plane node, you create a second, independent etcd cluster. The lab has only one control plane node, so the risk is small. In real clusters with three nodes this is a common mistake.

MAC Step 24

```
talosctl --nodes 172.16.0.170 kubeconfig --force-context-name talos
kubectl --context talos get nodes
```

Downloads the admin kubeconfig and merges it into `~/.kube/config`. Without `--force-context-name` the context would be called `admin@talos`.

WHAT YOU SHOULD SEE

Two nodes, `talos-cp` and `talos-w1`, both `Ready`, version `v1.36.6`. `kubectl config get-contexts` shows `rke2` and `talos` side by side.

RKE2

You manage the node like any Linux. The configuration is a file under `/etc/rancher/rke2/`, and changes take effect after `systemctl restart rke2-server`. You read logs with `journalctl`.

TALOS

The node has no shell. The entire configuration is the machine config, which you change with `talosctl patch machineconfig`. You read logs with `talosctl logs kubelet` or `talosctl dmesg`, always with `--nodes`.

2.11 Storage: local-path-provisioner

Neither cluster has a StorageClass yet. A PVC would stay `Pending` forever. Rancher's local-path-provisioner creates a directory for each PVC on the node where the pod runs. For a lab that is exactly right: no network storage, no extra disks.

The StorageClass is called `local-path` on both clusters. It is deliberately **not** made the default StorageClass. A PVC has to request it explicitly with `storageClassName: local-path`. A PVC without it stays `Pending`, and chapters 4 and 9 investigate exactly that failure.

MAC — Step 25

```
kubectl --context rke2 apply -f \
  https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.37/deploy/local-path-storage.
yaml
```

Installs version v0.0.37 in the namespace `local-path-storage`. On Ubuntu the default directory `/opt/local-path-provisioner` is writable, so no adjustment is needed.

Talos needs two changes to the same manifest. A kustomization in the repo applies them:

FILE `werkzeuge/local-path-talos/kustomization.yaml (excerpt)`

```
resources:
  - https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.37/deploy/local-path-storage.
yaml
patches:
  - patch: |-
      kind: ConfigMap
      # ... local-path-config: "paths": ["/var/mnt/local-path-provisioner"]
  - patch: |-
      apiVersion: v1
      kind: Namespace
      metadata:
        name: local-path-storage
```

```
        labels:
          pod-security.kubernetes.io/enforce: privileged
```

The first patch sends the volumes into the user volume from `patch-local-path.yaml`. The second allows privileged pods in the namespace. For every volume the provisioner starts a short-lived helper pod with `hostPath`, and Pod Security `baseline` on Talos would reject it.

MAC Step 26

```
kubectl --context talos apply -k werkzeuge/local-path-talos/
```

`-k` builds the kustomization and applies the result. `kubectl` downloads the manifest directly from GitHub in the process.

MAC Step 27

```
kubectl --context rke2 get storageclass
kubectl --context talos get storageclass
```

Shows the StorageClasses of both clusters.

WHAT YOU SHOULD SEE

Exactly one StorageClass on each cluster:

```
NAME         PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE
local-path   rancher.io/local-path   Delete          WaitForFirstConsumer
```

The output also has the columns `ALLOWVOLUMEEXPANSION` and `AGE`. What matters is the name: there is **no** `(default)` after it.

A small test shows whether a volume is really created: a PVC and a pod that writes a file into it.

MAC Step 28

```
for c in rke2 talos; do
  kubectl --context $c apply -f labor/pvc-test.yaml
  kubectl --context $c wait pod/schreiber -n dbg-02 \
    --for=condition=Ready --timeout=120s
  kubectl --context $c get pvc,pod -n dbg-02 -o wide
done
```

Creates the namespace `dbg-02`, the PVC `daten` and the pod `schreiber` (German for data and writer) on both clusters and waits until the pod is running. Because of `WaitForFirstConsumer`, the volume is only created once the pod is assigned to a node.

WHAT YOU SHOULD SEE

The PVC is `Bound`, the pod is `Running`, and `logs -n dbg-02 schreiber` shows the date and time. On Talos you can also see the volume on the node. Take the node's IP from the `NODE` column:

```
talosctl --nodes 172.16.0.171 ls /var/mnt/local-path-provisioner
```

MAC — Step 29

```
kubectl --context rke2 delete ns dbg-02
kubectl --context talos delete ns dbg-02
```

Cleans up. Because of `reclaimPolicy: Delete`, the provisioner also deletes the directories on the nodes.

2.12 Setting Up k9s

You already installed k9s. What is missing are this book's plugins. k9s looks for them in its configuration directory, and on macOS that is not under `~/.config`:

MAC — Step 30

```
k9s info
```

Shows where k9s expects its configuration, plugins, skins and logs.

WHAT YOU SHOULD SEE

The `Plugins` line names `~/Library/Application Support/k9s/plugins.yaml`, spelled out with your home directory. On Linux it says `~/.config/k9s/plugins.yaml`.

MAC — Step 31

```
mkdir -p "$HOME/Library/Application Support/k9s"
cp k9s/plugins.yaml "$HOME/Library/Application Support/k9s/plugins.yaml"
```

Copies the plugins into place. If you already have a `plugins.yaml` there, merge the two files by hand instead of overwriting it. Chapter 3 shows what the plugins do.

2.13 Wrap-Up: Both Clusters at a Glance

MAC — Step 32

```
kubectl --context rke2 get nodes -o wide
kubectl --context talos get nodes -o wide
```

The end of the chapter. These two commands are also where every investigation starts when you do not know what state the lab is in.

WHAT YOU SHOULD SEE

Four nodes, all `Ready`. For RKE2, `VERSION` shows `v1.36.6+rke2r1` and `OS-IMAGE` Ubuntu 24.04, for Talos `v1.36.6` and `Talos (v1.14.5)`. The `INTERNAL-IP` matches the table at the beginning of the chapter. Now is the right moment for the snapshots.

TRY IT YOURSELF

Compare what runs in `kube-system` on the two clusters: `kubectl --context rke2 get pods -n kube-system` and the same for `talos`. Which pod is responsible for DNS on which cluster, and what is it called? Which one for the pod network? Hint: on RKE2 the bundled components carry the prefix `rke2-`, on Talos they are simply called `coredns` and `kube-flannel`. Chapter 7 needs exactly these names.

WHAT YOU LEARNED

- RKE2 reads `/etc/rancher/rke2/config.yaml`. Server and agent share a token, and the agent registers through port 9345.
- Talos is controlled through a machine config that `talosctl gen config` creates from secrets and small patches. `bootstrap` runs exactly once.
- The contexts are called `rke2` and `talos`, and `TALOSCONFIG` points to `labor/talos/talosconfig`.
- Both clusters have the StorageClass `local-path`, but no default StorageClass. On Talos the volumes are stored under `/var/mnt/local-path-provisioner`, and the provisioner's namespace needs Pod Security `privileged`.
- On macOS, k9s expects its plugins under `~/Library/Application Support/k9s/`. `k9s info` shows the exact path.

CHAPTER 3

k9s as a Cockpit

You move through both clusters in k9s without a mouse, filter, read logs including the previous run, open shells and port forwards and start a debug container with a single keystroke. You also know when `kubectl` is the better choice.

Time: 45 minutes **Requires:** Chapter 2

3.1 Why a Second Tool

`kubectl` shows you a state at one point in time. For troubleshooting that is often not enough. You want to watch a pod restart or a rollout progress, and jump quickly between pod, logs, events and deployment while you do. That is exactly what k9s is built for: a terminal UI that refreshes every two seconds and puts every view a few keys away.

k9s talks to the same API as `kubectl` and uses the same kubeconfig. So it sees nothing that `kubectl` could not see, and it may do nothing your user is not allowed to do. Its only advantage is speed. That is why this book shows both paths in every scenario: `kubectl` for commands you want to document and repeat, k9s for a quick round through the cluster.

To have something to look at, first create a few practice objects:

FILE `szenarien/03-k9s/demo.yaml (excerpt)`

```
kind: Deployment
metadata:
  name: web
  namespace: dbg-03
spec:
  replicas: 2
  # ... busybox httpd on port 8080, plus Service web (80 -> 8080)
---
kind: Deployment
metadata:
  name: wackel
  namespace: dbg-03
# ...
          command:
            - sh
            - -c
            - |
              echo "start"
              sleep 15
              echo "FEHLER: Cache voll, gebe auf" >&2
              exit 3
```

`web` is a small web server with two replicas and a service. `wackel` (German for wobbly) crashes after 15 seconds with exit code 3 and ends up in `CrashLoopBackOff` after a few rounds. Its message means “ERROR: cache full, giving up”. Both run without root so that Talos does not print warnings.

MAC Step 1

```
kubectl --context rke2 apply -f szenarien/03-k9s/demo.yaml
```

Creates the namespace `dbg-03`, both deployments and the service.

3.2 Starting and Finding Your Way

MAC Step 2

```
k9s --context rke2 -n dbg-03
```

Starts k9s in the context `rke2` and directly in the namespace `dbg-03`. Without `-n`, k9s opens the namespace you last used in this context. `-A` starts in all namespaces, `-c deploy` with a view other than pods.

The screen has three areas. At the top left are the context, cluster, user and the versions of k9s and Kubernetes, and next to them the keys that apply in the current view. The middle shows the table, with a title above it listing the resource, namespace and number of objects. At the bottom, the **crumbs** show how you got here.

PITFALL

Before every action that changes something, look at the context at the top left. If you work with k9s in two terminals, one per cluster, it is easy to delete in the wrong one. One aid: k9s can load its own skin per context, for example red for the lab you are breaking right now.

You only need a few keys. `?` shows all keys of the current view. That is the most reliable source, because the key bindings depend on the resource and the version.

Key	Effect
`:`	Command line: open a resource, for example `:deploy⏎`, `:svc⏎`, `:events⏎`
`/`	Filter the current table
`Esc`	Cancel the filter or command line, go back one view
`Enter`	Open an object: a pod shows its containers, a deployment shows its pods
`?`	Help for the current view
`Ctrl-a`	All resources and their aliases

Key	Effect
`-`	Back to the previous command, like `cd -`
`:q⏎` or `Ctrl-c`	Quit k9s

3.3 Resources, Namespaces and Filters

After `:`, k9s accepts any name that `kubectl` also understands: singular, plural and short name. `:po`, `:pod` and `:pods` open the same view. CRDs work too, as soon as they are in the cluster, for example `:policyreports` in chapter 11.

K9S Step 3

```
:deploy dbg-03⏎
:pods kube-system⏎
:pods app=web⏎
:pods /wackel⏎
```

Open a resource with a namespace, show pods by a label selector and filter the list right when you open it.

There are three ways to switch the namespace. `0` shows all namespaces. The digits `1` to `9` jump to recently used namespaces, which k9s shows at the top in the key list. And `:ns⏎` lists all namespaces, where `u` makes the selected one the active one.

The `/` filter is the tool you will use most:

Input	Effect
`/web⏎`	Only rows that contain `web`. Regular expressions are allowed, for example `web|wackel`.
`/!Running⏎`	Inverted: all rows that do **not** contain `Running`
`/-l app=web⏎`	Filter by labels
`/-f wkl⏎`	Fuzzy search
`Ctrl-z`	Show only objects with faults, a second `Ctrl-z` switches back

K9S Step 4

```
:pods⏎
0
/!Running|Completed⏎
```

All pods in all namespaces that are neither running nor completed. A good first round when you do not know what is broken in a cluster.

You sort with capital letters: `Shift-s` by status, `Shift-a` by age, `Shift-n` by name. `Ctrl-w` shows additional columns, for pods for example IP and node. That corresponds to `kubectl get -o wide`.

3.4 A Pod in Detail

In the pod view, select a pod of `wackel`. All of the following keys act on the selected row:

Key	Effect	kubectl equivalent
`d`	Describe	`describe pod`
`y`	YAML of the object including `status`	`get -o yaml`
`e`	Edit in the editor	`edit`
`l`	Logs	`logs -f`
`p`	Logs of the previous run	`logs --previous`
`s`	Shell in the container	`exec -it ... -- sh`
`a`	Attach to the container	`attach`
`o`	Jump to the pod's node	`get node`
`Shift-j`	Jump to the owner, for example the ReplicaSet	—
`Shift-f`	Set up a port forward	`port-forward`
`Ctrl-d`	Delete, with confirmation	`delete`
`Ctrl-k`	Delete immediately, **without** confirmation	`delete --now`

K9S Step 5

```
:pods dbg-03⏎
/wackel⏎
d
```

Describe of the crashing pod. You scroll with the arrow keys, and `Esc` takes you back to the list.

WHAT YOU SHOULD SEE

Under `Last State` it says `Terminated`, `Reason: Error`, `Exit Code: 3`. At the very bottom the event `BackOff` repeats with `Back-off restarting failed container wackel`.

`Enter` on a pod opens the container view. There, `l`, `p` and `s` apply to exactly one container. You need this for pods with sidecars or init containers. In the pod view, k9s otherwise picks the first container or asks you.

PITFALL

`s` starts `bash` if the image has one, otherwise `sh`. If the image has no shell at all, as with distroless images, this fails with an error message from `exec`. Then only a debug container helps. The plugin on `Shift-d`, described further down, is there for exactly that.

3.5 Reading Logs

K9S Step 6

```
:pods dbg-03⏎
/wackel⏎
p
```

Shows the log of the previous run. It contains the last line before the crash: `FEHLER: Cache voll, gebe auf`.

The log view has keys of its own. They differ from the table, for example `w` here means line wrap and not namespace switch:

Key	Effect
`0`	Follow from now on (tail)
`1`	From the beginning (head)
`2` to `6`	Time window: 1, 5, 15, 30 minutes, 1 hour
`s`	Autoscroll on or off
`w`	Wrap long lines
`t`	Show timestamps
`f`	Full screen
`/`	Filter the log lines
`Ctrl-s`	Save the log to a file

Timestamps are almost always helpful when debugging. They show whether a failure just happened or has been going on for hours. If you always want to see them, set `showTime: true` under `k9s.logger` in the k9s `config.yaml`. There you also set how many lines k9s loads when it opens the log with `tail`, and line wrapping with `textWrap`.

3.6 Events, XRay and Pulses

K9S Step 7

```
:events dbg-03⏎
```

Events of the namespace as a table that keeps updating. `Ctrl-z` shows only warnings and errors.

Two views exist only in k9s. **XRay** shows objects as a tree: deployment, below it the ReplicaSet, below that the pods with their containers, ConfigMaps, Secrets and the ServiceAccount. A red element in the tree takes you straight to the cause. This helps especially when a deployment was accepted but no pods get created.

K9S Step 8

```
:xray deploy dbg-03⏎
```

Tree of all deployments in `dbg-03`. Allowed are, among others, `po`, `svc`, `dp`, `rs`, `sts` and `ds`.

Pulses (`:pulses⏎`) shows small trend charts for nodes, namespaces, services, events, pods, deployments, StatefulSets, DaemonSets, jobs, PVCs and other objects, each with the number of healthy and faulty ones, plus CPU and memory if a metrics-server is running. It is not a tool for a single cause, but a good way to see whether a rollout is making waves right now.

Older guides mention `:popeye` for checking the whole cluster. k9s 0.51 no longer includes this view. If you want to use Popeye, run it as a separate program.

3.7 Port Forward

K9S Step 9

```
:svc dbg-03⏎
/web⏎
Shift-f
```

Opens the dialog for a port forward to the service `web`. Enter `8080` under `Local Port` and confirm with `OK`.

MAC Step 10

```
curl -s localhost:8080
```

A second terminal queries the web server through the port forward.

WHAT YOU SHOULD SEE

The answer is `hallo aus web-…` (“hello from web-…”) with the name of one of the two pods. The `f` key in the service or pod view shows all active port forwards, and so does `:pf⏎`. There you end one with `Ctrl-d`. Port forwards only live as long as k9s does.

3.8 Contexts and Read-Only

`:ctx⏎` lists all contexts from your kubeconfig, and `Enter` switches. `:ctx talos⏎` is faster. k9s remembers the namespace and view separately for each context.

For this book I still recommend two terminals with one k9s each: `k9s --context rke2` and `k9s --context talos`. Then you see both clusters side by side and never have to think about where you are.

If you only want to look around in someone else's cluster or a production cluster, start k9s read-only:

MAC Step 11

```
k9s --context talos --readonly
```

Turns off all actions that change something: edit, delete, kill, shell, attach and all plugins marked as `dangerous`. To make it permanent, set `readOnly: true` in `config.yaml`, and `--write` lifts it again for a single start.

WHY IT MATTERS

Read-only does not protect against curiosity, it protects against typos. `Ctrl-k` deletes without asking. In a cluster where you are only looking for failures, there is no reason to take that risk.

3.9 This Book's Plugins

Plugins bind your own commands to a key. k9s replaces variables like `$NAME`, `$NAMESPACE`, `$CONTEXT` and, in the container view, `$POD` with the values of the selected row. You copied the file `k9s/plugins.yaml` to `~/Library/Application Support/k9s/` in chapter 2. It defines three plugins:

Key	View	Effect
`Shift-d`	Pods	Plugin `netshoot`: starts `nicolaka/netshoot:v0.15` as an ephemeral container in the selected pod. Shares the pod's network.
`Shift-d`	Containers	Plugin `netshoot-target`: as above, but also with `--target` on the se-

Key	View	Effect
		lected container. Sees its processes.
`Shift-e`	Pods, deployments, ReplicaSets, StatefulSets, DaemonSets, jobs, services, PVCs	Plugin `events`: `kubectl events --for` exactly this object, in chronological order, in `less`. Quit with `q`.

FILE `k9s/plugins.yaml (excerpt)`

```
plugins:
  netshoot:
    shortCut: Shift-D
    description: netshoot debug
    scopes:
      - pods
    dangerous: true
    confirm: true
    background: false
    command: kubectl
    args:
      - debug
      - -it
      - --context
      - $CONTEXT
      - -n
      - $NAMESPACE
      - $NAME
      - --image=nicolaka/netshoot:v0.15
      - --profile=baseline
  # ... netshoot-target (containers) and events
```

`scopes` defines the views in which the key applies. `confirm: true` shows the finished command before it starts and asks for confirmation. `dangerous: true` hides the plugin in read-only mode, because an ephemeral container changes the pod permanently. `--profile=baseline` is needed because Talos otherwise rejects the debug container (chapter 1). `background: false` hands the terminal over to `kubectl` until you leave the shell with `exit`.

The keys are chosen so that they do not collide with any k9s 0.51 binding. That matters: if a plugin uses a key that k9s already uses in that view, k9s does not load the plugin, shows `Plugins load failed!` at the top right and writes the reason (`duplicate plugin key`) to the log. Only with `override: true` does the plugin replace the built-in function. If a plugin is missing, use `?` to check whether the key appears, and look in the log file whose path `k9s info` shows.

K9S Step 12

```
:pods dbg-03⏎
/web⏎
Shift-d
```

Select one of the `web` pods and press `Shift-d`. Confirm the command shown. After a few seconds you land in a netshoot shell.

POD Step 13

```
ss -ltn
curl -s localhost:8080
exit
```

`ss` shows the web server listening on port 8080 in the **other** container. Both share the pod's network, which is why `curl` reaches it through `localhost`. With `exit` you return to k9s.

WHAT YOU SHOULD SEE

`ss -ltn` lists a line with `*:8080` in the state `LISTEN`, and `curl` returns `hallo aus web-…`. A `describe` of the pod (`d`) then shows a section `Ephemeral Containers` with the debug container in the state `Terminated`. Ephemeral containers cannot be removed. They only go away with the pod.

Finally, try `Shift-e` on the deployment `wackel` in the `:deploy⏎` view. You see the events of the deployment itself, meaning the scaling of the ReplicaSet, not those of the pods. On the pod, the same key shows the `BackOff` events. This separation is intentional and helps you in chapters 10 and 11, when pods are not even created.

3.10 Where kubectl Is Better

k9s does not replace `kubectl`. For some tasks the command line is clearly superior:

- **Documenting and sharing.** A command with its output can be copied into a ticket and repeated by others. A sequence of keystrokes cannot.
- **Scripts and loops.** Anything you repeat across several clusters or namespaces belongs in `kubectl` with `-o jsonpath` or `-o json`.
- **Questions to the API.** `kubectl explain`, `kubectl auth can-i --as`, `kubectl get --raw '/readyz?verbose'` and `kubectl debug --copy-to` have no counterpart in k9s.
- **State after a restart.** Like `--previous`, `p` shows exactly one previous run. Combinations like `--previous --all-containers --since=10m` only work with `kubectl`.
- **Short events.** k9s refreshes in two-second intervals. A pod that exists for only one second can disappear in between. `kubectl get pods -w` and events show every change.

And k9s only sees what your user is allowed to see. If a view stays empty even though `kubectl` with admin rights shows objects, the cause is RBAC, not k9s.

MAC Step 14

```
kubectl --context rke2 delete ns dbg-03
```

Cleans up. In k9s you would do this with `:ns⏎`, selecting the namespace and pressing `Ctrl-d`.

TRY IT YOURSELF

Apply `szenarien/03-k9s/demo.yaml` to `talos` and work only in `k9s --context talos`. Find out which node the `wackel` pod runs on (`o`), how often it has restarted already, and read the last log line before the crash. Then start `Shift-d` on a `web` pod and use `dig web.dbg-03.svc.cluster.local` to check whether the service name resolves. Hint: without `--profile=baseline` in the plugin, Talos would reject the debug container with `violates PodSecurity "baseline:latest"`. Delete the namespace afterwards.

WHAT YOU LEARNED

- k9s uses the same API and kubeconfig as `kubectl`. Its advantage is speed, not extra knowledge.
- `:` opens resources, `/` filters (also `/!`, `/-l`), `0` shows all namespaces, `Ctrl-z` only faults, `?` the keys of the view.
- `d`, `y`, `l` and `p` cover most investigations. `p` corresponds to `kubectl logs --previous`.
- `--readonly` turns off actions that change something and dangerous plugins.
- This book's plugins: `Shift-d` starts netshoot in the pod or, with `--target`, in the container, and `Shift-e` shows the events of an object.
- For documentation, scripts and API questions like `auth can-i`, `kubectl` is the better choice.

CHAPTER 4

Pods Stay Pending

You read the scheduler's messages, recognize whether a pod fails because of resources, labels, taints, volumes or a quota, and you fix each of these errors on RKE2 and Talos.

Time: 60 minutes **Requires:** Chapters 2 and 3

4.1 What Breaks in This Chapter

A pod in `Pending` does not have a node yet. No container runs, there are no logs, and `kubectl logs` stays empty. The answer is almost always in one of two places: in the pod's events, written by the scheduler, or, if the pod does not exist at all, in the events of the ReplicaSet.

Scenario	Symptom	Where the answer is
Requests too large	Pod `Pending`, event `FailedScheduling`	`Insufficient cpu` in the pod event
nodeSelector and taint	Pod `Pending`	`didn't match Pod's node affinity/selector`, `had untolerated taint(s)`
PVC without StorageClass	Pod and PVC `Pending`	`unbound immediate PersistentVolumeClaims`, events of the PVC
ResourceQuota	Deployment `0/2`, **no** pod	Event `FailedCreate` on the ReplicaSet

All scenarios run in the namespace `dbg-04`, which every `kaputt-*.yaml` creates itself (`kaputt` means "broken"). The quota from the last scenario applies to the whole namespace, so apply it last. On Talos, `kubectl apply` reports `Warning: would violate PodSecurity "restricted:latest": ...` for these manifests. Talos enforces only `baseline` cluster-wide and warns about `restricted`. The warning blocks nothing. There is more on this in Chapter 10.

4.2 The Scheduler Explains Itself

If the scheduler finds no node, it writes an event with the reason `FailedScheduling`. The text always follows the pattern `0/2 nodes are available: <reasons per node>. preemption: <result>`. The first part counts, for each reason, on how many nodes it applied. `2 Insufficient cpu` means: both nodes have too little free CPU. If there are several reasons, the numbers add up to the number of nodes, because the scheduler reports the first filter that fails for each node. The second part says whether the scheduler could evict other pods to make room. `Preemption is not helpful for scheduling` means:

evicting does not solve this problem, for example because a label is missing. Then you have to change something yourself.

4.3 Requests Too Large

Requests are reservations. The scheduler does not calculate with the actual load of a node, but with the sum of the requests of all pods on it. A pod that asks for more than a node can allocate never finds a place, even if the cluster is doing nothing at the moment.

FILE szenarien/04-pending/kaputt-ressourcen.yaml (excerpt)

```
      containers:
        - name: app
          image: busybox:1.37
          command: ["sleep", "infinity"]
          resources:
            requests:
              cpu: "6"
              memory: 256Mi
```

Six CPU cores as a request. The largest node in the lab has four.

MAC Step 1

```
kubectl --context rke2 apply -f szenarien/04-pending/kaputt-ressourcen.yaml
kubectl --context talos apply -f szenarien/04-pending/kaputt-ressourcen.yaml
```

Creates the namespace and the Deployment `gross` (“big”) on both clusters.

SYMPTOM

```
$ kubectl --context rke2 get pods -n dbg-04 -o wide
NAME                     READY   STATUS    RESTARTS   AGE   IP       NODE     NOMINATED
NODE   READINESS GATES
gross-6d4b9c7f8d-q2xkp   0/1     Pending   0          48s   <none>   <none>   <none>
<none>
```

No node, no IP, no restarts. The pod waits, and it stays that way.

Investigate

MAC Step 2

```
kubectl --context rke2 describe pod -n dbg-04 -l app=gross
```

`describe` with a label selector saves you from typing the pod name. The important part is the `Events` block at the very bottom.

```
Events:
  Type     Reason            Age   From               Message
  ----     ------            ----  ----               -------
  Warning  FailedScheduling  51s   default-scheduler  0/2 nodes are available: 2
```

```
Insufficient cpu.
  preemption: 0/2 nodes are available: 2 No preemption victims found for incoming pod.
```

Both nodes have too little CPU, and evicting other pods would not help either. `describe node` shows what a node can hand out (`Allocatable`) and how much of it is already taken:

MAC — Step 3

```
kubectl --context rke2 describe node rke2-cp | grep -A 8 "Allocated resources"
```

The `Requests` column is what counts, not `Limits`. Limits may add up to more than 100 percent, requests may not. You see the `Allocatable` block further up in the same output without `grep`.

```
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource           Requests     Limits
  --------           --------     ------
  cpu                1150m (28%)  200m (5%)
  memory             1004Mi (12%) 662Mi (8%)
  ephemeral-storage  0 (0%)       0 (0%)
```

In k9s you reach the same goal the same way, only faster between the views:

K9S — Step 4

```
:pods dbg-04⏎   select pod, d   (describe, events at the bottom)
                or Shift-E      (plugin: kubectl events --for)
:nodes⏎         select node, d
```

`Esc` takes you back to the list, and `/` filters the describe view for `FailedScheduling`. `Shift-E` is the plugin from Chapter 3.

CAUSE

The pod requests 6 CPUs, and no node can allocate more than 4. For every node, the scheduler checks whether `allocatable` minus the sum of the existing requests is enough for the new request. This is pure bookkeeping: whether the CPU is busy right now does not matter. A request larger than any single node therefore stays `Pending` forever.

Fix

Realistic requests are based on measured usage. In the lab, a quarter of a core is enough: `fix-ressourcen.yaml` sets `requests.cpu: 250m`, everything else is the same.

MAC — Step 5

```
kubectl --context rke2 apply -f szenarien/04-pending/fix-ressourcen.yaml
kubectl --context talos apply -f szenarien/04-pending/fix-ressourcen.yaml
```

The Deployment creates a new ReplicaSet, and the old pending pod disappears.

WHAT YOU SHOULD SEE

```
$ kubectl --context rke2 get pods -n dbg-04 -l app=gross
NAME                     READY   STATUS    RESTARTS   AGE
gross-7f5c8b9d64-mz7wd   1/1     Running   0          9s
```

The event history (`kubectl --context rke2 events -n dbg-04 --for pod/<name>`) now shows `Successfully assigned dbg-04/... to rke2-cp` or `to rke2-w1`.

WHY IT MATTERS

Partial success is more dangerous: three replicas of 1500m each fit only twice in the lab. The third pod stays `Pending`, and the Deployment shows a half-healthy `2/3`.

4.4 nodeSelector and Taint

A `nodeSelector` is a hard condition: the pod may only go to nodes that carry all the listed labels. A taint is the counterpart: the node rejects pods that do not explicitly tolerate it. Typical examples are nodes for special tasks, nodes in maintenance and control plane nodes. In the lab, you turn `rke2-w1` and `talos-w1` into a batch node with a label **and** a taint.

MAC — Step 6

```
kubectl --context rke2 label node rke2-w1 dbg-pool=batch
kubectl --context rke2 taint node rke2-w1 dbg-pool=batch:NoSchedule
kubectl --context talos label node talos-w1 dbg-pool=batch
kubectl --context talos taint node talos-w1 dbg-pool=batch:NoSchedule
```

`NoSchedule` only prevents new pods. What is already running on the node stays. `NoExecute` would also evict running pods without a toleration.

FILE `szenarien/04-pending/kaputt-taint.yaml (excerpt)`

```
  spec:
    nodeSelector:
      dbg-pool: batch
    containers:
      - name: app
        image: busybox:1.37
```

The pod wants to go to the batch node, but it has no toleration.

MAC — Step 7

```
kubectl --context rke2 apply -f szenarien/04-pending/kaputt-taint.yaml
kubectl --context talos apply -f szenarien/04-pending/kaputt-taint.yaml
```

Creates the Deployment `batch`.

SYMPTOM

The pod `batch-...` is `Pending`, even though there is a node with the matching label.

Investigate

MAC — Step 8

```
kubectl --context rke2 describe pod -n dbg-04 -l app=batch | tail -n 4
```

The last lines of `describe` are the events.

```
  Warning  FailedScheduling  20s   default-scheduler  0/2 nodes are available: 1
node(s) didn't
  match Pod's node affinity/selector, 1 node(s) had untolerated taint(s). preemption:
0/2 nodes
  are available: 2 Preemption is not helpful for scheduling.
```

Two reasons, and their numbers add up to the two nodes: `rke2-cp` fails on the selector, `rke2-w1` on the taint. If no node had the label at all, it would say `2 node(s) didn't match Pod's node affinity/selector`. The `NodeAffinity` filter checks `nodeSelector` and `requiredDuringScheduling...` together, hence the double name. The message does not tell you which label and which taint it means. Older versions still named the taint, Kubernetes 1.36 no longer does. You compare yourself:

MAC — Step 9

```
kubectl --context rke2 get nodes \
  -o custom-columns=NAME:.metadata.name,POOL:.metadata.labels.dbg-pool,TAINTS:.spec.taints
```

Shows the label and the taints for each node. `<none>` means: not present. What the pod requires and tolerates is in `describe pod` under `Node-Selectors` and `Tolerations`.

```
NAME      POOL    TAINTS
rke2-cp   <none>  <none>
rke2-w1   batch   [map[effect:NoSchedule key:dbg-pool value:batch]]
```

The pod only has the two tolerations `node.kubernetes.io/not-ready` and `unreachable` for 300 seconds, which every pod gets automatically. `dbg-pool` is not among them. In k9s you see the taints with `:nodes⏎`, select the node, press `y` for the YAML, and look under `spec.taints`.

CAUSE

The selector sends the pod to exactly one node, and that node rejects it with a taint. Selector and toleration answer different questions: the selector says where the pod **wants** to go, the toleration says where it **may** go. A special node needs both. Otherwise either foreign pods end up on it, or your own pods do not.

Fix

`fix-taint.yaml` adds the matching toleration to the pod template:

```
tolerations:
  - {key: dbg-pool, operator: Equal, value: batch, effect: NoSchedule}
```

MAC — Step 10

```
kubectl --context rke2 apply -f szenarien/04-pending/fix-taint.yaml
kubectl --context talos apply -f szenarien/04-pending/fix-taint.yaml
```

The new ReplicaSet creates a pod with the toleration.

WHAT YOU SHOULD SEE

`kubectl --context rke2 get pods -n dbg-04 -l app=batch -o wide` shows `Running` on `rke2-w1`, and on Talos on `talos-w1`.

Remove the taint and the label afterward. Otherwise future pods will no longer land on the worker:

MAC — Step 11

```
kubectl --context rke2 taint node rke2-w1 dbg-pool=batch:NoSchedule-
kubectl --context rke2 label node rke2-w1 dbg-pool-
kubectl --context talos taint node talos-w1 dbg-pool=batch:NoSchedule-
kubectl --context talos label node talos-w1 dbg-pool-
```

The trailing minus removes the taint or the label.

RKE2

In the lab, the RKE2 server accepts workloads and has no taint. If you want to keep server nodes free, set for example `node-taint: ["CriticalAddonsOnly=true:NoExecute"]` in `/etc/rancher/rke2/config.yaml`. Permanent labels go there under `node-label:`.

TALOS

Without `cluster.allowSchedulingOnControlPlanes: true`, `talos-cp` carries the taint `node-role.kubernetes.io/control-plane:NoSchedule`. In a cluster with only one worker, every pod then lands there, and if the worker is full or gone, everything stays `Pending` with `had untolerated taint(s)`. In the lab, scheduling on the control plane is allowed. Permanent labels go in `machine.nodeLabels`.

4.5 PVC Without StorageClass

A pod with a PersistentVolumeClaim can only start when the claim is bound to a volume or can be bound. In the lab, both clusters have the StorageClass `local-path`, but on purpose it is **not** the default. A PVC without `storageClassName` therefore gets no class, and nobody creates a volume.

FILE szenarien/04-pending/kaputt-pvc.yaml (excerpt)

```yaml
kind: PersistentVolumeClaim
metadata:
  name: daten
  namespace: dbg-04
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 1Gi
# ... Deployment "datenbank" mounts the PVC at /daten
```

No `storageClassName`. You see this in many Helm charts that rely on a default StorageClass.

MAC — Step 12

```
kubectl --context rke2 apply -f szenarien/04-pending/kaputt-pvc.yaml
kubectl --context talos apply -f szenarien/04-pending/kaputt-pvc.yaml
```

Creates the PVC `daten` (“data”) and the Deployment `datenbank` (“database”).

SYMPTOM

```
$ kubectl --context rke2 get pvc -n dbg-04
NAME    STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
VOLUMEATTRIBUTESCLASS   AGE
daten   Pending                                                  <unset>
31s
```

The PVC is `Pending`, and the `STORAGECLASS` column is empty. The pod `datenbank-...` is `Pending` too.

Investigate

MAC — Step 13

```
kubectl --context rke2 describe pod -n dbg-04 -l app=datenbank | tail -n 3
kubectl --context rke2 describe pvc -n dbg-04 daten | tail -n 4
```

First the pod, then the PVC. The real explanation is on the PVC.

```
  Warning  FailedScheduling  40s   default-scheduler  0/2 nodes are available: pod has
unbound
  immediate PersistentVolumeClaims. preemption: 0/2 nodes are available: 2 Preemption
is not
  helpful for scheduling.

  Type     Reason         Age              From                         Message
  ----     ------         ----             ----                         -------
```

```
  Normal  FailedBinding  5s (x4 over 40s) persistentvolume-controller  no persistent
volumes
  available for this claim and no storage class is set
```

Notice the shape of the scheduler message: no count per node, just a single sentence. Volume binding fails before the individual nodes are even checked. And the PVC message is of type `Normal`, not `Warning`. If you filter with `--types=Warning`, you miss it.

`kubectl --context rke2 get storageclass` shows `local-path` without the `(default)` suffix after the name. In k9s: `:pvc dbg-04⏎`, select the PVC, press `d`. The `STORAGECLASS` column is already empty in the list.

CAUSE

Without `storageClassName` and without a default StorageClass, there is nobody to provision a volume. The PV controller looks for an existing, matching PersistentVolume without a class, finds none and waits. The scheduler, in turn, does not place a pod with an unbound PVC in `Immediate` mode on a node. Both wait for each other, forever.

Fix

The PVC names the class explicitly. You can only change the spec of an existing PVC in limited ways, so you delete the Deployment and the PVC and create both again:

FILE `szenarien/04-pending/fix-pvc.yaml (excerpt)`

```yaml
spec:
  storageClassName: local-path
  accessModes: ["ReadWriteOnce"]
```

MAC — Step 14

```
kubectl --context rke2 -n dbg-04 delete deployment datenbank
kubectl --context rke2 -n dbg-04 delete pvc daten
kubectl --context rke2 apply -f szenarien/04-pending/fix-pvc.yaml
```

On Talos, run the same three commands with `--context talos`. Deleting the PVC first does not help: as long as a pod uses it, the finalizer `kubernetes.io/pvc-protection` keeps it in `Terminating`.

WHAT YOU SHOULD SEE

```
$ kubectl --context rke2 get pvc -n dbg-04
NAME    STATUS   VOLUME                                     CAPACITY   ACCESS MODES
STORAGECLASS   ...
daten   Bound    pvc-3c1e6a0b-8f5d-4c47-9b1e-2d7a4f0c9e18   1Gi        RWO
local-path     ...
```

Right after it is created, the PVC is briefly `Pending` with the event `WaitForFirstConsumer`: `local-path` only creates the volume once the pod has a node. That is not an error.

PITFALL

Alternatively, you make `local-path` the default class with the annotation `storageclass.kubernetes.io/is-default-class: "true"`. Since Kubernetes 1.28, existing PVCs without a class then get this class retroactively as well. In the lab, it deliberately stays without a default.

RKE2

RKE2 does not ship a StorageClass. `local-path` from Chapter 2 stores the data on the node under `/opt/local-path-provisioner`.

TALOS

Talos does not ship a StorageClass either. `local-path` writes under `/var/mnt/local-path-provisioner`, and the helper pod runs in the namespace `local-path-storage`, which is why that namespace carries `enforce=privileged`. If the label is missing, the helper pod fails, and the PVC stays `Pending` despite the class. Chapter 9 covers this case.

4.6 ResourceQuota: The Pod That Does Not Exist

The first three scenarios had a pod in `Pending`. This one has no pod at all. A ResourceQuota limits what a namespace may request in total. As soon as it limits CPU or memory, **every** container must also specify requests and limits for these resources. Quota admission checks this in the API server when the pod is created, and anything that does not comply is rejected.

Before you start, delete the Deployments of the other scenarios with `kubectl --context rke2 -n dbg-04 delete deployment --all` (and the same on Talos), so the numbers below stay easy to read.

FILE `szenarien/04-pending/kaputt-quota.yaml (excerpt)`

```
kind: ResourceQuota
metadata:
```

```
  name: dbg-quota
  namespace: dbg-04
spec:
  hard:
    pods: "10"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
---
# ... Deployment "web" with nginx:1.29, 2 replicas, no resources
```

MAC Step 15

```
kubectl --context rke2 apply -f szenarien/04-pending/kaputt-quota.yaml
kubectl --context talos apply -f szenarien/04-pending/kaputt-quota.yaml
```

`apply` reports `deployment.apps/web created`. No error, no warning.

SYMPTOM

```
$ kubectl --context rke2 get deploy,pods -n dbg-04 -l app=web
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/web   0/2     0            0           25s
```

The Deployment exists, but there is not a single pod, not even one in `Pending`.

Investigate

The chain is Deployment → ReplicaSet → Pod. If the last link is missing, the error is in the second-to-last:

MAC Step 16

```
kubectl --context rke2 get rs -n dbg-04 -l app=web
kubectl --context rke2 describe rs -n dbg-04 -l app=web | tail -n 4
```

The ReplicaSet wants two pods (`DESIRED`) but has none (`CURRENT`). Its events say why.

```
NAME             DESIRED   CURRENT   READY   AGE
web-5d8c6b7f9c   2         0         0       31s

  Warning  FailedCreate  12s (x5 over 31s)  replicaset-controller  Error creating:
pods
  "web-5d8c6b7f9c-x8k2p" is forbidden: failed quota: dbg-quota: must specify
limits.cpu for: web;
  limits.memory for: web; requests.cpu for: web; requests.memory for: web
```

The Deployment knows about the problem too: `kubectl describe deploy` shows the line `ReplicaFailure True FailedCreate` under `Conditions`. In k9s, the fastest way is `:deploy dbg-04⏎`, then `Enter` on `web`. That takes you to the ReplicaSet, where `d` shows the events. `:quota dbg-04⏎` with `d` shows how full the quota is, or on the command line `kubectl --context rke2 describe quota -n dbg-04`.

CAUSE

The quota limits `requests.cpu`, `requests.memory`, `limits.cpu` and `limits.memory`. So admission requires exactly these four values from every container in `dbg-04`. The nginx container has none, so the API server rejects every pod. The ReplicaSet keeps trying with growing intervals and logs every attempt as `FailedCreate`. Because no pod is created, there is no pod event either, and `kubectl get pods` stays empty.

Fix

FILE `szenarien/04-pending/fix-quota.yaml (excerpt)`

```
        resources:
          requests:
            cpu: 100m
            memory: 64Mi
          limits:
            cpu: 250m
            memory: 128Mi
```

MAC

Step 17

```
kubectl --context rke2 apply -f szenarien/04-pending/fix-quota.yaml
kubectl --context talos apply -f szenarien/04-pending/fix-quota.yaml
```

New pod template, new ReplicaSet. The old one is scaled to zero.

WHAT YOU SHOULD SEE

```
$ kubectl --context rke2 describe quota -n dbg-04
Name:            dbg-quota
Namespace:       dbg-04
Resource         Used   Hard
--------         ----   ----
limits.cpu       500m   2
limits.memory    256Mi  2Gi
pods             2      10
requests.cpu     200m   1
requests.memory  128Mi  1Gi
```

Two pods are running, and the quota counts exactly their requests and limits.

If the quota is used up, the message is `exceeded quota: dbg-quota, requested: ..., used: ..., limited: ...`. The place stays the same: the events on the ReplicaSet. Rejections by Pod Security (Chapter 10) and by webhooks like Kyverno (Chapter 11) end up there too.

TRY IT YOURSELF

Apply `szenarien/04-pending/aufgabe-limitrange.yaml` to one of the two clusters. The Deployment `rechner` ("calculator") never reaches `1/1`. Find out why without reading the file first, and fix it with your own change.

Hint: there is no pod here either. The events on the ReplicaSet name a LimitRange whose `max` is smaller than the container's limit. Lower `limits.cpu` to at most `500m`. You do not need to add requests: if a container specifies only limits, Kubernetes sets the requests to the same value, and that also satisfies the quota.

Finally, you clean up. This removes the quota, the LimitRange, the PVC and all Deployments:

MAC — Step 18

```
kubectl --context rke2 delete namespace dbg-04
kubectl --context talos delete namespace dbg-04
```

The local-path volume is deleted along with it because of `RECLAIMPOLICY Delete`.

WHAT YOU LEARNED

- `Pending` means: no node. There are no logs. The answer is in the pod's events (`describe`, `kubectl events --for`).
- The scheduler message counts reasons per node. Several reasons add up to the number of nodes. "Preemption is not helpful" means: you have to change something yourself.
- Requests are reservations against `allocatable`, not against the actual load.
- The selector says where a pod wants to go, the toleration says where it may go. You show which label is missing and which taint is in the way with `custom-columns` on the nodes.
- Neither lab distribution has a default StorageClass. PVCs there need `storageClassName`.
- If the pod is missing entirely, the answer is on the ReplicaSet (`FailedCreate`): quota, LimitRange, Pod Security or webhook.

CHAPTER 5

Images and Container Start

You tell the pull errors apart by their message, check images directly on the node, set up a registry mirror on RKE2 and Talos, and find errors that happen between the pull and the start of a container.

Time: 60 minutes **Requires:** Chapters 2 and 4

5.1 What Breaks in This Chapter

After scheduling, it is the kubelet's turn. It has containerd pull the image, assembles the container's configuration from environment variables, Secrets and volumes, and starts it. Each of these three steps can fail, and each step has its own status.

Step	Status in `get pods`	Scenario
Pull the image	`ErrImagePull`, `ImagePullBackOff`	Typo in the tag, registry unreachable, private repository, rate limit
Build the configuration	`CreateContainerConfigError`	ConfigMap missing, key missing in the Secret
Start the process	`CrashLoopBackOff`, log `exec format error`	Script without a shebang, wrong architecture

All scenarios run in the namespace `dbg-05`. In this chapter, the scheduler has always placed the pod already. The `NODE` column in `kubectl get pods -o wide` tells you which node you have to look at.

5.2 Three Pull Errors, Three Messages

`ImagePullBackOff` is not an error but a waiting state: the kubelet tried a pull, it failed, and now it waits with a growing interval (up to five minutes) for the next attempt. The cause is in the message of the **first** failure. The manifest creates three broken Deployments at once:

FILE `szenarien/05-images/kaputt-pull.yaml (excerpt)`

```
  name: tippfehler
# ...
        - name: web
          image: nginx:1.29-alpnie
---
```

```
  name: unerreichbar
# ...
          image: registry.labor.invalid:5000/web:1.0
---
  name: privat
# ...
          image: docker.io/dbglabor/privat-app:1.0
```

Two swapped letters in the tag, a registry that does not exist (the domain `.invalid` is reserved for this), and a repository you have no credentials for. The Deployment names mean “typo”, “unreachable” and “private”.

MAC Step 1

```
kubectl --context rke2 apply -f szenarien/05-images/kaputt-pull.yaml
kubectl --context talos apply -f szenarien/05-images/kaputt-pull.yaml
```

Creates the namespace and three Deployments.

SYMPTOM

```
$ kubectl --context rke2 get pods -n dbg-05
NAME                              READY   STATUS             RESTARTS   AGE
privat-6c7d9f8b5-4kq9m            0/1     ImagePullBackOff   0          58s
tippfehler-7b9c5d4f6-p8x2l        0/1     ErrImagePull       0          58s
unerreichbar-5f6d8c7b9-r3n7v      0/1     ImagePullBackOff   0          58s
```

The status switches between `ErrImagePull` (just failed) and `ImagePullBackOff` (waiting). You cannot tell the three cases apart by the status alone.

Investigate

MAC Step 2

```
kubectl --context rke2 describe pod -n dbg-05 -l app=tippfehler | tail -n 6
```

The events show the whole sequence of a pull attempt.

```
  Normal   Pulling    40s (x3 over 82s)  kubelet  Pulling image "nginx:1.29-alpnie"
  Warning  Failed     39s (x3 over 81s)  kubelet  Failed to pull image "nginx:1.29-
alpnie": rpc
  error: code = NotFound desc = failed to pull and unpack image "docker.io/library/
nginx:1.29-
  alpnie": failed to resolve reference "docker.io/library/nginx:1.29-alpnie":
docker.io/library/
  nginx:1.29-alpnie: not found
  Warning  Failed     39s (x3 over 81s)  kubelet  Error: ErrImagePull
  Normal   BackOff    12s (x4 over 80s)  kubelet  Back-off pulling image "nginx:1.29-
alpnie"
  Warning  Failed     12s (x4 over 80s)  kubelet  Error: ImagePullBackOff
```

Only the `Failed to pull image` line matters. Read it from the end: the last link is the actual answer from containerd or the registry. Before it is what containerd was trying to do. Also note that the

kubelet expands the short name `nginx` to `docker.io/library/nginx`. For the other two pods, you fetch only this line:

MAC — Step 3

```
kubectl --context rke2 events -n dbg-05 --types=Warning | grep "Failed to pull"
```

Filters all pull errors in the namespace. On Talos the output looks the same, only the address of the DNS resolver may differ.

The three messages end like this:

Deployment	End of the message (shortened)
`tippfehler`	`docker.io/library/nginx:1.29-alpnie: not found`
`unerreichbar`	`dial tcp: lookup registry.labor.invalid ...: no such host`
`privat`	`pull access denied, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed`

In k9s: `:pods dbg-05⏎`, select the pod, press `d` (events at the bottom) or `Shift-E` (plugin from Chapter 3).

CAUSE

Three different errors under the same status. `not found` means: the registry knows the repository, but not this tag. `no such host` means: the **node** cannot resolve the name of the registry. containerd on the node does the pull, not the pod, so the node's DNS is what counts, not CoreDNS. `pull access denied` means: the registry refuses access. Docker Hub deliberately does not distinguish between "does not exist" and "you are not allowed to see it".

Other endings you should know:

Message ends with	Likely cause
`i/o timeout`	Firewall or proxy between node and registry
`x509: certificate signed by unknown authority`	Registry with its own CA that the node does not trust
`401 Unauthorized`	Credentials present, but wrong or expired
`429 Too Many Requests`, `toomanyrequests`	Docker Hub rate limit, see below
`no match for platform in manifest`	The image does not exist for the node's architecture

Fix

Only a corrected manifest fixes the typo and the wrong registry:

MAC Step 4

```
kubectl --context rke2 apply -f szenarien/05-images/fix-pull.yaml
kubectl --context talos apply -f szenarien/05-images/fix-pull.yaml
```

Sets the image `nginx:1.29-alpine` for `tippfehler` and `unerreichbar`.

For a private repository, the pod needs credentials in a Secret of type `kubernetes.io/dockerconfigjson` in the **same** namespace. You create it with `kubectl -n dbg-05 create secret docker-registry regcred --docker-server=docker.io --docker-username=... --docker-password=...`, ideally with a read-only token, and reference it in the pod:

FILE `szenarien/05-images/fix-pull-privat.yaml (excerpt)`

```
    spec:
      imagePullSecrets:
        - name: regcred
      containers:
        - name: web
          image: docker.io/dbglabor/privat-app:1.0
```

The repository `dbglabor/privat-app` does not exist, so in the lab this pod stays stuck even with the Secret. Replace the image with a private repository of your own to walk through the process. Alternatively, you attach the Secret to the ServiceAccount: `kubectl -n dbg-05 patch serviceaccount default -p '{"imagePullSecrets":[{"name":"regcred"}]}'`. Then all new pods of this ServiceAccount get it automatically.

WHAT YOU SHOULD SEE

`tippfehler` and `unerreichbar` are `Running`. The events show `Successfully pulled image "nginx:1.29-alpine" in ...`. If you do not have a private repository of your own, delete the pod `privat`: `kubectl --context rke2 -n dbg-05 delete deployment privat`.

5.3 Looking on the Node

Sometimes you want to know whether a pull works on the node itself, without the kubelet in between: for example, to check credentials, a mirror or a proxy. On RKE2 you do this with `crictl`, on Talos with `talosctl image`. Both talk to the same containerd instance that the kubelet uses.

RKE2-W1 Step 5

```
sudo -i
export PATH=$PATH:/var/lib/rancher/rke2/bin
export CRI_CONFIG_FILE=/var/lib/rancher/rke2/agent/etc/crictl.yaml
crictl pull nginx:1.29-alpnie
crictl pull nginx:1.29-alpine
crictl images | grep nginx
```

The first pull fails with the same message as in the event. The second ends with `Image is up to date for sha256:...`. `crictl images` lists the columns `IMAGE`, `TAG`, `IMAGE ID` and `SIZE`.

MAC Step 6

```
talosctl -n 172.16.0.171 image pull docker.io/library/nginx:1.29-alpine
talosctl -n 172.16.0.171 image list | grep nginx
```

`image pull` reports `172.16.0.171: pulled image ...`. `image list` shows the columns `NODE`, `IMAGE`, `DIGEST`, `SIZE` and more. Without further options, `talosctl image` works in the namespace `cri`, that is, with the images of the Kubernetes workloads.

RKE2

More detailed pull errors are in the containerd log `/var/lib/rancher/rke2/agent/containerd/containerd.log`.

TALOS

You read the log of the workload runtime with `talosctl -n 172.16.0.171 logs cri`.

5.4 Rate Limit and Registry Mirror

Docker Hub limits anonymous pulls to 100 per six hours per IP address. Behind a NAT, all nodes and all colleagues share this one address. When the limit is reached, pulls from `docker.io` fail with `429 Too Many Requests`, and Docker's message contains `toomanyrequests: You have reached your pull rate limit`. In the lab, you do not provoke this on purpose. But you set up the remedy that also helps against a slow or failed registry: a mirror at the node level.

A mirror is not configured in the pod, but in containerd on every node. The pod still says `nginx:1.29-alpine`, and containerd asks the mirror first. In the lab, that is `mirror.gcr.io`, a public cache by Google for popular images from Docker Hub.

RKE2: registries.yaml

FILE labor/rke2/registries-mirror.yaml (excerpt)

```
mirrors:
  docker.io:
    endpoint:
      - "https://mirror.gcr.io"
# Credentials for your own registry (do not commit them to the repo):
# configs:
#   "registry.example.com:5000":
#     auth:
#       username: xxxxxx
#       password: xxxxxx
```

The format is specific to RKE2. RKE2 generates the containerd configuration from it at startup. Credentials go under `configs`, with the hostname of the registry as the key.

Copy the file with `scp` and your SSH user from Chapter 2 to `/tmp/registries.yaml` on every node and install it there:

RKE2-W1 Step 7

```
sudo install -m 600 /tmp/registries.yaml /etc/rancher/rke2/registries.yaml
sudo systemctl restart rke2-agent
sudo cat /var/lib/rancher/rke2/agent/etc/containerd/certs.d/docker.io/hosts.toml
```

RKE2 reads `registries.yaml` only at startup, hence the restart (on `rke2-cp` : `rke2-server`). Running pods survive it. The generated `hosts.toml` must contain an entry for `https://mirror.gcr.io`.

PITFALL

RKE2 always tries the original registry as a last resort, even when a mirror is configured. So a broken mirror does not stand out right away. Pulls just get slower and count against the limit again. If you want to turn off the fallback, set the option `disable-default-registry-endpoint: true` in `/etc/rancher/rke2/config.yaml`.

Talos: RegistryMirrorConfig

Talos 1.14 describes mirrors in a separate configuration document. The old way via `machine.registries` in the machine config still works, but it is deprecated.

FILE `labor/talos/patch-registry.yaml (excerpt)`

```
apiVersion: v1alpha1
kind: RegistryMirrorConfig
name: docker.io
endpoints:
  - url: https://mirror.gcr.io
```

`name` is the registry that is mirrored. Credentials for a mirror go in another document of type `RegistryAuthConfig` with `name`, `username` and `password`, and your own CAs go in `RegistryTLSConfig`. With `skipFallback: true`, Talos no longer asks the original registry at all.

MAC Step 8

```
talosctl -n 172.16.0.170,172.16.0.171 patch machineconfig \
  --patch @labor/talos/patch-registry.yaml
talosctl -n 172.16.0.171 ls /etc/cri/conf.d/hosts
```

The patch adds the document on both nodes. From it, Talos writes the containerd configuration under `/etc/cri/conf.d/hosts`, with an entry for `docker.io`. `talosctl patch` reports whether the change was applied without a reboot.

WHAT YOU SHOULD SEE

Pulling an image from Docker Hub that is not on the node yet still works: on RKE2 with `crictl pull busybox:1.37`, on Talos with `talosctl -n 172.16.0.171 image pull docker.io/library/busybox:1.37`. The image shows up under its original name `docker.io/library/busybox`, not under the mirror's name. Whether the mirror was really asked is shown by the containerd log on RKE2 and by `talosctl logs cri` on Talos.

5.5 CreateContainerConfigError

The image is there, but the kubelet cannot configure the container. If a ConfigMap or Secret referenced in `env` or `envFrom` is missing, the container does not even start.

FILE `szenarien/05-images/kaputt-config.yaml (excerpt)`

```yaml
kind: Secret
metadata:
  name: app-secret
stringData:
  db-password: geheim-nur-im-labor
---
# ... Deployment "app"
          envFrom:
            - configMapRef:
                name: app-config
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: app-secret
                  key: DB_PASSWORD
```

Two errors: the ConfigMap `app-config` does not exist, and the Secret has the key `db-password`, not `DB_PASSWORD`.

MAC Step 9

```
kubectl --context rke2 apply -f szenarien/05-images/kaputt-config.yaml
```

On Talos, this scenario behaves the same way.

SYMPTOM

```
$ kubectl --context rke2 get pods -n dbg-05 -l app=app
NAME                   READY   STATUS                       RESTARTS   AGE
app-6f9d7c8b5d-w2xkq   0/1     CreateContainerConfigError   0          21s
```

Investigate

MAC Step 10

```
kubectl --context rke2 events -n dbg-05 --types=Warning
```

All warnings in the namespace, the newest at the bottom. In k9s: select the pod, press `Shift-E`.

```
LAST SEEN   TYPE      REASON    OBJECT                     MESSAGE
3s          Warning   Failed    Pod/app-6f9d7c8b5d-w2xkq   Error: configmap "app-
config" not found
```

The kubelet only ever reports the first error it runs into. Create the ConfigMap and see what comes next:

MAC Step 11

```
kubectl --context rke2 -n dbg-05 create configmap app-config \
  --from-literal=DB_HOST=db.dbg-05.svc.cluster.local
```

The kubelet tries again on its own. After a few seconds, there is a new event.

```
Warning   Failed    Pod/app-6f9d7c8b5d-w2xkq   Error: couldn't find key DB_PASSWORD in
Secret
dbg-05/app-secret
```

`kubectl --context rke2 describe secret app-secret -n dbg-05` shows which keys the Secret really has under `Data`, without printing the values: only `db-password`.

CAUSE

The kubelet resolves environment variables right before the container starts. If the object or the key is missing and the reference is not `optional: true`, it aborts, sets `CreateContainerConfigError` and tries again periodically. At this point the pod is already scheduled and the image is pulled, so at first glance the error looks like an image problem.

Fix

MAC Step 12

```
kubectl --context rke2 apply -f szenarien/05-images/fix-config.yaml
```

Creates the ConfigMap declaratively and references the correct key `db-password`.

WHAT YOU SHOULD SEE

The pod is `Running`, and `kubectl --context rke2 logs -n dbg-05 deploy/app` shows `Verbinde mit db.dbg-05.svc.cluster.local` ("Connecting to …").

PITFALL

If the missing ConfigMap is mounted as a **volume** instead of an environment variable, the error looks different: the pod hangs in `ContainerCreating`, and the event is `FailedMount` with `MountVolume.SetUp failed for volume "config" : configmap "app-config" not found`. The same cause, a different status.

5.6 exec format error

This error occurs when the kernel cannot execute the file that is supposed to start as a process. The classic case is an image for the wrong CPU architecture, for example built for `arm64` on a Mac with Apple Silicon and started on `amd64` nodes. In the lab, you reproduce it with a second, equally common trigger: a start script without a shebang line.

FILE `szenarien/05-images/kaputt-exec.yaml (excerpt)`

```yaml
kind: ConfigMap
data:
  start.sh: |
    echo "Starte Dienst"
    exec sleep infinity
---
# ... Deployment "skript"
          command: ["/scripts/start.sh"]
          volumeMounts:
            - name: skript
              mountPath: /scripts
      volumes:
        - name: skript
          configMap:
            name: start-skript
            defaultMode: 0755
```

The script is executable, but the first line `#!/bin/sh` is missing. `skript` means “script”, and `Starte Dienst` means “Starting service”.

MAC Step 13

```
kubectl --context rke2 apply -f szenarien/05-images/kaputt-exec.yaml
kubectl --context talos apply -f szenarien/05-images/kaputt-exec.yaml
```

Creates the ConfigMap and the Deployment `skript`.

SYMPTOM

After a short time, `kubectl --context rke2 get pods -n dbg-05 -l app=skript` shows `CrashLoopBackOff` with a rising number in `RESTARTS`.

Investigate

MAC — Step 14

```
kubectl --context rke2 logs -n dbg-05 deploy/skript --previous
```

`--previous` reads the log of the last terminated container. In k9s: select the pod, press `p`.

```
exec /scripts/start.sh: exec format error
```

This line does not come from the script, but from the container runtime (runc) that tried to start the process. You find out in two steps whether the architecture or the shebang is to blame:

MAC — Step 15

```
kubectl --context rke2 get nodes -L kubernetes.io/arch
kubectl --context rke2 get cm start-skript -n dbg-05 -o jsonpath='{.data.start\.sh}' | head -n 1
```

The first line shows the architecture of each node in the `ARCH` column. The second shows the first line of the script: `#!/bin/sh` is missing here.

CAUSE

runc starts the container's command directly with the `execve` system call, without a shell. The kernel recognizes executable files by their first bytes: ELF binaries for its own architecture, or scripts with `#!`. It rejects everything else with `ENOEXEC`, and that shows up as `exec format error`. In an interactive shell, the same script would run, because in this case the shell interprets it itself.

Fix

MAC — Step 16

```
kubectl --context rke2 apply -f szenarien/05-images/fix-exec.yaml
kubectl --context rke2 -n dbg-05 rollout restart deployment skript
```

The new ConfigMap starts with `#!/bin/sh`. The restart makes sure the pod sees the new version right away. On Talos, run the same commands with `--context talos`.

WHAT YOU SHOULD SEE

`kubectl --context rke2 logs -n dbg-05 deploy/skript` shows `Starte Dienst`, and the pod is `Running` with `RESTARTS 0`.

WHY IT MATTERS

With the wrong architecture, the pull often fails already (`no match for platform in manifest`) if the image does not offer the node's architecture at all. If the metadata is right but the binary is not, you get `exec format error` . `docker buildx imagetools inspect nginx:1.29` shows which platforms an image has.

TRY IT YOURSELF

Apply `szenarien/05-images/aufgabe-pullpolicy.yaml` with `--context talos` . The Deployment `offline` does not start. Find the cause through the events and get the pod running **without** changing the manifest.

Hint: the status is `ErrImageNeverPull` , and the message is `Container image "nginx:1.29-alpine-slim" is not present with pull policy of Never` . With `imagePullPolicy: Never` , the kubelet never pulls by itself. Preload the image with `talosctl image pull` on the node where the pod is. The pod starts on the kubelet's next attempt.

Clean up on both clusters:

MAC Step 17

```
kubectl --context rke2 delete namespace dbg-05
kubectl --context talos delete namespace dbg-05
```

The registry mirror stays active. It does no harm and spares the rate limit in the following chapters.

WHAT YOU LEARNED

- `ImagePullBackOff` is a waiting state. The cause is in the `Failed to pull image` event, at the end of the message.
- `not found` , `no such host` , `pull access denied` , `x509` , `429` : each ending points to a different cause. The node does the pull, not the pod.
- `crictl` on RKE2 and `talosctl image` on Talos check pulls directly against containerd, without the kubelet.
- Mirrors and registry credentials at the node level: on RKE2 `/etc/rancher/rke2/registries.yaml` plus a restart, on Talos `RegistryMirrorConfig` via `talosctl patch` .
- `CreateContainerConfigError` reports missing ConfigMaps, Secrets and keys, always only the first error.
- `exec format error` means: the kernel cannot execute the file. Wrong architecture or a missing shebang.

CHAPTER 6

Crashes and Restarts

You read exit codes and the log of the previous container, recognize OOMKilled and probe failures, and investigate containers without a shell using `kubectl debug`.

Time: 75 minutes **Requires:** Chapters 1, 3 and 5

6.1 What Breaks in This Chapter

Now the container runs, but not for long or not correctly. The two most important tools for this are the `Last State` block in `kubectl describe pod` and the log of the **previous** container. After a restart, `kubectl logs` shows only the new container. The error message of the old one is in `kubectl logs --previous`.

Scenario	Symptom	First clue
Wrong command, app error	`CrashLoopBackOff`	Exit code 127 or 1, `logs --previous`
Memory limit	`OOMKilled`, then `CrashLoopBackOff`	Exit code 137, reason `OOMKilled`
Liveness too strict, readiness wrong	Restarts, or `0/1` without restarts	Events `Unhealthy`
`readOnlyRootFilesystem`	`CrashLoopBackOff`	`Read-only file system` in the log
Distroless image	`exec` fails, no shell	`kubectl debug`

All scenarios run in the namespace `dbg-06`. You need the exit codes from Chapter 1 all the time here: 1 is an application error, 126 and 127 mean “not executable” and “not found” in a shell, and 137 is 128 + 9, that is, `SIGKILL`.

6.2 CrashLoopBackOff: Command and Application

Like `ImagePullBackOff`, `CrashLoopBackOff` is a waiting state. The container has exited, the kubelet restarts it and doubles the waiting time after every crash, from 10 seconds up to at most 5 minutes. The cause is not in the status, but in the exit code and the log.

FILE `szenarien/06-crash/kaputt-crash.yaml (excerpt)`

```
  name: tippfehler
# ...
          command: ["sh", "-c", "serv -p 8080 -h /www"]
```

```
---
  name: appfehler
# ...
          command:
            - sh
            - -c
            - |
              echo "Konfiguration wird geladen"
              if [ -z "$DATABASE_URL" ]; then
                echo "FEHLER: DATABASE_URL ist nicht gesetzt" >&2
                exit 1
              fi
              exec sleep infinity
```

`tippfehler` (“typo”) is supposed to start `httpd`, but it says `serv`. `appfehler` (“app error”) aborts because an environment variable is missing, like many real applications do at startup. Its messages mean “Loading configuration” and “ERROR: DATABASE_URL is not set”.

MAC Step 1

```
kubectl --context rke2 apply -f szenarien/06-crash/kaputt-crash.yaml
kubectl --context talos apply -f szenarien/06-crash/kaputt-crash.yaml
```

Creates the namespace and two Deployments.

SYMPTOM

```
$ kubectl --context rke2 get pods -n dbg-06
NAME                          READY   STATUS             RESTARTS      AGE
appfehler-5c8d7b6f4-n4w8k     0/1     CrashLoopBackOff   3 (25s ago)   71s
tippfehler-6b9f8c7d5-k2v7x    0/1     CrashLoopBackOff   3 (28s ago)   71s
```

Investigate

MAC Step 2

```
kubectl --context rke2 describe pod -n dbg-06 -l app=tippfehler | grep -A 9 "State:"
```

The excerpt around `State` and `Last State` shows how the last container ended.

```
  State:          Waiting
    Reason:       CrashLoopBackOff
  Last State:     Terminated
    Reason:       Error
    Exit Code:    127
    Started:      Mon, 09 Nov 2026 10:12:41 +0100
    Finished:     Mon, 09 Nov 2026 10:12:41 +0100
  Ready:          False
  Restart Count:  3
```

Started and Finished in the same second, exit code 127: the shell did not find a command. The log confirms it:

MAC Step 3

```
kubectl --context rke2 logs -n dbg-06 deploy/tippfehler --previous
kubectl --context rke2 logs -n dbg-06 deploy/appfehler --previous
```

`--previous` reads the previous, crashed container. Without the option, depending on the timing, you catch a container that just started and has an empty log.

```
sh: serv: not found
Konfiguration wird geladen
FEHLER: DATABASE_URL ist nicht gesetzt
```

`appfehler` ends with exit code 1, and the application wrote the message itself. In k9s the path is shorter: `:pods dbg-06⏎`, select the pod, `p` opens the logs of the previous container (`l` those of the current one), and `d` shows `Last State`.

CAUSE

`tippfehler`: the shell does not find `serv` and exits with 127. `appfehler`: the application checks its configuration and exits with 1. In both cases the container is done, and because the `restartPolicy` of a Deployment is always `Always`, the kubelet restarts it. If this repeats, it waits longer and longer: `CrashLoopBackOff`.

Fix

MAC Step 4

```
kubectl --context rke2 apply -f szenarien/06-crash/fix-crash.yaml
kubectl --context talos apply -f szenarien/06-crash/fix-crash.yaml
```

`tippfehler` now starts `httpd`, and `appfehler` gets `DATABASE_URL` as an environment variable.

WHAT YOU SHOULD SEE

Both pods are `Running` with `RESTARTS 0`, because the Deployment created new pods.

PITFALL

If the wrong command is directly in `command` instead of after `sh -c`, that is `command: ["serv"]`, there is no shell that could report 127. Then the runtime cannot start the process at all. `Last State` shows `Reason: StartError` with exit code 128, and the message contains `exec: "serv": executable file not found in $PATH`. In this case, there is no log.

6.3 OOMKilled

A memory limit is a hard boundary. If a container exceeds it, the kernel terminates it with `SIGKILL`. There is no warning, no log line from the application and no orderly shutdown.

FILE `szenarien/06-crash/kaputt-oom.yaml (excerpt)`

```
        - name: app
          image: polinux/stress:1.0.4
          command: ["/usr/local/bin/stress"]
          args: ["--vm", "1", "--vm-bytes", "200M", "--vm-hang", "0"]
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              memory: 128Mi
```

`stress` allocates 200 MB and holds it. The limit allows 128 MiB.

MAC — Step 5

```
kubectl --context rke2 apply -f szenarien/06-crash/kaputt-oom.yaml
kubectl --context talos apply -f szenarien/06-crash/kaputt-oom.yaml
```

Creates the Deployment `speicher` (“memory”).

SYMPTOM

`kubectl --context rke2 get pods -n dbg-06 -l app=speicher` briefly shows `OOMKilled` in the `STATUS` column, then `CrashLoopBackOff`. If you only look now and then, you only see the latter.

Investigate

MAC — Step 6

```
kubectl --context rke2 get pod -n dbg-06 -l app=speicher \
  -o jsonpath='{.items[0].status.containerStatuses[0].lastState.terminated}' | jq .
```

Reads the last terminated state directly from the status. Without `jq`, you get the same information on one line.

```
{
  "containerID": "containerd://9f2c4e1b7a...",
  "exitCode": 137,
  "finishedAt": "2026-11-09T09:31:07Z",
  "reason": "OOMKilled",
  "startedAt": "2026-11-09T09:31:06Z"
}
```

`reason: OOMKilled` and exit code 137 are unambiguous. The log hardly helps here: `logs --previous` shows only `stress: info: [1] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd`, then nothing more. The node’s kernel provides the proof:

RKE2

On the pod's node (the `NODE` column in `get pods -o wide`), via SSH:

```
sudo journalctl -k --since "10 min ago" \
  | grep -i "out of memory"
```

TALOS

Without SSH, from the Mac:

```
talosctl -n 172.16.0.171 dmesg \
  | grep -i "out of memory"
```

Both return a line like `Memory cgroup out of memory: Killed process 23817 (stress) total-vm:208712kB, anon-rss:130816kB, ...`.

CAUSE

The memory limit is enforced as a cgroup boundary in the kernel. If the container exceeds it, the kernel's OOM killer picks a process in this cgroup and terminates it with `SIGKILL` (signal 9, hence 128 + 9 = 137). The kubelet learns about it through the runtime and sets the reason `OOMKilled`. Requests do not help: they are only bookkeeping for the scheduler.

Fix

Either the application really needs more memory, and you raise the limit. Or it uses too much, and then it is an application bug, for example a leak or a cache that is too large. In the lab, it gets more:

MAC Step 7

```
kubectl --context rke2 apply -f szenarien/06-crash/fix-oom.yaml
kubectl --context talos apply -f szenarien/06-crash/fix-oom.yaml
```

Sets the memory request and limit to `256Mi`.

WHAT YOU SHOULD SEE

The pod stays `Running`. On RKE2, `kubectl --context rke2 top pod -n dbg-06 -l app=speicher` shows about 200 MiB. On Talos the metrics-server is still missing (Chapter 15), so `kubectl top` only reports `error: Metrics API not available` there.

PITFALL

`OOMKilled` affects a container that exceeds its **own** limit. If the **node** runs short, the kubelet evicts whole pods. They then show `Failed` with the reason `Evicted` (Chapter 13).

6.4 Probes: Too Strict and Aimed Wrong

Probes are the questions the kubelet asks a container regularly. The liveness probe asks "are you still alive?" and restarts the container after repeated no's. The readiness probe asks "can you

accept requests?" and takes the pod out of the Service on a no. Setting either of them wrong produces two completely different symptoms.

FILE szenarien/06-crash/kaputt-probes.yaml (excerpt)

```
  name: langsam
# ...
              echo "Lade Daten (dauert 40 Sekunden) ..."
              sleep 40
              # ... then httpd on port 8080
          livenessProbe:
            httpGet: {path: /healthz, port: 8080}
            initialDelaySeconds: 10
            periodSeconds: 5
            failureThreshold: 3
---
  name: nieready
# ... httpd on port 8080, serves only /healthz
          readinessProbe:
            httpGet: {path: /ready, port: 8080}
# ... plus a Service "nieready"
```

`langsam` ("slow") needs 40 seconds until its first sign of life ("Lade Daten" means "Loading data, takes 40 seconds"), but the liveness probe gives it only about 20. `nieready` ("never ready") probes a path that does not exist.

MAC Step 8

```
kubectl --context rke2 apply -f szenarien/06-crash/kaputt-probes.yaml
kubectl --context talos apply -f szenarien/06-crash/kaputt-probes.yaml
```

Two Deployments and a Service.

SYMPTOM

```
$ kubectl --context rke2 get pods -n dbg-06 -l 'app in (langsam,nieready)'
NAME                         READY   STATUS    RESTARTS      AGE
langsam-7d6c9b8f5-x4q2m      1/1     Running   3 (12s ago)   2m5s
nieready-5f7b8d9c6-j6t3p     0/1     Running   0             2m5s
```

`langsam` runs, but keeps restarting. `nieready` runs without restarts, but never becomes ready.

Investigate

MAC Step 9

```
kubectl --context rke2 events -n dbg-06 | grep -E "Unhealthy|Killing"
```

Probe failures have the reason `Unhealthy` (type `Warning`). The restart is reported by an event of type `Normal` with the reason `Killing`, which is why there is no `--types=Warning` filter here.

```
Warning   Unhealthy   Pod/langsam-7d6c9b8f5-x4q2m    Liveness probe failed: Get
"http://10.42.1.23:8080/healthz": dial tcp 10.42.1.23:8080: connect: connection
refused
Warning   Unhealthy   Pod/nieready-5f7b8d9c6-j6t3p   Readiness probe failed: HTTP
```

```
probe failed
with statuscode: 404
Normal    Killing     Pod/langsam-7d6c9b8f5-x4q2m    Container app failed liveness
probe, will be
restarted
```

For `langsam`, it is worth looking at `Last State` and the previous log as well: exit code 137, and the last line is `Lade Daten (dauert 40 Sekunden) ...`. The container was terminated in the middle of loading. Exit code 137 happens because the shell ignores the kubelet's `SIGTERM`, and `SIGKILL` follows after the grace period. For `nieready`, what matters is what the Service sees:

MAC Step 10

```
kubectl --context rke2 get endpointslices -n dbg-06 -l kubernetes.io/service-name=nieready \
  -o jsonpath='{range .items[*].endpoints[*]}{.addresses[0]} ready={.conditions.ready}{"\n"}{end}'
```

Lists every address of the EndpointSlice with its state.

```
10.42.1.24 ready=false
```

In k9s, `d` on the pod shows the probe configuration under `Liveness` and `Readiness`, including `delay`, `period` and `#failure`.

CAUSE

`langsam`: the liveness probe starts after 10 seconds and gives up after three failures 5 seconds apart, so after about 20 seconds. The application needs 40. So the kubelet kills it every time before it reaches the goal, a cycle with no way out. `nieready`: the readiness probe asks for `/ready`, and httpd returns 404. The pod stays in the EndpointSlice, but with `ready=false`, and the Service sends no traffic to it. Readiness never restarts anything, so there is no crash here, only silence.

Fix

For slow starts, there is the startup probe. As long as it has not succeeded, the kubelet suspends liveness and readiness. After that, the strict liveness probe takes over as usual.

FILE szenarien/06-crash/fix-probes.yaml (excerpt)

```yaml
          startupProbe:
            httpGet: {path: /healthz, port: 8080}
            periodSeconds: 5
            failureThreshold: 24
          livenessProbe:
            httpGet: {path: /healthz, port: 8080}
            periodSeconds: 5
            failureThreshold: 3
# ... for nieready:
          readinessProbe:
            httpGet: {path: /healthz, port: 8080}
```

The startup probe gives the container up to 24 × 5 = 120 seconds. The liveness probe's `initialDelaySeconds` is dropped.

MAC Step 11

```
kubectl --context rke2 apply -f szenarien/06-crash/fix-probes.yaml
kubectl --context talos apply -f szenarien/06-crash/fix-probes.yaml
```

Both Deployments roll out again.

WHAT YOU SHOULD SEE

After a little more than 40 seconds, `get pods` shows both with `1/1 Running` and `RESTARTS 0`. The EndpointSlice reports `ready=true`, and `kubectl --context rke2 run test -n dbg-06 --rm -it --image=busybox:1.37 --restart=Never -- wget -qO- nieready/healthz` answers with `ok`.

6.5 readOnlyRootFilesystem

A read-only root file system is a common hardening measure, even though Pod Security `restricted` does not require it. But many images write at startup to places the developer never consciously chose.

FILE `szenarien/06-crash/kaputt-readonly.yaml (excerpt)`

```
      - name: web
        image: nginx:1.29
        securityContext:
          readOnlyRootFilesystem: true
```

MAC Step 12

```
kubectl --context rke2 apply -f szenarien/06-crash/kaputt-readonly.yaml
```

It behaves the same on Talos.

SYMPTOM

After a few seconds, `webro` is in `CrashLoopBackOff`, and `Last State` shows exit code 1.

MAC Step 13

```
kubectl --context rke2 logs -n dbg-06 deploy/webro --previous
```

The log of the crashed container.

```
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf
(read-only file system?)
```

```
...
2026/11/09 09:44:12 [emerg] 1#1: mkdir() "/var/cache/nginx/client_temp" failed (30:
Read-only file system)
nginx: [emerg] mkdir() "/var/cache/nginx/client_temp" failed (30: Read-only file
system)
```

The `info` line is harmless. Only `[emerg]` stops nginx.

CAUSE

With `readOnlyRootFilesystem: true`, the image's file system is mounted read-only. At startup, nginx needs writable directories for temporary files (`/var/cache/nginx`) and for the PID file (under `/run`). Error code 30 is `EROFS`, "Read-only file system". Volumes stay writable even with a read-only root.

FILE `szenarien/06-crash/fix-readonly.yaml (excerpt)`

```yaml
          volumeMounts:
            - name: cache
              mountPath: /var/cache/nginx
            - name: run
              mountPath: /run
      volumes:
        - name: cache
          emptyDir: {}
        - name: run
          emptyDir:
            medium: Memory
            sizeLimit: 16Mi
```

Two `emptyDir` volumes exactly at the places where nginx writes. Everything else stays protected.

MAC Step 14

```
kubectl --context rke2 apply -f szenarien/06-crash/fix-readonly.yaml
```

The same on Talos.

WHAT YOU SHOULD SEE

`webro` is `Running`, and the log ends with `Configuration complete; ready for start up` and the startup messages of the worker processes. `kubectl --context rke2 exec -n dbg-06 deploy/webro -- touch /test` still fails with `Read-only file system`, as intended.

6.6 Distroless: Debugging Without a Shell

Distroless images contain only the application: no shell, no `ls`. `kubectl exec ... -- sh` ends with a message that contains `exec: "sh": executable file not found in $PATH`. CoreDNS serves as the example, because its official image is distroless.

FILE szenarien/06-crash/kaputt-distroless.yaml (excerpt)

```
kind: ConfigMap
data:
  corefile: |
    .:1053 {
        whoami
        health :8080
        log
    }
---
# ... Deployment "dns-test"
          image: registry.k8s.io/coredns/coredns:v1.14.7
          args: ["-conf", "/etc/coredns/Corefile"]
          volumeMounts:
            - name: config
              mountPath: /etc/coredns
```

MAC Step 15

```
kubectl --context rke2 apply -f szenarien/06-crash/kaputt-distroless.yaml
kubectl --context rke2 logs -n dbg-06 deploy/dns-test --previous
```

The same on Talos. The pod crashes immediately.

```
open /etc/coredns/Corefile: no such file or directory
```

SYMPTOM

`dns-test` is in `CrashLoopBackOff`, exit code 1. The file is supposedly missing, even though the ConfigMap is mounted. You cannot look with `exec`: there is no shell, and the container barely runs.

Investigating with an Ephemeral Container

`kubectl debug` adds an extra container with an image of your choice to the pod. It shares the pod's network, but it only gets volumes via `--custom`:

FILE szenarien/06-crash/debug-custom.yaml (excerpt)

```
securityContext:
  runAsUser: 65532
  runAsGroup: 65532
volumeMounts:
  - name: config
    mountPath: /etc/coredns
```

Not a complete manifest, only the fields that `kubectl debug` applies to the debug container. User 65532 is the user `nonroot` of the CoreDNS image.

MAC Step 16

```
POD=$(kubectl --context rke2 get pod -n dbg-06 -l app=dns-test -o name)
kubectl --context rke2 debug -it -n dbg-06 $POD --image=busybox:1.37 \
  --profile=baseline --custom=szenarien/06-crash/debug-custom.yaml -- sh
```

Starts busybox as an ephemeral container in the broken pod and opens a shell. `--profile=baseline` is required on Talos, see below.

POD — Step 17

```
ls -l /etc/coredns
lrwxrwxrwx    1 root     root            15 Nov  9 10:31 corefile -> ..data/corefile
```

In the debug container, shown here with its output. You leave it with `exit`, and it stays in the pod spec in a terminated state.

There it is: `corefile` in lowercase. Before you change the ConfigMap, you test the theory on a copy of the pod with a changed command:

MAC — Step 18

```
kubectl --context rke2 debug -n dbg-06 $POD --copy-to=dns-test-kopie --profile=baseline \
  --container=coredns -- /coredns -conf /etc/coredns/corefile
kubectl --context rke2 logs -n dbg-06 dns-test-kopie -c coredns
```

`--copy-to` creates a new pod (here `dns-test-kopie`, "copy") that matches the original, but without labels, so without the Service and the Deployment. `--container` with a command replaces that container's start command. The log begins with `.:1053` and `CoreDNS-1.14.7`: with the right file name, it runs.

CAUSE

When a ConfigMap is mounted, its key becomes the file name, case-sensitive. The ConfigMap provides `corefile`, and CoreDNS looks for `Corefile`. Without a shell in the image, you bring the tool for looking around yourself with `kubectl debug`.

Fix and Verify

MAC — Step 19

```
kubectl --context rke2 -n dbg-06 delete pod dns-test-kopie
kubectl --context rke2 apply -f szenarien/06-crash/fix-distroless.yaml
kubectl --context rke2 -n dbg-06 rollout restart deployment dns-test
```

Delete the copy, apply the ConfigMap with the key `Corefile`, and restart the pods so they see the changed ConfigMap right away.

Now the container runs, and you can attach to its process namespace with `--target`:

MAC — Step 20

```
POD=$(kubectl --context rke2 get pod -n dbg-06 -l app=dns-test -o name)
kubectl --context rke2 debug -it -n dbg-06 $POD --image=busybox:1.37 --target=coredns \
  --profile=baseline --custom=szenarien/06-crash/debug-custom.yaml -- sh
```

`--target=coredns` shares the processes of this container with the debug container.

POD Step 21

```
ps
ls /proc/1/root/etc/coredns/
```

PID 1 is now CoreDNS. Through `/proc/1/root` you see its file system, because the debug container runs as the same user 65532.

WHAT YOU SHOULD SEE

```
PID   USER     TIME  COMMAND
    1 65532     0:00 /coredns -conf /etc/coredns/Corefile
   14 65532     0:00 sh
   21 65532     0:00 ps
Corefile
```

`kubectl get pods -n dbg-06 -l app=dns-test` shows `1/1 Running`.

RKE2

Without the CIS profile, RKE2 does not enforce a Pod Security Standard. The default profile `general` of `kubectl debug` works. It adds the capability `SYS_PTRACE`, which lets a debug container running as root look into other processes too.

TALOS

Talos enforces `baseline` cluster-wide. With the profile `general`, the API server rejects the debug container: `violates PodSecurity "baseline:latest": non-default capabilities (container "debugger-..." must not include "SYS_PTRACE" in securityContext.capabilities.add)`.

Hence `--profile=baseline`, and for `/proc/1/root` the same user as the target process. The k9s plugin `Shift-D` from Chapter 3 also uses `baseline`.

TRY IT YOURSELF

Apply `szenarien/06-crash/aufgabe-probe.yaml`. The pod `waechter` ("guard") is running, and the application answers on port 8080, yet `RESTARTS` keeps rising. Find the cause and fix it with `kubectl edit` (in k9s: `e`).

Hint: the `Unhealthy` events say `dial tcp ...:80: connect: connection refused`. The liveness probe checks port 80 via `tcpSocket`, but httpd listens on 8080.

Clean up:

MAC Step 22

```
kubectl --context rke2 delete namespace dbg-06
kubectl --context talos delete namespace dbg-06
```

Ephemeral containers cannot be removed individually. They only disappear with their pod.

WHAT YOU LEARNED

- `CrashLoopBackOff` is a waiting state. The cause: the exit code in `Last State` and the log from `kubectl logs --previous` (k9s: `p`).
- 127 means command not found (in a shell). `StartError` with 128 means: the runtime could not start the process. 1 is an application error.
- `OOMKilled` with 137 is the kernel stepping in at the cgroup boundary. Proof: `dmesg` or `journalctl -k`.
- Liveness restarts, readiness takes the pod out of the Service. Slow starts belong in a startup probe.
- `readOnlyRootFilesystem` needs `emptyDir` volumes at the places where the application writes.
- Without a shell: `kubectl debug` with an ephemeral container, `--target`, `--custom` and `--copy-to`. On Talos, always with `--profile=baseline`.

CHAPTER 7

Services, DNS and NetworkPolicies

You follow a request from the client pod through the name, the Service and the EndpointSlice to the target pod. You can tell at which stop it gets stuck, and you know why a NetworkPolicy works on RKE2 and not on Talos.

Time: 75 minutes **Requires:** Chapters 2 and 3

7.1 What Breaks in This Chapter

A request to `http://backend` passes four stops: DNS turns the name into the IP of the Service, kube-proxy redirects the Service IP to a pod IP from the EndpointSlice, the packet travels through the CNI to the target pod, and there a process has to listen on the port. Each stop has its own typical failure.

Scenario	Symptom	Stop
Selector does not match	`Connection refused` or timeout, Service without endpoints	EndpointSlice
Wrong `targetPort`	`Could not connect to server`, even though endpoints exist	Target pod
Wrong name	`Could not resolve host`, `NXDOMAIN`	DNS
Default deny without DNS	Name does not resolve, IP works	NetworkPolicy
Policy without effect	Everything works, even though it should be blocked	CNI

All tests run from a tool pod `client` with the image `nicolaka/netshoot:v0.15`. It contains `curl`, `dig`, `nslookup`, `ss` and `tcpdump`. The target is `traefik/whoami`, a tiny web server on port 80 that includes its own pod name in its response. That way you see right away which pod answered.

7.2 The Service Finds No Pods

FILE `szenarien/07-netzwerk/kaputt-selektor.yaml (excerpt)`

```yaml
kind: Deployment
metadata:
  name: backend
```

```
spec:
  replicas: 2
  template:
    metadata:
      labels:
        app: backend
# ...
---
kind: Service
metadata:
  name: backend
spec:
  selector:
    app: backend-api        # ERROR: the pods are labeled app=backend
  ports:
    - name: http
      port: 80
      targetPort: 80
```

The full file creates the namespace `dbg-07`, the pod `client`, the Deployment and the Service. The Service selector looks for a label that no pod carries.

MAC

Step 1

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/kaputt-selektor.yaml
kubectl --context rke2 -n dbg-07 exec client -- curl -sS -m 5 http://backend
```

Creates everything and calls the Service from the client pod. `-m 5` gives up after five seconds, `-sS` hides the progress bar but still shows errors.

SYMPTOM

```
curl: (7) Failed to connect to backend port 80 after 3 ms: Could not connect to server
command terminated with exit code 7
```

The name resolves, otherwise you would get `Could not resolve host`. The connection fails immediately. Depending on the proxy mode and the CNI, you may see a timeout after five seconds instead of an immediate rejection. Both mean the same thing: nobody is waiting behind the Service IP.

The first question is not “What is wrong with the network?” but “Does the Service have any targets at all?”. The answer is in the EndpointSlice, which the EndpointSlice controller maintains for every Service with a selector.

MAC

Step 2

```
kubectl --context rke2 -n dbg-07 get endpointslices -l kubernetes.io/service-name=backend
```

Every EndpointSlice carries the label `kubernetes.io/service-name` with the name of its Service. The older `kubectl get endpoints` still works, but since Kubernetes 1.33 it prints a deprecation warning.

```
NAME            ADDRESSTYPE   PORTS     ENDPOINTS   AGE
backend-7xqzl   IPv4          <unset>   <unset>     41s
```

`<unset>` in the `ENDPOINTS` column means: not a single pod matches. Now compare the selector and the labels directly:

MAC Step 3

```
kubectl --context rke2 -n dbg-07 get svc backend -o jsonpath='{.spec.selector}{"\n"}'
kubectl --context rke2 -n dbg-07 get pods --show-labels
```

The first line shows what the Service looks for, the second what the pods carry.

```
{"app":"backend-api"}
NAME                       READY   STATUS    RESTARTS   AGE   LABELS
backend-5b8d9f6c7d-4kq2m   1/1     Running   0          58s   app=backend,pod-
template-hash=...
backend-5b8d9f6c7d-xv7jt   1/1     Running   0          58s   app=backend,pod-
template-hash=...
client                     1/1     Running   0          58s   app=client
```

The cross-check with the Service selector returns an empty list: `kubectl --context rke2 -n dbg-07 get pods -l app=backend-api` answers `No resources found in dbg-07 namespace.` In k9s you see the same thing as a tree:

K9S Step 4

```
:xray svc dbg-07⏎
```

XRay shows every Service with the pods it reaches below it. Nothing hangs below `backend`. With `Esc` and `:pods dbg-07⏎` you get to the pod list, and `y` on a pod shows its labels in the YAML.

CAUSE

A Service is just a selector plus a virtual IP. The EndpointSlice controller keeps looking for all pods in the namespace whose labels match the selector and that are ready, and it adds them to the EndpointSlice. If no pod matches, the slice stays empty, and kube-proxy has no target for the Service IP. Kubernetes does not report this as an error anywhere. There is no event and no status. You get the same empty slice when the selector is right but the pods are not ready (Chapter 6). `0/1` in `kubectl get pods` points to the probe, `1/1` to the selector.

MAC Step 5

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/fix-service.yaml
kubectl --context rke2 -n dbg-07 get endpointslices -l kubernetes.io/service-name=backend
```

Replaces the selector with `app: backend`. The file also uses `targetPort: http` to point to the named port from the pod template instead of a number.

WHAT YOU SHOULD SEE

```
NAME            ADDRESSTYPE   PORTS   ENDPOINTS               AGE
backend-7xqzl   IPv4          80      10.42.0.21,10.42.1.17   2m
```

Two pod IPs, port 80. `kubectl --context rke2 -n dbg-07 exec client -- curl -s http://backend` answers with `Hostname: backend-5b8d9f6c7d-...`. If you call it several times, the name switches between the two pods.

7.3 Endpoints Exist, Connection Still Refused

FILE `szenarien/07-netzwerk/kaputt-port.yaml (excerpt)`

```yaml
spec:
  selector:
    app: backend
  ports:
    - name: http
      port: 80
      targetPort: 8080      # ERROR: whoami listens on port 80
```

MAC — Step 6

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/kaputt-port.yaml
kubectl --context rke2 -n dbg-07 exec client -- curl -sS -m 5 http://backend
```

This time the selector is right, only the target port is wrong.

SYMPTOM

Again `curl: (7) Failed to connect to backend port 80 ... Could not connect to server`. But this time the EndpointSlice is filled:

```
NAME            ADDRESSTYPE   PORTS   ENDPOINTS               AGE
backend-7xqzl   IPv4          8080    10.42.0.21,10.42.1.17   4m
```

The `PORTS` column already gives the error away: the slice shows the port on which the **pod** is addressed, not the Service port. To be sure, ask the pod directly and bypass the Service:

MAC — Step 7

```
kubectl --context rke2 -n dbg-07 exec client -- curl -s -o /dev/null -w '%{http_code}\n'
http://10.42.0.21:80
```

Calls a pod IP from the slice directly on port 80 and prints only the HTTP status: `200`. The same call on port 8080 is refused.

You see which ports a container has really opened with `ss`. The whoami image contains no shell and no `ss`. But an ephemeral container with netshoot shares the network namespace of the pod and sees the same sockets:

MAC Step 8

```
POD=$(kubectl --context rke2 -n dbg-07 get pod -l app=backend -o name | head -n1)
kubectl --context rke2 -n dbg-07 debug -it "$POD" \
  --image=nicolaka/netshoot:v0.15 --profile=baseline --target=whoami -- ss -tlnp
```

`--target=whoami` also attaches the debug container to the process namespace of the container `whoami`, so `ss` shows the process name too. `--profile=baseline` is not needed on RKE2, but it is required on Talos: the default profile `general` adds `SYS_PTRACE`, and Talos' PSA `baseline` rejects that. In k9s, the plugin from Chapter 3 does the same step: `Shift-D` in the container view of the pod.

```
State  Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0      4096               *:80              *:*     users:
(("whoami",pid=1,fd=3))
```

CAUSE

`targetPort` is the port in the pod that kube-proxy forwards the packets to. Nobody listens there, so the pod's kernel answers with a TCP reset. The Service, the EndpointSlice and the pods are all "healthy". Only their agreement on the port is wrong. Named ports (`targetPort: http`) avoid this error, because the port number then lives in only one place: the pod template.

MAC Step 9

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/fix-service.yaml
```

Sets `targetPort` back to the named port `http`.

WHAT YOU SHOULD SEE

The EndpointSlice shows port `80`, and `curl -s http://backend` from the client pod returns the whoami response.

7.4 Names, Namespaces and ndots

DNS errors look different from connection errors: `curl: (6) Could not resolve host`. Before you suspect CoreDNS, look at how the pod resolves names.

MAC Step 10

```
kubectl --context rke2 -n dbg-07 exec client -- cat /etc/resolv.conf
```

The kubelet writes this file when the pod starts.

```
search dbg-07.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.43.0.10
options ndots:5
```

With `ndots:5`, every name with fewer than five dots counts as relative. The resolver then appends the search domains one after the other. `backend` first becomes `backend.dbg-07.svc.cluster.local`, and that exists. `backend.dbg07` (a typo, the hyphen is missing) becomes `backend.dbg07.dbg-07.svc.cluster.local`, then `backend.dbg07.svc.cluster.local`, then `backend.dbg07.cluster.local` and finally `backend.dbg07.` as an absolute name. All four queries end with `NXDOMAIN`.

MAC Step 11

```
kubectl --context rke2 -n dbg-07 exec client -- nslookup backend.dbg07
kubectl --context rke2 -n dbg-07 exec client -- nslookup backend.dbg-07
```

The first name is misspelled, the second is the correct short form `<service>.<namespace>`.

```
** server can't find backend.dbg07: NXDOMAIN
# ...
Name:   backend.dbg-07.svc.cluster.local
Address: 10.43.118.204
```

In practice, the most common case is not a typo but a missing namespace: an application in `dbg-07` calls `postgres`, but the Service lives in `datenbank`. The short name only works in your own namespace. Across namespaces you need at least `postgres.datenbank`. In the lab, CoreDNS itself is a good example, because its Service has a different name on each cluster:

MAC Step 12

```
kubectl --context rke2 -n kube-system get svc -l k8s-app=kube-dns
kubectl --context talos -n kube-system get svc -l k8s-app=kube-dns
```

Both distributions set the label `k8s-app=kube-dns`, but the names differ.

```
NAME                       TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)         AGE
rke2-coredns-rke2-coredns  ClusterIP   10.43.0.10   <none>        53/UDP,53/TCP   3d
NAME       TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)                  AGE
kube-dns   ClusterIP   10.96.0.10   <none>        53/UDP,53/TCP,9153/TCP   3d
```

So a script that queries `kube-dns.kube-system` works on Talos and fails on RKE2 with `NXDOMAIN`. This book is about differences like that.

Investigating CoreDNS Itself

Only when `kubernetes.default` does not resolve either is the problem in CoreDNS or on the way there.

MAC Step 13

```
kubectl --context rke2 -n dbg-07 exec client -- dig +short kubernetes.default.svc.cluster.local
kubectl --context rke2 -n kube-system get pods -l k8s-app=kube-dns -o wide
kubectl --context rke2 -n kube-system logs deploy/rke2-coredns-rke2-coredns --tail=20
```

First the function, then the pods, then their logs. `dig +short` returns only the address, on RKE2 `10.43.0.1`.

In k9s: `:pods kube-system⏎`, then `/coredns⏎`, `l` for the logs and `d` for restarts and probes.

Messages like `[ERROR] plugin/errors: 2 example.org. A: read udp ...: i/o timeout` mean that CoreDNS cannot reach its upstream server. This only affects external names. CoreDNS answers internal names like `backend.dbg-07.svc.cluster.local` itself, from the Kubernetes API.

RKE2

Deployment and Service are called `rke2-coredns-rke2-coredns`, and the Corefile is in the ConfigMap of the same name. You do not make changes there, but through a `HelmChartConfig` named `rke2-coredns` in `kube-system`. Otherwise the Helm controller overwrites them. Service IP `10.43.0.10`, the upstream is the node's `/etc/resolv.conf`.

TALOS

Deployment `coredns`, Service `kube-dns`, ConfigMap `coredns`, Service IP `10.96.0.10`. For new clusters, Talos enables `forwardKubeDNSToHost`: CoreDNS sends external queries to the node's DNS cache at `169.254.116.108`. You then look for upstream errors on the node: `talosctl -n 172.16.0.171 get dnsupstream` shows whether the upstream servers are reachable, and `talosctl -n 172.16.0.171 logs dns-resolve-cache` shows the cache logs.

7.5 Default Deny Blocks DNS

A NetworkPolicy that selects everything with `podSelector: {}` and lists `Egress` blocks all outgoing traffic that no other policy makes an exception for. That includes DNS.

FILE `szenarien/07-netzwerk/kaputt-policy.yaml (excerpt)`

```yaml
kind: NetworkPolicy
metadata:
  name: default-deny
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
---
kind: NetworkPolicy
metadata:
  name: client-zu-backend
spec:
  podSelector:
    matchLabels:
      app: client
  policyTypes:
    - Egress
  egress:
    - to:
```

```
        - podSelector:
            matchLabels:
              app: backend
      ports:
        - protocol: TCP
          port: 80
# ... plus backend-von-client: ingress to backend only from client
```

The idea was: everything is closed, only the client may reach the backend. What was forgotten: the client first has to resolve the name `backend`.

MAC Step 14

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/kaputt-policy.yaml
kubectl --context rke2 -n dbg-07 exec client -- curl -sS -m 10 http://backend
```

Applies the three policies and tests again with `curl`.

SYMPTOM

```
curl: (6) Could not resolve host: backend
command terminated with exit code 6
```

The message appears after a few seconds of waiting. The call via the Service IP, for example `curl -s http://10.43.118.204`, on the other hand works right away.

This pattern is clear: the IP works, the name does not. To confirm it, ask the DNS server directly:

MAC Step 15

```
kubectl --context rke2 -n dbg-07 exec client -- dig +time=2 +tries=1 backend.dbg-07.svc.cluster.local
kubectl --context rke2 -n dbg-07 get networkpolicy
```

Without an answer, `dig` reports a timeout. Then you list all policies in the namespace.

```
;; communications error to 10.43.0.10#53: timed out
;; no servers could be reached
```

```
NAME                 POD-SELECTOR   AGE
backend-von-client   app=backend    1m
client-zu-backend    app=client     1m
default-deny         <none>         1m
```

`<none>` for `default-deny` means: empty selector, the policy applies to every pod in the namespace. With `kubectl describe networkpolicy client-zu-backend` you see the allowed targets spelled out. Port 53 does not show up anywhere. In k9s, open `:netpol dbg-07⏎` and press `d` on the highlighted policy.

CAUSE

NetworkPolicies are allowlists, and they add up. As soon as any policy selects a pod for `Egress`, all outgoing traffic that no policy explicitly allows is blocked for that pod. The query to CoreDNS is outgoing traffic to a pod in `kube-system` on UDP and TCP port 53. Because the policy applies after the Service IP has been translated to the pod IP, the rule must select the CoreDNS pods, not the Service IP.

FILE `szenarien/07-netzwerk/fix-policy.yaml (excerpt)`

```yaml
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
```

`namespaceSelector` and `podSelector` are in the **same** list entry, so both apply at once: CoreDNS pods in `kube-system`. With a hyphen in front of `podSelector` they would be two entries, and the rule would allow all pods in `kube-system` **or** all pods with `k8s-app=kube-dns` in `dbg-07`. Kubernetes sets the label `kubernetes.io/metadata.name` on every namespace automatically.

MAC — Step 16

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/fix-policy.yaml
kubectl --context rke2 -n dbg-07 exec client -- curl -s -m 10 http://backend
```

Adds a fourth policy that allows DNS for all pods in the namespace.

WHAT YOU SHOULD SEE

`curl` returns the whoami response right away. The cross-check shows that everything else stays closed: `kubectl --context rke2 -n dbg-07 exec client -- curl -sS -m 5 https://kubernetes.io` ends with a timeout, because `client` may only reach `backend`.

PITFALL

Do not forget TCP port 53. DNS falls back to TCP when an answer does not fit into a UDP packet. With UDP only, most queries work, but some large answers fail now and then, and troubleshooting becomes tedious.

7.6 The Policy That Does Nothing

Now the same on Talos. Apply the broken policy, this time expecting DNS to fail:

MAC — Step 17

```
kubectl --context talos apply -f szenarien/07-netzwerk/kaputt-selektor.yaml \
  -f szenarien/07-netzwerk/fix-service.yaml -f szenarien/07-netzwerk/kaputt-policy.yaml
kubectl --context talos -n dbg-07 exec client -- curl -s -m 5 http://backend
kubectl --context talos -n dbg-07 exec client -- curl -sS -m 5 -o /dev/null -w '%{http_code}\n' https://
kubernetes.io
```

Rebuilds the starting point of this chapter and tests both the allowed and a blocked path. During `apply` you see warnings `would violate PodSecurity "restricted:latest"`. These are only warnings, Talos enforces `baseline`.

SYMPTOM

Both calls work. `curl http://backend` returns the whoami response, and the call to the internet returns an HTTP status instead of a timeout. The policies exist, `kubectl get networkpolicy -n dbg-07` lists all three. There is no error and no warning.

The Kubernetes API stores NetworkPolicies, but it does not enforce them itself. The CNI plugin does that. `kubectl --context talos -n kube-system get ds` shows which CNI is running: on Talos you find `kube-flannel`, on RKE2 `rke2-canal`.

CAUSE

Talos installs Flannel by default. Flannel only builds the overlay network between the nodes and knows nothing about NetworkPolicies. The API still accepts the objects, because they are part of core Kubernetes. Nobody evaluates them. Canal on RKE2 combines Flannel for the overlay with the policy part of Calico (Felix), which turns the rules into iptables rules on every node.

Here the solution is not a patch but an architecture decision. On Talos you set `cluster.network.cni.name: none` when you generate the machine config, and then install Cilium or Calico yourself. On a running cluster, switching the CNI is a rebuild with downtime, not a quick fix. That is why Talos stays on Flannel in the lab, and this scenario stays a demonstration.

RKE2

Canal enforces NetworkPolicies. Felix writes the rules into iptables chains with the prefix `cali-`. On the node, `sudo iptables-save | grep -c cali-` gives you a rough count. A new policy takes effect after a few seconds.

TALOS

Flannel ignores NetworkPolicies completely. If you need policies on Talos, you have to replace the CNI when you build the cluster (`cni.name: none` plus Cilium or Calico). The Talos docs describe both options.

WHY IT MATTERS

Test every policy with a cross-check that **must** fail. A green test of the allowed path only proves that the policy does not block too much. Only the blocked path shows you whether it blocks anything at all. On Talos you would never have noticed the problem otherwise.

MAC

Step 18

```
kubectl --context rke2 delete ns dbg-07
kubectl --context talos delete ns dbg-07
```

Cleans up both clusters. Policies, Services and pods disappear with the namespace.

TRY IT YOURSELF

Apply `kaputt-selektor.yaml` and `fix-service.yaml` on RKE2 again. Then add a second label `version: v2` to the selector of the Service `backend` (`kubectl edit svc backend` or `e` in k9s). What does the EndpointSlice show, and what does `curl` show? How do you get the Service filled again without taking the selector back?

Hint: the labels of a selector are combined with AND. No pod carries `version: v2`, so the slice becomes empty. Add the label to the pod template of the Deployment (not to the Deployment's `selector`, which is immutable). After the rollout, the slice fills up again.

WHAT YOU LEARNED

- Check the EndpointSlice first: a Service without endpoints is a selector or readiness problem, not a network problem.
- The `PORTS` column of the EndpointSlice shows the port in the pod. Named ports prevent mismatches between Service and pod.
- `ndots:5` and the search domains explain why short names only work in your own namespace. CoreDNS is called `rke2-coredns-rke2-coredns` on RKE2, and `coredns` or `kube-dns` on Talos.
- A default deny policy with `Egress` needs an exception for UDP and TCP 53 to the CoreDNS pods.
- NetworkPolicies only work if the CNI enforces them: Canal on RKE2 does, Flannel on Talos does not. Every policy needs a cross-check that must fail.

CHAPTER 8

Ingress, Gateway and TLS

You find the ingress controller on both clusters, read its logs and tell 404, 503 and the wrong certificate apart by their cause. On Talos you install Traefik yourself, so that both clusters behave the same.

Time: 75 minutes **Requires:** Chapters 2 and 7

8.1 What Breaks in This Chapter

An Ingress is only a description. An ingress controller turns it into routing rules. In this book, that is Traefik on both clusters. So between your `curl` and the pod there are three questions: Does the controller consider this Ingress its own? Does it find the host and path? Does the target have ready pods on the right port? With HTTPS, the certificate comes on top.

Scenario	Symptom	Traefik log
Wrong `ingressClassName`	`404 page not found`	nothing
Wrong Service port in the Ingress	`404 page not found`	`Cannot create service`
No ready pods	`503 Service Unavailable`	Access log with 503
TLS Secret with wrong keys	Certificate `TRAEFIK DEFAULT CERT`	`Error configuring TLS`
Expired certificate	`certificate has expired` in the client	nothing

The column on the right is the key: Traefik reports some errors loudly and others not at all. The silence itself is a clue.

8.2 Traefik on Both Clusters

RKE2: Included

Since RKE2 1.36, Traefik is the default ingress controller for new clusters. In the lab, `ingress-controller: traefik` is also set explicitly in `config-cp.yaml` (Chapter 2). ingress-nginx is still available as an option in 1.36, but it is deprecated and will be removed in 1.37. RKE2 installs Traefik through its Helm controller as the chart `rke2-traefik` in `kube-system`.

MAC Step 1

```
kubectl --context rke2 -n kube-system get helmchart rke2-traefik
kubectl --context rke2 -n kube-system get ds rke2-traefik -o wide
kubectl --context rke2 get ingressclass
```

Shows the HelmChart object, the DaemonSet and the IngressClass.

```
NAME           DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR
AGE
rke2-traefik   2         2         2       2            2           kubernetes.io/
os=linux   3d
NAME      CONTROLLER                       PARAMETERS   AGE
traefik   traefik.io/ingress-controller    <none>       3d
```

The RKE2 chart differs from the upstream chart in three ways: Traefik runs as a **DaemonSet**, it binds the **hostPorts** 80 and 443 on every node, and the Service is only of type `ClusterIP`. So you reach Traefik directly through the node IPs `172.16.0.160` and `172.16.0.161`, without a load balancer. The IngressClass is called `traefik`, and it is the default class as long as Traefik is the selected controller.

You never set your own values for a bundled chart with `helm upgrade`. You use a `HelmChartConfig` with the same name instead. For troubleshooting in this chapter, turn on the access logs and the dashboard:

FILE `werkzeuge/rke2-traefik-config.yaml`

```
apiVersion: helm.cattle.io/v1
kind: HelmChartConfig
metadata:
  name: rke2-traefik
  namespace: kube-system
spec:
  valuesContent: |-
    accessLog:
      enabled: true
    ingressRoute:
      dashboard:
        enabled: true
```

The Helm controller merges these values with the RKE2 defaults and rolls out the DaemonSet again. You get the same result if you put the file into `/var/lib/rancher/rke2/server/manifests/` on `rke2-cp`.

MAC Step 2

```
kubectl --context rke2 apply -f werkzeuge/rke2-traefik-config.yaml
kubectl --context rke2 -n kube-system rollout status ds/rke2-traefik
```

The Helm controller starts a job `helm-install-rke2-traefik`. If it fails, for example because of a YAML error in `valuesContent`, you find the reason in `kubectl logs job/helm-install-rke2-traefik -n kube-system`.

Talos: Install It Yourself

Talos does not ship an ingress controller. You install Traefik from the official chart, with values that recreate the RKE2 behavior.

FILE `werkzeuge/traefik-values-talos.yaml (excerpt)`

```yaml
deployment:
  kind: DaemonSet
ingressClass:
  enabled: true
  isDefaultClass: true
  name: traefik
ports:
  web:
    hostPort: 80
  websecure:
    hostPort: 443
service:
  spec:
    type: ClusterIP
ingressRoute:
  dashboard:
    enabled: true
accessLog:
  enabled: true
```

Same class, same ports, same behavior as on RKE2. Internally, Traefik listens on 8000 and 8443 and runs as UID 65532. The `portmap` plugin of the CNI maps them to 80 and 443.

MAC — Step 3

```
kubectl --context talos create namespace traefik
kubectl --context talos label namespace traefik pod-security.kubernetes.io/enforce=privileged
helm --kube-context talos upgrade --install traefik traefik/traefik \
  --version 42.3.1 -n traefik -f werkzeuge/traefik-values-talos.yaml --wait
```

The label is needed because Talos enforces `baseline` cluster-wide, and `baseline` does not allow hostPorts. Without the label, the API accepts the DaemonSet, but the controller cannot create any pods.

PITFALL

If the label is missing, `helm --wait` only reports a timeout. You find the reason on the DaemonSet, not on the pod, because there is no pod: `kubectl --context talos -n traefik describe ds traefik` shows the event `FailedCreate` with `violates PodSecurity "baseline:latest": hostPort (container "traefik" uses hostPorts 80, 443)`. More on this in Chapter 10.

WHAT YOU SHOULD SEE

`curl -s http://172.16.0.171/` and `curl -s http://172.16.0.161/` both answer with `404 page not found`. Here that is a good sign: Traefik is running and answering, there is just no route yet.

8.3 404: The Controller Does Not Feel Responsible

FILE `szenarien/08-ingress/kaputt-klasse.yaml (excerpt)`

```yaml
kind: Ingress
metadata:
  name: shop
  namespace: dbg-08
spec:
  ingressClassName: nginx     # ERROR: copied from an old tutorial
  rules:
    - host: shop.dbg08.example
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: shop
                port:
                  number: 80
```

The file also creates the namespace `dbg-08`, the Deployment `shop` with two whoami pods and the Service `shop`. The error is realistic: countless tutorials and charts still set `nginx` as the class.

MAC Step 4

```
kubectl --context rke2 apply -f szenarien/08-ingress/kaputt-klasse.yaml
curl -s --resolve shop.dbg08.example:80:172.16.0.161 http://shop.dbg08.example/
```

`--resolve` makes `shop.dbg08.example` point to `172.16.0.161` for this call. That way you do not need a DNS entry, and `curl` still sends the right `Host` header and, with HTTPS, the right SNI name.

SYMPTOM

`404 page not found`

The pods are running, the Service has endpoints, the Ingress exists. Still, Traefik answers with its default 404 page.

A 404 from Traefik means: no route matches this request. The Ingress tells you whether that is because of the host, the path or the missing route itself:

MAC Step 5

```
kubectl --context rke2 -n dbg-08 get ingress shop \
  -o custom-columns=NAME:.metadata.name,CLASS:.spec.ingressClassName,HOSTS:.spec.rules[*].host
kubectl --context rke2 get ingressclass
```

Compares the class of the Ingress with the classes that exist in the cluster.

```
NAME   CLASS   HOSTS
shop   nginx   shop.dbg08.example
NAME      CONTROLLER                      PARAMETERS   AGE
traefik   traefik.io/ingress-controller   <none>       3d
```

There is no IngressClass `nginx`. The Traefik logs stay empty, and there is no event on the Ingress. In k9s you check this with `:ingress dbg-08⏎` and `d`: the line `Ingress Class:` shows `nginx`, and `Events` says `<none>`.

CAUSE

Traefik only processes Ingresses whose `ingressClassName` points to an IngressClass with the controller `traefik.io/ingress-controller`, or Ingresses without a class if its own class is the default class. Traefik silently skips an Ingress with a foreign class, because it might belong to another controller. That is why there is no log and no event, and the request simply has no route.

MAC Step 6

```
kubectl --context rke2 apply -f szenarien/08-ingress/fix-ingress.yaml
curl -s --resolve shop.dbg08.example:80:172.16.0.161 http://shop.dbg08.example/ | head -n2
```

Sets `ingressClassName: traefik`. The class is a normal field and can be changed. You do not have to recreate the Ingress.

WHAT YOU SHOULD SEE

```
Hostname: shop-6c9b7d5f48-r8wzn
IP: 127.0.0.1
```

A call without `--resolve`, so `curl http://172.16.0.161/`, still returns 404. The Ingress requires the host `shop.dbg08.example`, and without it the rule does not match. This is also a common cause of a 404.

8.4 404 Despite the Right Class

FILE `szenarien/08-ingress/kaputt-port.yaml (excerpt)`

```
spec:
  ingressClassName: traefik
  rules:
    - host: shop.dbg08.example
      http:
        paths:
```

```
          - path: /
            pathType: Prefix
            backend:
              service:
                name: shop
                port:
                  number: 8080   # ERROR: the Service only has port 80
```

MAC Step 7

```
kubectl --context rke2 apply -f szenarien/08-ingress/kaputt-port.yaml
curl -s --resolve shop.dbg08.example:80:172.16.0.161 http://shop.dbg08.example/
```

Again you get `404 page not found`. But this time Traefik is responsible.

SYMPTOM

Same answer as in the previous scenario. From the outside, you cannot tell the two errors apart. Only the Traefik log separates them.

MAC Step 8

```
kubectl --context rke2 -n kube-system logs -l app.kubernetes.io/name=rke2-traefik \
  --prefix --since=5m | grep -v '"GET'
```

`-l` reads the logs of all Traefik pods, `--prefix` puts the pod name in front, and `grep -v` hides the access log lines. On Talos the label is `app.kubernetes.io/name=traefik` in the namespace `traefik`.

```
[pod/rke2-traefik-8kq4d/rke2-traefik] 2026-11-09T19:42:07Z ERR Cannot create service
  error="service port not found" ingress=shop namespace=dbg-08 providerName=kubernetes
  serviceName=shop servicePort=&ServiceBackendPort{Name:,Number:8080,}
```

The line repeats with every configuration change in the cluster. In k9s you find it with `:pods kube-system⏎`, `/traefik⏎`, `l`, and then `/` with the search term `ERR` in the log window.

CAUSE

Traefik looks for a port with the number 8080 or a matching name in the Service `shop`. There is none, so Traefik cannot build a backend service and drops the whole path. Without the path there is no route, and the request ends at the default 404. The Kubernetes API does not check this cross-reference when you create the Ingress. Remember: `service.port` in the Ingress means the **Service** port, not the container port.

MAC Step 9

```
kubectl --context rke2 apply -f szenarien/08-ingress/fix-ingress.yaml
```

Port 80, as in the Service.

WHAT YOU SHOULD SEE

`curl` with `--resolve` returns the whoami response again, and no new `ERR` lines show up in the log.

8.5 503: The Route Exists, but Nobody Is Ready

FILE `szenarien/08-ingress/kaputt-bereit.yaml (excerpt)`

```
      containers:
        - name: whoami
          image: traefik/whoami:v1.11.0
          readinessProbe:
            tcpSocket:
              port: 8080         # ERROR: whoami listens on port 80
            periodSeconds: 5
```

MAC

Step 10

```
kubectl --context rke2 -n dbg-08 delete deploy shop
kubectl --context rke2 apply -f szenarien/08-ingress/kaputt-bereit.yaml
kubectl --context rke2 -n dbg-08 get pods
curl -s -i --resolve shop.dbg08.example:80:172.16.0.161 http://shop.dbg08.example/ | head -n1
```

Deletes the old Deployment first. With a normal update, the old, ready pods would stay, because the rollout would get stuck on the new ones (Chapter 15), and you would not see the error from the outside.

SYMPTOM

```
NAME                    READY   STATUS    RESTARTS   AGE
shop-7f4d8c9b6d-2xkqv   0/1     Running   0          2m
shop-7f4d8c9b6d-tn5wz   0/1     Running   0          2m
HTTP/1.1 503 Service Unavailable
```

Traefik knows the route and the Service, but the Service has no ready endpoints. With `allowEmptyServices`, which both charts turn on, Traefik then answers with 503 instead of 404. The access log confirms it:

```
172.16.0.1 - - [09/Nov/2026:19:51:33 +0000] "GET / HTTP/1.1" 503 19 "-" "curl/8.7.1"
41
  "dbg-08-shop-shop-dbg08-example@kubernetes" "-" 0ms
```

The second-to-last field shows the server URL that Traefik forwarded to. `"-"` means: there was no server. So the cause is not in the Ingress but one layer further down. From here on, the chain from Chapter 7 applies: look at the EndpointSlice, then at the readiness of the pods.

MAC

Step 11

```
kubectl --context rke2 -n dbg-08 events --types=Warning
```

The probe events are attached to the pods, not to the Deployment. They say `Readiness probe failed: dial tcp 10.42.1.31:8080: connect: connection refused`. In k9s: `:events dbg-08⏎`.

CAUSE

The readiness probe checks a port where nobody listens. The pods are running, but they never count as ready. The EndpointSlice controller adds them with `ready: false`. Traefik only takes ready endpoints, so it has no server for the Service.

MAC Step 12

```
kubectl --context rke2 apply -f szenarien/08-ingress/fix-bereit.yaml
kubectl --context rke2 -n dbg-08 rollout status deploy/shop
```

The probe now checks the path `/health` via HTTP on the named port `http`. whoami answers it with `200`.

WHAT YOU SHOULD SEE

Both pods show `1/1`, `curl -i` returns `HTTP/1.1 200 OK`, and the access log shows a server URL like `"http://10.42.1.34:80"`.

Status	What it means in Traefik
404	No route: wrong class, wrong host or path, backend service cannot be built
503	Route exists, but no ready endpoints
502	Endpoint exists, but the connection to the pod fails, for example a wrong `targetPort` (Chapter 7)
504	Pod accepts the connection but does not answer in time

8.6 TLS: The Wrong Certificate

For HTTPS, the Ingress needs a Secret with a certificate and a key. Because keys do not belong in the repo, you create them with a script.

The script `szenarien/08-ingress/zertifikat.sh` creates a lab CA once and uses it to sign a certificate for `shop.dbg08.example`. Pass `gueltig` (valid) for a certificate that is valid for 30 days, or `abgelaufen` (expired) for one that expired in February 2025. The files end up in `szenarien/08-ingress/tls/`, which `.gitignore` keeps out of the repo. For the expired certificate, the script uses `openssl x509 -not_before/-not_after`, which only exists since OpenSSL 3.4. The LibreSSL that comes with macOS is not enough, so use OpenSSL from Homebrew.

Secret with the Wrong Key Names

MAC Step 13

```
szenarien/08-ingress/zertifikat.sh gueltig
kubectl --context rke2 -n dbg-08 create secret generic shop-tls \
  --from-file=cert.pem=szenarien/08-ingress/tls/shop.crt \
  --from-file=key.pem=szenarien/08-ingress/tls/shop.key
kubectl --context rke2 apply -f szenarien/08-ingress/ingress-tls.yaml
```

A generic Secret with the file names as keys. For type `Opaque`, the API checks nothing. `ingress-tls.yaml` is the Ingress from `fix-ingress.yaml` plus a `tls` section with `secretName: shop-tls`.

MAC Step 14

```
curl -sS --cacert szenarien/08-ingress/tls/ca.crt \
  --resolve shop.dbg08.example:443:172.16.0.161 https://shop.dbg08.example/
openssl s_client -connect 172.16.0.161:443 -servername shop.dbg08.example </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -issuer
```

First the call that verifies against the lab CA, then a look at the certificate that Traefik actually serves.

SYMPTOM

```
curl: (60) SSL certificate problem: self-signed certificate
subject=CN=TRAEFIK DEFAULT CERT
issuer=CN=TRAEFIK DEFAULT CERT
```

Traefik serves its own self-generated default certificate, not the one from the Secret. With `curl -k` the page works, and that is exactly why the error often shows up late in the browser.

MAC Step 15

```
kubectl --context rke2 -n kube-system logs -l app.kubernetes.io/name=rke2-traefik \
  --since=5m | grep 'configuring TLS'
kubectl --context rke2 -n dbg-08 get secret shop-tls -o jsonpath='{.type}{"\n"}'
```

Traefik names the error word for word. The Secret reveals its type, and `kubectl describe secret shop-tls` shows the keys `cert.pem` and `key.pem`.

```
2026-11-09T20:03:12Z ERR Error configuring TLS error="secret dbg-08/shop-tls is
missing
  the following TLS data entries: tls.crt, tls.key" ingress=shop namespace=dbg-08
providerName=kubernetes
Opaque
```

CAUSE

Ingress controllers expect a Secret of type `kubernetes.io/tls` with exactly the keys `tls.crt` and `tls.key`. Traefik does not find the keys, discards the certificate and uses the default certificate for the host. The route itself stays active, which is why HTTPS keeps working with `-k`.

MAC Step 16

```
kubectl --context rke2 -n dbg-08 delete secret shop-tls
kubectl --context rke2 -n dbg-08 create secret tls shop-tls \
  --cert=szenarien/08-ingress/tls/shop.crt --key=szenarien/08-ingress/tls/shop.key
```

`create secret tls` sets the type and keys correctly. For this type, the API rejects a Secret without `tls.crt` or `tls.key` right away. Traefik notices the new Secret on its own, no restart needed.

WHAT YOU SHOULD SEE

`curl --cacert ... https://shop.dbg08.example/` returns the whoami response without `-k`, and `openssl s_client` shows `subject=CN=shop.dbg08.example` and `issuer=CN=dbg-08 Labor-CA`.

Expired Certificate

MAC Step 17

```
szenarien/08-ingress/zertifikat.sh abgelaufen
kubectl --context rke2 -n dbg-08 create secret tls shop-tls \
  --cert=szenarien/08-ingress/tls/shop.crt --key=szenarien/08-ingress/tls/shop.key \
  --dry-run=client -o yaml | kubectl --context rke2 apply -f -
curl -sS --cacert szenarien/08-ingress/tls/ca.crt \
  --resolve shop.dbg08.example:443:172.16.0.161 https://shop.dbg08.example/
```

Replaces the Secret with the expired certificate. `--dry-run=client -o yaml | apply` is the usual pattern for overwriting an existing Secret.

SYMPTOM

```
curl: (60) SSL certificate problem: certificate has expired
```

Traefik logs nothing. For Traefik, the Secret is formally fine, and it serves the certificate.

Here only a look at the served certificate itself helps. `openssl s_client ... | openssl x509 -noout -subject -dates` shows `notAfter=Feb  1 00:00:00 2025 GMT`. The same date is in the Secret, which you can check without the network:

MAC Step 18

```
kubectl --context rke2 -n dbg-08 get secret shop-tls -o jsonpath='{.data.tls\.crt}' \
  | base64 -d | openssl x509 -noout -enddate -checkend 0
```

`-checkend 0` checks whether the certificate is still valid right now and answers with `Certificate will expire` and exit code 1. With `-checkend 1209600` you check the next 14 days. That works well for a cron job or an alert rule.

CAUSE

Neither Kubernetes nor Traefik checks how long a certificate is valid. That is the client's job. Without cert-manager or another renewal mechanism, every certificate silently expires at some point, and the first sign is an error on the user's side.

To fix it, create a new certificate from the same CA with `zertifikat.sh gueltig` and replace the Secret with the same `--dry-run=client -o yaml | apply` pattern.

WHAT YOU SHOULD SEE

`curl --cacert ...` returns the whoami response again, and `-checkend 0` reports `Certificate will not expire`.

RKE2

All scenarios work as shown. Traefik pods are in the namespace `kube-system`, label `app.kubernetes.io/name=rke2-traefik`. You change values through the `HelmChartConfig` `rke2-traefik`, never with `helm upgrade`, otherwise the Helm controller overwrites your change.

TALOS

Replace `--context rke2` with `--context talos` and `172.16.0.161` with `172.16.0.171`. Traefik pods are in the namespace `traefik`, label `app.kubernetes.io/name=traefik`. You change values with `helm upgrade` and `werkzeuge/traefik-values-talos.yaml`.

8.7 The Dashboard and the Gateway API

The Traefik dashboard shows all routers, services and middlewares that Traefik has built from the Ingresses, including errors and warnings. It runs on the internal entrypoint `traefik` (port 8080) and cannot be reached from the outside:

MAC — Step 19

```
kubectl --context rke2 -n kube-system port-forward ds/rke2-traefik 8080:8080
```

Then open `http://localhost:8080/dashboard/` in your browser. On Talos the command is `kubectl --context talos -n traefik port-forward ds/traefik 8080:8080`. In k9s you get the same with `Shift-f` on a Traefik pod.

An Ingress that is missing from the dashboard under **HTTP Routers** was not processed.

The Gateway API is the successor of Ingress: `GatewayClass` and `Gateway` belong to the platform team, `HTTPRoute` belongs to the application team. RKE2 1.36 ships the Gateway API CRDs together with Traefik, but Traefik's Gateway provider is off. You turn it on with `providers.kubernetesGateway.enabled: true` in the `HelmChartConfig`. Starting with RKE2 1.37, the CRDs

come from a separate chart. On Talos you install them yourself. For troubleshooting, the Gateway API is nicer because it sets status conditions: `kubectl describe httproute` shows `Accepted` and `ResolvedRefs` with reasons like `BackendNotFound` or `NotAllowedByListeners` instead of a silent 404.

MAC Step 20

```
kubectl --context rke2 delete ns dbg-08
rm -f szenarien/08-ingress/tls/*.crt szenarien/08-ingress/tls/*.key szenarien/08-ingress/tls/*.srl
```

Cleans up the namespace and deletes the generated certificates and keys.

TRY IT YOURSELF

Repeat the scenario from “404: The Controller Does Not Feel Responsible” on Talos, but remove the `ingressClassName` line completely instead of setting it to `traefik`. Does the Ingress work? Why?

Hint: an Ingress without a class gets the default class when it is created, which is the IngressClass with the annotation `ingressclass.kubernetes.io/is-default-class: "true"`. In `traefik-values-talos.yaml`, `isDefaultClass: true` sets exactly this annotation. If there is no default class, the field stays empty, and whether a controller feels responsible depends on its configuration. So always set the class explicitly.

WHAT YOU LEARNED

- RKE2 1.36 ships Traefik as a DaemonSet with hostPorts 80/443 and the IngressClass `traefik`. You set values through the `HelmChartConfig` `rke2-traefik`. On Talos you install the same behavior with Helm, in a namespace with PSA `privileged`.
- `curl --resolve` tests host rules and SNI without a DNS entry.
- 404 without a log: Traefik is not responsible (class), or host and path do not match. 404 with `Cannot create service`: the Ingress points to a port that the Service does not have.
- In Traefik, 503 means: the route exists, but there are no ready endpoints. Continue with the EndpointSlice and readiness.
- `TRAEFIK DEFAULT CERT` points to an unusable TLS Secret. Only the client reports expired certificates, and you can check them with `openssl x509 -checkend`.

CHAPTER 9

Storage

You read the events of PVC, PV and provisioner, see why a volume is not created or not mounted, and find a volume's data on the node, on RKE2 via SSH and on Talos with `talosctl`.

Time: 75 minutes **Requires:** Chapter 2 (StorageClass `local-path`) and Chapter 4

9.1 What Breaks in This Chapter

Both lab clusters use the local-path-provisioner from Chapter 2. For every PersistentVolumeClaim, it creates a directory on exactly one node and a matching PersistentVolume of type `hostPath` that is bound to this node. That is simple and fast, but it has quirks that look different in production with network storage. This chapter covers both.

Scenario	Symptom	Where you see it
Wrong StorageClass name	PVC and pod `Pending`	Events on the PVC
Provisioner without `privileged` (Talos)	PVC `Pending`, `ProvisioningFailed`	Events on the PVC
RWO volume, two nodes	Second replica `Pending`	Events on the pod
Wrong permissions in the volume	`Permission denied`, CrashLoopBackOff	Pod logs
emptyDir overflows	Pod is evicted, a new one starts	Status and events

The order of the investigation is always the same: pod, then PVC, then PV, then provisioner, then node. `kubectl get pod,pvc,pv` shows the first three stops, the provisioner log in `local-path-storage` shows the fourth.

WHY IT MATTERS

The StorageClass `local-path` uses `volumeBindingMode: WaitForFirstConsumer`. So a new PVC stays `Pending` until a pod uses it, and the event says `waiting for first consumer to be created before binding`. That is not an error. Only when the scheduler has picked a node for the pod does the provisioner create the volume, on exactly that node. So a `Pending` PVC is only suspicious when a pod is already waiting for it.

9.2 The PVC Cannot Find Its StorageClass

FILE szenarien/09-storage/kaputt-klasse.yaml (excerpt)

```
kind: PersistentVolumeClaim
metadata:
  name: daten
  namespace: dbg-09
spec:
  storageClassName: local-paht    # ERROR: typo
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

The file also creates the namespace `dbg-09` and a pod `app` that mounts the PVC at `/data`.

MAC — Step 1

```
kubectl --context rke2 apply -f szenarien/09-storage/kaputt-klasse.yaml
kubectl --context rke2 -n dbg-09 get pod,pvc
```

Creates the PVC and the pod and shows both.

SYMPTOM

```
NAME      READY   STATUS    RESTARTS   AGE
pod/app   0/1     Pending   0          35s

NAME                           STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
AGE
persistentvolumeclaim/daten   Pending                                        local-paht
35s
```

Both are stuck. The pod is waiting for the PVC, so the search starts at the PVC:

MAC — Step 2

```
kubectl --context rke2 -n dbg-09 describe pvc daten | tail -n 4
kubectl --context rke2 get storageclass
```

The last lines of `describe` are the events. Then the list of all StorageClasses.

```
  Type     Reason              Age               From                         Message
  ----     ------              ----              ----                         -------
  Warning  ProvisioningFailed  4s (x4 over 49s)  persistentvolume-controller
storageclass.storage.k8s.io "local-paht" not found
NAME         PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION   AGE
local-path   rancher.io/local-path   Delete          WaitForFirstConsumer   false
3d
```

The pod has the event `0/2 nodes are available: pod has unbound immediate PersistentVolumeClaims.` The scheduler treats a PVC with an unknown class like one with immediate binding that just has not been bound yet. In k9s you find the same events with `:pvc dbg-09⏎` and `d` on the PVC.

CAUSE

The PersistentVolume controller looks for the StorageClass `local-paht` to find the right provisioner. It does not exist, so no volume is ever created, and the PVC stays `Pending`. If `storageClassName` is missing entirely, you end up in a similar dead end: neither lab cluster has a default StorageClass, so the PVC waits for a matching, manually created PV (Chapter 4).

The obvious fix is a `kubectl edit`. The API rejects it:

```
error: persistentvolumeclaims "daten" is invalid: spec: Forbidden: spec is immutable
after
creation except resources.requests and volumeAttributesClassName for bound claims
```

The spec of a PVC cannot be changed after it is created. So you delete the pod and the PVC and create both again. In the lab you lose no data, because there never was a volume.

MAC Step 3

```
kubectl --context rke2 -n dbg-09 delete pod app
kubectl --context rke2 -n dbg-09 delete pvc daten
kubectl --context rke2 apply -f szenarien/09-storage/fix-klasse.yaml
kubectl --context rke2 -n dbg-09 get pvc daten
```

Delete the pod first, otherwise the finalizer `kubernetes.io/pvc-protection` holds on to the PVC. `fix-klasse.yaml` contains the PVC and the pod, this time with `storageClassName: local-path`.

WHAT YOU SHOULD SEE

```
NAME    STATUS   VOLUME                                     CAPACITY   ACCESS MODES
STORAGECLASS   AGE
daten   Bound    pvc-3f1c2a9e-8b7d-4e21-9c55-1d0a6b2e7f43   1Gi        RWO
local-path     12s
```

`kubectl --context rke2 -n dbg-09 events --for pvc/daten` shows the sequence: `WaitForFirstConsumer`, `ExternalProvisioning`, `Provisioning` and `ProvisioningSucceeded` from the provisioner `rancher.io/local-path`.

9.3 Talos: The Helper Pod Is Not Allowed to Start

The local-path-provisioner does not create the directory itself. Instead, it briefly starts a helper pod in the namespace `local-path-storage` that mounts the node's directory via `hostPath`. That is exactly what the Pod Security Standard `baseline` forbids, and Talos enforces it cluster-wide. That

is why you labeled the namespace with `pod-security.kubernetes.io/enforce=privileged` in Chapter 2. This scenario shows what happens without that label.

MAC Step 4

```
kubectl --context talos label namespace local-path-storage pod-security.kubernetes.io/enforce-
kubectl --context talos apply -f szenarien/09-storage/pvc-test.yaml
```

The minus at the end removes the label. After that, the Talos default `baseline` applies again. `pvc-test.yaml` only contains a PVC `probe` and a pod that uses it.

SYMPTOM

The pod and the PVC stay `Pending`. This time the StorageClass exists, and the `WaitForFirstConsumer` event is long over, because the pod has been assigned a node.

MAC Step 5

```
kubectl --context talos -n dbg-09 events --for pvc/probe
```

The events of the PVC, sorted by time.

```
LAST SEEN   TYPE      REASON                 OBJECT                         MESSAGE
40s         Normal    WaitForFirstConsumer   PersistentVolumeClaim/probe    waiting for
first consumer to be created before binding
38s         Normal    Provisioning           PersistentVolumeClaim/probe    External
provisioner is provisioning volume for claim "dbg-09/probe"
38s         Warning   ProvisioningFailed     PersistentVolumeClaim/probe    failed to
provision volume with StorageClass "local-path": failed to create volume pvc-8e2d...:
pods "helper-pod-create-pvc-8e2d..." is forbidden: violates PodSecurity
"baseline:latest": hostPath volumes (volume "data")
```

The message names everything: the helper pod, the standard `baseline` and the forbidden field. The provisioner retries with increasing delays. You see every attempt in the provisioner log:

MAC Step 6

```
kubectl --context talos -n local-path-storage logs deploy/local-path-provisioner --tail=20
```

In k9s: `:pods local-path-storage⏎`, highlight the pod, `l`. The line `create the helper pod helper-pod-create-pvc-... into local-path-storage` comes right before the error message.

CAUSE

Pod Security Admission checks every pod against the namespace's standard when the pod is created. The helper pod needs a `hostPath` volume to create a directory on the node, and `baseline` forbids `hostPath` completely. The API rejects the pod, and the provisioner cannot create the volume. The application itself is not affected: its pod only mounts a PVC, not a `hostPath`, so it may run in a `baseline` namespace.

MAC Step 7

```
kubectl --context talos label namespace local-path-storage \
  pod-security.kubernetes.io/enforce=privileged
kubectl --context talos -n dbg-09 get pvc probe -w
```

Sets the label again. The provisioner's next attempt succeeds, and `-w` shows the change to `Bound`. Stop it with `Ctrl-C`.

WHAT YOU SHOULD SEE

The PVC is `Bound`, the pod is `Running`. The node now has the directory that the provisioner created:

```
$ talosctl -n 172.16.0.171 ls /var/mnt/local-path-provisioner
NODE           NAME
172.16.0.171   .
172.16.0.171   pvc-8e2d4b1a-77c3-4f0e-a9d2-5b6c0e1f3a88_dbg-09_probe
```

If the pod runs on `talos-cp`, ask `-n 172.16.0.170` instead. `kubectl get pod probe -o wide` tells you the node.

RKE2

RKE2 without the CIS profile does not enforce any Pod Security Standard. The helper pod starts even without the label. With `profile: cis`, however, `restricted` applies cluster-wide, and you need an exemption for `local-path-storage` (Chapter 10). The volumes live under `/opt/local-path-provisioner`, the provisioner's default path.

TALOS

Talos enforces `baseline` in all namespaces except `kube-system`. The provisioner's namespace needs `enforce=privileged`. The volumes live in the user volume `local-path-provisioner` under `/var/mnt/local-path-provisioner` on the EPHEMERAL partition. `talosctl -n 172.16.0.171 get volumestatus u-local-path-provisioner` shows its state.

9.4 Two Replicas, One Volume, Two Nodes

FILE szenarien/09-storage/kaputt-rwo.yaml (excerpt)

```yaml
kind: Deployment
metadata:
  name: cache
spec:
  replicas: 2
  template:
    spec:
      affinity:
        podAntiAffinity:         # each replica on a different node
          requiredDuringSchedulingIgnoredDuringExecution:
            - topologyKey: kubernetes.io/hostname
              labelSelector:
                matchLabels:
                  app: cache
      # ...
      volumes:
        - name: daten
          persistentVolumeClaim:
            claimName: cache     # ERROR: both replicas share one RWO volume
```

The PVC `cache` has `accessModes: [ReadWriteOnce]`. The anti-affinity is meant to spread the replicas across both nodes, so that losing one node does not take out both.

MAC — Step 8

```
kubectl --context rke2 apply -f szenarien/09-storage/kaputt-rwo.yaml
kubectl --context rke2 -n dbg-09 get pods -l app=cache -o wide
```

`-o wide` shows which node each pod landed on.

SYMPTOM

```
NAME                     READY   STATUS    RESTARTS   AGE   IP           NODE      ...
cache-6f8d7b9c5d-hx2lp   1/1     Running   0          40s   10.42.1.41   rke2-w1   ...
cache-6f8d7b9c5d-q9m4z   0/1     Pending   0          40s   <none>       <none>    ...
```

MAC — Step 9

```
kubectl --context rke2 -n dbg-09 describe pod cache-6f8d7b9c5d-q9m4z | grep -A3 Events
```

The events of the waiting pod. In k9s: `:pods dbg-09⏎`, highlight the pending pod, `d`.

```
  Warning  FailedScheduling  25s  default-scheduler  0/2 nodes are available: 1
node(s) didn't match
  PersistentVolume's node affinity, 1 node(s) didn't match pod anti-affinity rules.
preemption: 0/2
  nodes are available: 2 Preemption is not helpful for scheduling.
```

Read the message node by node: `rke2-w1` is ruled out because a replica already runs there (anti-affinity). `rke2-cp` is ruled out because the PV only exists on `rke2-w1`. The node affinity

of the PV confirms this: `kubectl --context rke2 get pv -o yaml` shows the node `rke2-w1` under `spec.nodeAffinity`.

CAUSE

A local-path volume is a directory on exactly one node. That is why the PV carries a node affinity, and every pod that uses it has to run on that node. The anti-affinity demands the opposite. Both at once is impossible. By the way, `ReadWriteOnce` means "writable by one **node**", not "by one pod". Without the anti-affinity, both replicas would run on `rke2-w1` and share the directory, which would be just as wrong for most applications.

WHY IT MATTERS

With network storage such as Longhorn, Ceph RBD or cloud disks, the same error looks different. The scheduler places the second replica on the other node, but the volume is already attached to the first node. The pod stays in `ContainerCreating`, and the event says `Multi-Attach error for volume "pvc-..." Volume is already used by pod(s) ...`. The cause and the fix are the same. If you want to guarantee that really only one pod writes, use `ReadWriteOncePod`.

The fix: every replica gets its own volume. That is exactly what a StatefulSet with `volumeClaimTemplates` is for.

FILE `szenarien/09-storage/fix-rwo.yaml (excerpt)`

```yaml
kind: StatefulSet
metadata:
  name: cache
spec:
  serviceName: cache
  replicas: 2
  # ... same anti-affinity and same container
  volumeClaimTemplates:
    - metadata:
        name: daten
      spec:
        storageClassName: local-path
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 1Gi
```

The StatefulSet creates the PVCs `daten-cache-0` and `daten-cache-1`. Each one is only created when its pod has a node, and thanks to `WaitForFirstConsumer` it ends up on that node. The file also contains the headless Service that `serviceName` requires.

MAC Step 10

```
kubectl --context rke2 -n dbg-09 delete deploy cache
kubectl --context rke2 -n dbg-09 delete pvc cache
kubectl --context rke2 apply -f szenarien/09-storage/fix-rwo.yaml
kubectl --context rke2 -n dbg-09 get pods,pvc -l app=cache -o wide
```

Removes the old Deployment and the old PVC and creates the StatefulSet. The StatefulSet controller copies the selector's `matchLabels` onto every PVC it creates from `volumeClaimTemplates`. That is why `-l app=cache` finds the pods and the PVCs at once.

WHAT YOU SHOULD SEE

`cache-0` and `cache-1` run on different nodes, and `kubectl get pvc -n dbg-09` shows `daten-cache-0` and `daten-cache-1` as `Bound`. The PVCs stay when you delete the StatefulSet. That way the data even survives a `delete sts`.

9.5 Permission Denied on the Volume

FILE `szenarien/09-storage/kaputt-rechte.yaml (excerpt)`

```yaml
      initContainers:
        - name: vorbereiten       # runs as root and creates the directory
          image: busybox:1.37
          command: ["sh", "-c", "mkdir -p /data/notizen"]
      containers:
        - name: app
          image: busybox:1.37
          command: ["sh", "-c", "date >> /data/notizen/log.txt && sleep infinity"]
          securityContext:
            runAsUser: 1000
            runAsGroup: 1000
```

A typical pattern after hardening: the application no longer runs as root, but an older init container still does.

MAC Step 11

```
kubectl --context rke2 apply -f szenarien/09-storage/kaputt-rechte.yaml
kubectl --context rke2 -n dbg-09 get pods -l app=notizen
kubectl --context rke2 -n dbg-09 logs deploy/notizen -c app --previous
```

`--previous` shows the output of the last crashed container. In k9s: highlight the pod, `l`, then `p` for the previous logs.

SYMPTOM

```
NAME                       READY   STATUS             RESTARTS      AGE
notizen-7c8f9d6b5f-l4wqk   0/1     CrashLoopBackOff   3 (25s ago)   80s
sh: can't create /data/notizen/log.txt: Permission denied
```

Who owns the directory? A look at the node tells you. The PV says where the volume lives:

MAC Step 12

```
PV=$(kubectl --context rke2 -n dbg-09 get pvc notizen -o jsonpath='{.spec.volumeName}')
kubectl --context rke2 get pv "$PV" -o jsonpath='{.spec.hostPath.path}{"\n"}'
kubectl --context rke2 -n dbg-09 get pod -l app=notizen -o wide
```

The path of the directory on the node, and the node itself.

RKE2-W1 Step 13

```
sudo ls -ln /opt/local-path-provisioner/pvc-5a0e..._dbg-09_notizen/
```

On RKE2 you look directly via SSH. `-n` shows numeric UIDs.

```
drwxr-xr-x 2 0 0 4096 Nov  9 20:31 notizen
```

On Talos you ask the same with `talosctl -n 172.16.0.171 ls -l /var/mnt/local-path-provisioner/pvc-..._dbg-09_notizen`. The `UID` and `GID` columns also show `0`.

CAUSE

The provisioner creates the root directory of the volume with permissions `0777`, so anyone may write there. But the init container created the subdirectory `notizen` as root with `0755`. The application runs as UID 1000 and may not create anything in it. So the error is not in Kubernetes but in the file permissions on the volume.

The reflex would be `fsGroup: 1000` in the pod's `securityContext`. For many volume types, the kubelet then changes the group of all files when it mounts the volume. Not here:

PITFALL

By default, local-path creates PVs of type `hostPath`, and the kubelet does not apply `fsGroup` to `hostPath`. The pod starts with `fsGroup`, but the permissions on the volume stay the same, and so does the error. This costs a lot of time if you do not know it. With most CSI drivers, on the other hand, `fsGroup` works.

The reliable fix: whoever creates the directory hands it over to the application's user.

MAC Step 14

```
kubectl --context rke2 apply -f szenarien/09-storage/fix-rechte.yaml
kubectl --context rke2 -n dbg-09 rollout status deploy/notizen
```

The init container now runs `mkdir -p /data/notizen && chown -R 1000:1000 /data/notizen`. It still runs as root. That is fine for a one-off step, but it violates the `restricted` standard (Chapter 10).

WHAT YOU SHOULD SEE

The pod is `1/1 Running`, and `kubectl --context rke2 -n dbg-09 exec deploy/notizen -c app -- cat /data/notizen/log.txt` shows a timestamp. `ls -ln` on the node now shows `1000 1000` for `notizen`.

9.6 When Space Runs Out

Not all storage is a PVC. An `emptyDir` lives on the node's disk under `/var/lib/kubelet/pods/`, and the kubelet monitors its size. If a pod exceeds its limit, the kubelet removes it: the pod is **evicted**.

FILE `szenarien/09-storage/kaputt-platte.yaml (excerpt)`

```yaml
    containers:
      - name: export
        image: busybox:1.37
        command: ["sh", "-c", "dd if=/dev/zero of=/cache/export.tmp bs=1M count=80 && sleep infinity"]
        volumeMounts:
          - name: cache
            mountPath: /cache
    volumes:
      - name: cache
        emptyDir:
          sizeLimit: 50Mi        # ERROR: smaller than the intermediate data
```

MAC — Step 15

```
kubectl --context talos apply -f szenarien/09-storage/kaputt-platte.yaml
kubectl --context talos -n dbg-09 get pods -l app=export -w
```

This time on Talos. The behavior is the same on both clusters. After a few seconds, the first pod is evicted, and the Deployment starts a new one.

SYMPTOM

After a short time, the running pod disappears, a new one starts, and the old ones stay with `0/1`. Depending on the timing, their status is `Error`, `ContainerStatusUnknown` or `Evicted`. `RESTARTS` is 0 everywhere, because this is not a restart but a new pod.

MAC — Step 16

```
kubectl --context talos -n dbg-09 get pods -l app=export \
  -o custom-
columns=NAME:.metadata.name,PHASE:.status.phase,REASON:.status.reason,MESSAGE:.status.message
```

`status.reason` and `status.message` explain every eviction. In k9s: `:events dbg-09⏎`, where you see the reason `Evicted`.

```
NAME                      PHASE     REASON    MESSAGE
export-6d5c8f7b9d-9kq2x   Failed    Evicted   Usage of EmptyDir volume "cache" exceeds
```

```
the limit "50Mi".
export-6d5c8f7b9d-x7wlm   Running   <none>     <none>
```

CAUSE

`sizeLimit` does not limit an `emptyDir` the way a file system would. The process can keep writing, and `dd` reports no error. But the kubelet measures the usage regularly, and as soon as it is above the limit, it ends the pod. The Deployment replaces it, the new pod writes the same 80 MiB, and the cycle starts over.

MAC Step 17

```
kubectl --context talos apply -f szenarien/09-storage/fix-platte.yaml
kubectl --context talos -n dbg-09 delete pod --field-selector=status.phase=Failed
```

Raises `sizeLimit` to 200 MiB and sets a request and limit for `ephemeral-storage`, so that the scheduler plans for the space. The second command removes the evicted pods, which would otherwise stay around as leftovers.

WHAT YOU SHOULD SEE

One pod `1/1 Running` that is still running after several minutes. `kubectl describe pod` shows the entry `ephemeral-storage: 200Mi` under `Requests`.

If not a single pod but the whole node runs low, the kubelet sets the condition `DiskPressure`, adds the taint `node.kubernetes.io/disk-pressure` to the node and evicts pods by priority. Chapter 13 shows how to investigate that. For storage, one lesson remains: local-path volumes live on the same disk as images, logs and `emptyDir`.

RKE2

Volumes under `/opt/local-path-provisioner` on the root partition. Check via SSH with `df -h /opt /var/lib/kubelet` and `sudo du -sh /opt/local-path-provisioner/*`. A full volume fills up Ubuntu's system disk.

TALOS

Volumes under `/var/mnt/local-path-provisioner` on the EPHEMERAL partition, together with images and kubelet data. Check with `talosctl -n 172.16.0.171 mounts` (usage per mount) and `talosctl -n 172.16.0.171 usage /var/mnt/local-path-provisioner`.

MAC Step 18

```
kubectl --context rke2 delete ns dbg-09
kubectl --context talos delete ns dbg-09
```

Cleans up both clusters. Because `local-path` has `reclaimPolicy: Delete`, the provisioner deletes the PVs and the directories on the nodes together with the PVCs. To do that, it starts helper pods again, `helper-pod-delete-...`.

TRY IT YOURSELF

On RKE2, create the PVC from `fix-klasse.yaml` without a pod. Wait a minute. What status does the PVC have, and which event do you see? Then create a pod that uses the PVC with `nodeSelector: kubernetes.io/hostname: rke2-cp`. On which node does the volume live afterwards?

Hint: without a pod, the PVC stays `Pending` with `WaitForFirstConsumer`, which is correct. As soon as the pod is scheduled on `rke2-cp`, the provisioner creates the directory there. The node affinity in the PV shows `rke2-cp`. Every later pod with this PVC also has to run on `rke2-cp`.

WHAT YOU LEARNED

- For storage problems, follow the order pod, PVC, PV, provisioner, node. The events on the PVC are almost always the fastest way to the cause.
- `WaitForFirstConsumer` is not an error. The spec of a PVC is immutable, so a wrong StorageClass means creating it again.
- On Talos, the local-path-provisioner needs a namespace with PSA `privileged`, otherwise the API rejects its helper pods (`hostPath volumes`).
- local-path volumes are bound to one node. `ReadWriteOnce` applies per node. Several replicas with their own data need a StatefulSet with `volumeClaimTemplates`.
- `fsGroup` has no effect on `hostPath` PVs. You set the permissions in the volume when you create it, for example in the init container. An `emptyDir` above its `sizeLimit` leads to eviction, not to a write error.

CHAPTER 10

RBAC and Pod Security

You read `Forbidden` messages like a log entry, check permissions with `kubectl auth can-i`, find wrongly bound roles and recognize when Pod Security Admission rejects a pod even though the Deployment was accepted.

Time: 60 minutes **Requires:** Chapters 2 and 3

10.1 What Breaks in This Chapter

Two mechanisms say "no" in the cluster, and both answer with `Forbidden`. RBAC decides **who** may perform an action on the API. Pod Security Admission (PSA) decides **which** pods may exist at all, no matter who creates them. The messages look alike, but troubleshooting them is completely different.

Scenario	Symptom	Where the answer is
RoleBinding in the wrong namespace	Pod in `CrashLoopBackOff`, log: `cannot list resource "pods"`	`kubectl auth can-i --as`, `get rolebindings -A`
Role instead of ClusterRole	Log: `... at the cluster scope`	`kubectl api-resources --namespaced=false`
hostPath on Talos	DaemonSet `0/2`, no pod	Event `FailedCreate` on the DaemonSet
Namespace with `restricted`	Deployment `0/2`, no pod	Event `FailedCreate` on the ReplicaSet

All scenarios run in the namespace `dbg-10`. Play them in the order of the chapter. The PSA scenarios set labels on the namespace, and these apply to everything in it.

10.2 Reading a Forbidden Message

Every RBAC rejection follows the same sentence structure. Once you know it, you do not have to guess:

```
Error from server (Forbidden): pods is forbidden: User
"system:serviceaccount:dbg-10:app"
cannot list resource "pods" in API group "" in the namespace "dbg-10"
```

Part	Meaning
`User "system:serviceaccount:dbg-10:app"`	The identity. For ServiceAccounts it is always `system:serviceaccount:<namespace>:<name>`.
`cannot list`	The verb: `get`, `list`, `watch`, `create`, `update`, `patch`, `delete`.
`resource "pods"`	The resource in the plural. For subresources, for example `pods/log` or `pods/exec`.
`in API group ""`	The API group. `""` is the core group (pods, Services, ConfigMaps). Deployments live in `apps`.
`in the namespace "dbg-10"`	The scope. For cluster-wide resources it says `at the cluster scope` here.

You translate these five parts one to one into a rule of a Role. If one of them is missing from the role, or if the role is bound to the wrong identity, you get `Forbidden`.

10.3 RoleBinding in the Wrong Namespace

A Role only applies in its namespace, and so does a RoleBinding. A common copy-and-paste mistake: the RoleBinding comes from another project, and nobody changed its `metadata.namespace`.

FILE `szenarien/10-rbac-psa/kaputt-binding.yaml (excerpt)`

```yaml
kind: RoleBinding
metadata:
  name: pod-reader
  namespace: default        # ERROR: copied, namespace not changed
subjects:
  - kind: ServiceAccount
    name: app
    namespace: dbg-10
roleRef:
  kind: Role
  name: pod-reader
# ...
      containers:
        - name: kubectl
          image: registry.k8s.io/kubectl:v1.36.6
          args: ["get", "pods", "-n", "dbg-10", "--watch"]
```

The container runs `kubectl get pods --watch` with the token of the ServiceAccount `app`. If that works, it keeps running. If it fails, `kubectl` exits with exit code 1. The pod meets `restricted`, so PSA does not get in your way later.

MAC Step 1

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/kaputt-binding.yaml
kubectl --context talos apply -f szenarien/10-rbac-psa/kaputt-binding.yaml
```

Creates the namespace, the ServiceAccount, the Role, the wrong RoleBinding and the Deployment `pod-lister`.

SYMPTOM

```
$ kubectl --context rke2 get pods -n dbg-10
NAME                          READY   STATUS             RESTARTS      AGE
pod-lister-7d9f6c8b5-k2x8w    0/1     CrashLoopBackOff   3 (28s ago)   75s
```

A crash loop like the ones in Chapter 6. Only the log shows that this is not a bug in the program.

Investigate

MAC — Step 2

```
kubectl --context rke2 logs -n dbg-10 deploy/pod-lister
```

During a restart cycle, the container may be gone at that moment. Then `--previous` helps.

```
Error from server (Forbidden): pods is forbidden: User
"system:serviceaccount:dbg-10:app"
cannot list resource "pods" in API group "" in the namespace "dbg-10"
```

Now you ask the API server itself whether this identity is allowed to do this. `--as` lets you act as someone else without knowing their token. Your admin account may do that, normal users may not.

MAC — Step 3

```
kubectl --context rke2 auth can-i list pods -n dbg-10 \
  --as=system:serviceaccount:dbg-10:app
```

Answers with `yes` or `no`. This is the same check that the API server runs for real requests.

```
no
```

What is the ServiceAccount allowed to do at all? `--list` shows all rules that apply to it in this namespace:

MAC — Step 4

```
kubectl --context rke2 auth can-i --list -n dbg-10 \
  --as=system:serviceaccount:dbg-10:app
```

This shows only the basic permissions every authenticated user has (`selfsubjectaccessreviews`, `/api/*` and similar), but no pods.

The Role does exist, though. So you look for the binding, in all namespaces:

MAC — Step 5

```
kubectl --context rke2 get rolebindings -A -o wide | grep -E 'NAMESPACE|pod-reader'
```

`-o wide` shows the role and the subjects. That way you see where the binding really is.

```
NAMESPACE   NAME          ROLE               AGE   USERS   GROUPS   SERVICEACCOUNTS
default     pod-reader    Role/pod-reader    3m                        dbg-10/app
```

The binding is in `default` and refers to a Role `pod-reader` in `default`. That Role does not exist there. The API server does not check this when you create the binding. A RoleBinding to a missing role is allowed and has no effect.

K9S Step 6

```
:sa dbg-10⏎        select app, Enter = effective rules
:rolebindings⏎     0 = all namespaces, d = describe
```

In the ServiceAccount view, `Enter` shows the rules that apply to the account through bindings. If the list stays empty, no binding applies. `u` shows which pods use the account.

CAUSE

Role and RoleBinding are namespaced, and a RoleBinding always refers to a Role in its **own** namespace. The binding in `default` looks for a Role `pod-reader` there, finds none and therefore grants nothing. The ServiceAccount in `dbg-10` has no permissions.

Fix

FILE `szenarien/10-rbac-psa/fix-binding.yaml (excerpt)`

```yaml
kind: RoleBinding
metadata:
  name: pod-reader
  namespace: dbg-10
```

MAC Step 7

```
kubectl --context rke2 delete rolebinding pod-reader -n default
kubectl --context rke2 apply -f szenarien/10-rbac-psa/fix-binding.yaml
kubectl --context rke2 rollout restart deploy/pod-lister -n dbg-10
```

Deletes the stray binding and creates the correct one. RBAC is checked again on every request. The restart only saves you from waiting for the backoff to end. On Talos, run the same commands with `--context talos`.

WHAT YOU SHOULD SEE

`kubectl --context rke2 auth can-i list pods -n dbg-10 --as=system:serviceaccount:dbg-10:app` answers `yes`.

The pod runs, and its log shows the pod list:

```
NAME                          READY   STATUS    RESTARTS   AGE
pod-lister-6b8d7f9c4-q7tlm    1/1     Running   0          5s
```

10.4 Role Instead of ClusterRole

Some resources do not belong to any namespace: nodes, PersistentVolumes, namespaces themselves, StorageClasses, ClusterRoles. A Role cannot grant permissions on them. The API server still accepts such a Role, because it does not check whether rules make sense.

FILE `szenarien/10-rbac-psa/kaputt-clusterrole.yaml (excerpt)`

```yaml
kind: Role
metadata:
  name: node-reader
  namespace: dbg-10
rules:
  - apiGroups: [""]
    resources: ["nodes"]
    verbs: ["get", "list", "watch"]
```

The file also contains a ServiceAccount `node-reader`, a matching RoleBinding in `dbg-10` and a Deployment that runs `kubectl get nodes --watch`.

MAC — Step 8

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/kaputt-clusterrole.yaml
```

This time the binding is in the right namespace.

SYMPTOM

The pod `node-lister` goes into `CrashLoopBackOff`. The log:

```
Error from server (Forbidden): nodes is forbidden: User
"system:serviceaccount:dbg-10:node-reader"
cannot list resource "nodes" in API group "" at the cluster scope
```

Investigate

The key part is `at the cluster scope`. The request has no namespace, so only cluster-wide bindings can help. `api-resources` tells you whether a resource is namespaced:

MAC — Step 9

```
kubectl --context rke2 api-resources --namespaced=false | grep -E 'NAME|^nodes'
```

Lists only cluster-wide resources. With `--namespaced=true` you get the opposite list.

```
NAME    SHORTNAMES   APIVERSION   NAMESPACED   KIND
nodes   no           v1           false        Node
```

`kubectl auth can-i list nodes -n dbg-10 --as=system:serviceaccount:dbg-10:node-reader` answers `no`, even though the Role names exactly this permission. Adding `-n` changes nothing, because nodes have no namespace.

CAUSE

RBAC evaluates a Role only for requests **inside** its namespace. `list nodes` is a request without a namespace, and for those only ClusterRoleBindings count. A RoleBinding that points to a ClusterRole does not help either: it grants the ClusterRole's permissions only inside the namespace of the binding.

Fix

FILE `szenarien/10-rbac-psa/fix-clusterrole.yaml (excerpt)`

```yaml
kind: ClusterRole
metadata:
  name: dbg-10-node-reader
rules:
  - apiGroups: [""]
    resources: ["nodes"]
    verbs: ["get", "list", "watch"]
---
kind: ClusterRoleBinding
metadata:
  name: dbg-10-node-reader
subjects:
  - kind: ServiceAccount
    name: node-reader
    namespace: dbg-10
```

MAC Step 10

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/fix-clusterrole.yaml
kubectl --context rke2 rollout restart deploy/node-lister -n dbg-10
```

Grants read access to nodes cluster-wide, but only to this one ServiceAccount.

WHAT YOU SHOULD SEE

`auth can-i list nodes --as=system:serviceaccount:dbg-10:node-reader` answers `yes`, and `kubectl --context rke2 logs -n dbg-10 deploy/node-lister` shows `rke2-cp` and `rke2-w1`.

PITFALL

ClusterRole and ClusterRoleBinding do not belong to any namespace, so `kubectl delete ns dbg-10` does not remove them. Forgotten ClusterRoleBindings are a classic finding in audits: if someone later creates a ServiceAccount with the same name, it inherits the permissions.

10.5 Pod Security: What the Distributions Set

PSA checks pods against one of three levels: `privileged` (everything allowed), `baseline` (no known escapes such as `privileged`, `hostPath` or host namespaces) and `restricted` (in addition no root,

all capabilities dropped, seccomp). Per namespace, labels set the level for three modes: `enforce` rejects, `warn` returns a warning to `kubectl`, `audit` writes to the audit log. If a label is missing, the cluster-wide default of the API server applies. And that default is exactly what differs in the lab:

RKE2

Without `profile` in `/etc/rancher/rke2/config.yaml`, RKE2 writes only `enforce: privileged` to `/etc/rancher/rke2/rke2-pss.yaml`. There is no default for `warn` and `audit`, so there are no warnings either. With `profile: cis`, this becomes `restricted` for all three modes, with `kube-system`, `compliance-operator-system` and `tigera-operator` exempted. You set your own file with `pod-security-admission-config-file`.

TALOS

In the machine config under `cluster.apiServer.admissionControl`, Talos sets `enforce: baseline`, `audit: restricted` and `warn: restricted` cluster-wide. Only `kube-system` is exempted. That is why on Talos you see warnings `would violate PodSecurity "restricted:latest"` for many manifests, and they block nothing. You can change the default with a machine config patch.

So the same manifest can run on RKE2 and fail on Talos. Check your cluster's default before you look for a problem elsewhere:

MAC — Step 11

```
talosctl -n 172.16.0.170 get machineconfig -o yaml | grep -A 14 'name: PodSecurity'
```

Shows the PSA block from the active machine config of the control plane node. On RKE2 you read the file `/etc/rancher/rke2/rke2-pss.yaml` on `rke2-cp` instead.

10.6 hostPath: Runs on RKE2, Not on Talos

A small log collector should read `/var/log` on every node. A typical candidate for a DaemonSet with `hostPath`. The name `log-sammler` means "log collector".

FILE szenarien/10-rbac-psa/kaputt-hostpath.yaml (excerpt)

```
kind: DaemonSet
metadata:
  name: log-sammler
  namespace: dbg-10
# ...
      volumes:
        - name: varlog
          hostPath:
            path: /var/log
```

MAC — Step 12

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/kaputt-hostpath.yaml
kubectl --context talos apply -f szenarien/10-rbac-psa/kaputt-hostpath.yaml
```

On RKE2 there is no complaint. On Talos, `kubectl` prints a warning.

```
Warning: would violate PodSecurity "restricted:latest": allowPrivilegeEscalation !=
false
(container "sammler" must set securityContext.allowPrivilegeEscalation=false), ...
restricted volume types (volume "varlog" uses restricted volume type "hostPath"), ...
daemonset.apps/log-sammler created
```

SYMPTOM

```
$ kubectl --context talos get ds -n dbg-10
NAME          DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
log-sammler   2         0         0       0            0           <none>          40s
```

Two pods desired, zero created. On RKE2 the same line shows `2 2 2`.

Investigate

Where there is no pod, there is no `describe pod`. The message is attached to the object that tried to create the pod:

MAC — Step 13

```
kubectl --context talos events -n dbg-10 --for daemonset/log-sammler
```

The events of the DaemonSet. For Deployments you look one level deeper, at the ReplicaSet.

```
LAST SEEN   TYPE      REASON         OBJECT                  MESSAGE
12s         Warning   FailedCreate   DaemonSet/log-sammler   Error creating: pods
"log-sammler-8kq2d" is
forbidden: violates PodSecurity "baseline:latest": hostPath volumes (volume "varlog")
```

K9S — Step 14

```
:ds dbg-10⏎        select log-sammler, d = describe
:events⏎           / FailedCreate⏎
```

The warning is at the end of the DaemonSet's describe view. In the event view, you filter by the reason with `/`.

`kubectl --context talos get ns dbg-10 --show-labels` confirms it: there is no label `pod-security.kubernetes.io/...`, so the Talos default applies.

CAUSE

The warning during `apply` came from the `warn` mode (Talos: `restricted`) and did not stop the DaemonSet. In `enforce` mode, PSA checks only pods, not their controllers. That is why the DaemonSet was accepted, and only the DaemonSet controller got the rejection against `baseline` when it tried to create each pod. RKE2 without the CIS profile enforces `privileged` and lets the pod through.

Fix

There are two clean ways: the pod does without `hostPath`, or the namespace gets a deliberate exception. A log collector needs the node, so you take the second one. You set `warn` and `audit` to `baseline` so that the exception stays visible.

FILE `szenarien/10-rbac-psa/fix-hostpath.yaml`

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dbg-10
  labels:
    pod-security.kubernetes.io/enforce: privileged
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/warn: baseline
    pod-security.kubernetes.io/warn-version: latest
    pod-security.kubernetes.io/audit: baseline
    pod-security.kubernetes.io/audit-version: latest
```

MAC Step 15

```
kubectl --context talos apply -f szenarien/10-rbac-psa/fix-hostpath.yaml
```

The DaemonSet controller tries again after a delay. You do not need to restart anything.

WHAT YOU SHOULD SEE

`kubectl --context talos get ds -n dbg-10` shows `2 2 2`. Every change to the DaemonSet still brings the warning `would violate PodSecurity "baseline:latest": hostPath volumes ...`. That is intended: the exception keeps reminding you of itself.

PITFALL

The same thing hits `kubectl debug` on Talos. The default profile `general` adds the capability `SYS_PTRACE`, and `baseline` rejects the debug container. Always use `--profile=baseline` there, just like the k9s plugin from Chapter 3 does.

10.7 restricted: Deployment Yes, Pods No

For the last scenario you set up the namespace again, this time with the strictest level. This is the normal case for application namespaces.

MAC Step 16

```
kubectl --context rke2 delete ns dbg-10
kubectl --context talos delete ns dbg-10
```

Cleans up the previous scenarios. Remember the ClusterRole from the second scenario, it stays.

FILE szenarien/10-rbac-psa/kaputt-restricted.yaml (excerpt)

```
kind: Namespace
metadata:
  name: dbg-10
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/warn: restricted
# ...
      containers:
        - name: web
          image: nginx:1.29-alpine
```

An ordinary nginx image without a `securityContext`. It runs as root.

MAC Step 17

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/kaputt-restricted.yaml
```

Here RKE2 is just as strict as Talos, because the label on the namespace overrides any cluster-wide default.

```
namespace/dbg-10 created
Warning: would violate PodSecurity "restricted:latest": allowPrivilegeEscalation !=
false
(container "web" must set securityContext.allowPrivilegeEscalation=false), ...
deployment.apps/web created
```

SYMPTOM

```
$ kubectl --context rke2 get deploy,rs,pods -n dbg-10
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/web   0/2     0            0           30s

NAME                              DESIRED   CURRENT   READY   AGE
replicaset.apps/web-5d7c9b8f6d   2         0         0       30s
```

`created`, and still not a single pod. In pipelines the warning above easily gets lost, and then this picture is all you see.

Investigate

MAC Step 18

```
kubectl --context rke2 describe rs -n dbg-10 -l app=web | tail -4
```

The ReplicaSet creates the pods, so it got the rejection.

```
  Warning  FailedCreate  12s (x14 over 52s)  replicaset-controller  Error creating:
pods
"web-5d7c9b8f6d-x7k2p" is forbidden: violates PodSecurity "restricted:latest":
allowPrivilegeEscalation != false (container "web" must set securityContext.
allowPrivilegeEscalation=false), unrestricted capabilities (container "web" must set
securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or container
"web"
```

```
must set securityContext.runAsNonRoot=true), seccompProfile (pod or container "web"
must
set securityContext.seccompProfile.type to "RuntimeDefault" or "Localhost")
```

The message is a checklist. Each parenthesis names the field that is missing. Before you tighten an existing namespace, `kubectl label --dry-run=server --overwrite ns <name> pod-security.kubernetes.io/enforce=restricted` checks all running pods against the new level without setting the label.

CAUSE

`enforce` applies only to pods. A Deployment contains a pod template, but it is checked only in the `warn` and `audit` modes. The Deployment is created, so is the ReplicaSet, and only the pods fail. Without a `warn` label (the normal case on RKE2 without CIS), you would not even get the warning during `apply`.

Fix

Do not lower the level, fix the pod. nginx offers an image that runs without root on port 8080.

FILE `szenarien/10-rbac-psa/fix-restricted.yaml (excerpt)`

```yaml
    spec:
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: web
          image: nginxinc/nginx-unprivileged:1.29-alpine
          ports:
            - containerPort: 8080
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
```

MAC

Step 19

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/fix-restricted.yaml
kubectl --context talos apply -f szenarien/10-rbac-psa/kaputt-restricted.yaml \
  -f szenarien/10-rbac-psa/fix-restricted.yaml
```

On Talos you create the namespace and the fix in one step. This time there is no warning.

WHAT YOU SHOULD SEE

```
$ kubectl --context rke2 get pods -n dbg-10
NAME                   READY   STATUS    RESTARTS   AGE
web-6f8b7d5c9b-4tq8z   1/1     Running   0          8s
web-6f8b7d5c9b-n2wlx   1/1     Running   0          8s
```

TRY IT YOURSELF

`szenarien/10-rbac-psa/aufgabe-job.yaml` creates a Job that is supposed to read ConfigMaps in `dbg-10`. It fails with `Forbidden`, even though the Role and the RoleBinding are in the right namespace and name the right ServiceAccount. Find the error using only the log and `kubectl auth can-i`.

Hint: read the part `in API group "..."` in the message and compare it with the `apiGroups` of the Role.

MAC Step 20

```
kubectl --context rke2 delete ns dbg-10
kubectl --context talos delete ns dbg-10
kubectl --context rke2 delete -f szenarien/10-rbac-psa/fix-clusterrole.yaml
```

Cleans up the chapter, including the cluster-wide objects.

WHAT YOU LEARNED

- A `Forbidden` message names the identity, verb, resource, API group and scope, exactly what a role has to contain.
- `kubectl auth can-i ... --as=system:serviceaccount:<ns>:<name>` answers permission questions without involving the pod. `--list` shows everything.
- RoleBindings only work in their namespace. Cluster-wide resources need a ClusterRole and a ClusterRoleBinding.
- PSA enforces levels only for pods. Deployments and DaemonSets are accepted, and the rejection shows up as `FailedCreate` on the controller.
- RKE2 without CIS enforces `privileged`, Talos enforces `baseline`. Labels on the namespace override both defaults.

CHAPTER 11

Failures Caused by Kyverno

You recognize when Kyverno is behind a rejection or an unexpected change, read its messages, find the responsible policy, get back into the cluster when the webhook is down and test policies in advance without a cluster.

Time: 75 minutes **Requires:** Chapters 2, 3 and 10

11.1 What Breaks in This Chapter

Kyverno is an admission webhook. The API server sends it every matching request before it stores it. Kyverno can reject (`ValidatingPolicy`), change (`MutatingPolicy`) or, if it does not answer, bring the cluster to a halt. To users, all three cases at first look like "Kubernetes is acting up".

Scenario	Symptom	Where the answer is
Policy rejects Deployment	`apply` fails immediately	`admission webhook "vpol..." denied the request`
Policy rejects only pods	Deployment `0/1`, no pod	Event `FailedCreate` on the ReplicaSet
Mutation changes the pod	`CreateContainerConfigError`	Live object versus manifest, `kubectl get mpol`
Webhook unreachable	**Every** change in `dbg-11` fails	`failed calling webhook`, `failurePolicy`
PolicyException has no effect	Exception created, pods still rejected	Warning during `apply`, startup arguments

All policies in this chapter are under `policies/` and apply only in the namespace `dbg-11`. Create each policy at the start of its scenario and delete it again at the end, so that the scenarios do not interfere with each other.

11.2 Installing Kyverno

FILE `werkzeuge/kyverno-values.yaml (excerpt)`

```yaml
admissionController:
  replicas: 1
backgroundController:
  replicas: 1
# ... cleanupController, reportsController likewise
features:
  policyExceptions:
```

```
    enabled: false
    namespace: ""
```

One instance per controller saves memory in the lab. The price: if the admission controller fails, nobody answers webhook requests anymore. PolicyExceptions stay off for now, as the chart's default. You need that in the last scenario.

MAC Step 1

```
helm repo add kyverno https://kyverno.github.io/kyverno/
helm repo update
for ctx in rke2 talos; do
  helm --kube-context $ctx upgrade --install kyverno kyverno/kyverno \
    -n kyverno --create-namespace --version 3.9.2 \
    -f werkzeuge/kyverno-values.yaml --wait --timeout 5m
done
```

Chart 3.9.2 ships Kyverno 1.19.2. The loop installs the same version on both clusters.

MAC Step 2

```
kubectl --context rke2 get deploy -n kyverno
```

Four Deployments: `kyverno-admission-controller`, `kyverno-background-controller`, `kyverno-cleanup-controller`, `kyverno-reports-controller`, each `1/1`.

The Kyverno pods meet `restricted`, so there are no PSA warnings on Talos. On both clusters, Kyverno excludes `kube-system` and `kyverno` itself.

PITFALL

Many guides still use `kind: ClusterPolicy` from `kyverno.io/v1`. In Kyverno 1.19 it is deprecated, and `kubectl apply` warns with `kyverno.io/v1 ClusterPolicy is deprecated and will be removed in a future release`. This chapter uses the CEL-based policies from `policies.kyverno.io/v1`, whose messages look different.

11.3 Policy Rejects the Deployment

FILE `policies/require-team-label.yaml (excerpt)`

```
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: require-team-label
spec:
  validationActions: [Deny]
  matchConstraints:
    namespaceSelector:
      matchExpressions:
        - key: kubernetes.io/metadata.name
          operator: In
```

```
          values: [dbg-11]
    resourceRules:
      - apiGroups: [""]
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: Pods brauchen das Label team.
      expression: >-
        object.metadata.?labels.team.orValue('') != ''
```

The policy checks only pods. But Kyverno automatically generates the same rule for Deployments, StatefulSets, DaemonSets, Jobs and CronJobs. This is called autogen, and it is the default for CEL policies when exactly one rule matches `pods` without an `objectSelector`. The message is German, as in the whole repo: “Pods need the label team.”

MAC Step 3

```
kubectl --context rke2 apply -f policies/require-team-label.yaml
kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-label.yaml
```

The Deployment `web` has only the label `app` in its pod template.

SYMPTOM

```
namespace/dbg-11 created
Error from server: error when creating "szenarien/11-kyverno/kaputt-label.yaml":
admission webhook
"vpol.validate.kyverno.svc-fail" denied the request: Policy require-team-label failed:
Pods
brauchen das Label team.
```

Investigate

The message already contains everything: the webhook name starts with `vpol` (ValidatingPolicy, `mpol` for MutatingPolicy, just `validate.kyverno.svc` for old ClusterPolicies), and the suffix `-fail` stands for `failurePolicy: Fail`. Then come the name of the policy and its `message` text, word for word.

MAC Step 4

```
kubectl --context rke2 get vpol
kubectl --context rke2 get vpol require-team-label -o jsonpath='{.status.autogen}{"\n"}'
```

`vpol` is the short form of `validatingpolicies`, and the `READY` column should show `True`. `status.autogen` lists the controllers for which Kyverno generated the rule as well.

K9S Step 5

```
:vpol⏎             select require-team-label, y = YAML
:polr dbg-11⏎      policy reports in the namespace
```

k9s knows the Kyverno CRDs by their short names. In the YAML view, you search for `message` with `/`.

PITFALL

Old ClusterPolicies answer with `resource Deployment/dbg-11/web was blocked due to the following policies`, followed by the policy, the rule and `validation error: ... failed at path ...`. If there are several CEL policies with different selectors in the cluster, Kyverno appends a short hash to the webhook name, for example `vpol.validate.kyverno.svc-fail-1a2b3c4d`.

CAUSE

The pod template contains no label `team`. Because of autogen, Kyverno already checks the Deployment and not only the pod. For that, the expression from the policy was rewritten to `object.spec.template.metadata`. That is why `apply` fails immediately and visibly.

Fix

FILE `szenarien/11-kyverno/fix-label.yaml (excerpt)`

```yaml
  template:
    metadata:
      labels:
        app: web
        team: shop
```

MAC Step 6

```
kubectl --context rke2 apply -f szenarien/11-kyverno/fix-label.yaml
```

This time you get `deployment.apps/web created`.

WHAT YOU SHOULD SEE

`kubectl --context rke2 get pods -n dbg-11 -l app=web` shows one pod in `Running`. Then clean up: `kubectl --context rke2 delete -f policies/require-team-label.yaml`.

11.4 Policy Rejects Only the Pods

The more dangerous case is when Kyverno lets the Deployment through and only rejects the pods. Then the developer sees `created`, the pipeline is green, and still nothing runs.

FILE `policies/require-memory-limit.yaml (excerpt)`

```yaml
  matchConstraints:
    objectSelector:
      matchExpressions:
        - key: policy.nyrvex.com/skip
```

```
          operator: DoesNotExist
    resourceRules:
      - apiGroups: [""]
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: Jeder Container braucht ein Memory-Limit.
      expression: >-
        object.spec.containers.all(c,
          c.?resources.?limits.?memory.orValue('') != '')
```

Someone wanted to build in an opt-out by label and added an `objectSelector`. With an `objectSelector`, Kyverno turns autogen off, because labels on the Deployment and labels on the pod are different things. The message means "Every container needs a memory limit."

MAC Step 7

```
kubectl --context rke2 apply -f policies/require-memory-limit.yaml
kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-limit.yaml
```

The Deployment `api` sets no memory limit.

SYMPTOM

```
$ kubectl --context rke2 get deploy,rs -n dbg-11 -l app=api
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/api   0/1     0            0           25s

NAME                              DESIRED   CURRENT   READY   AGE
replicaset.apps/api-6c8d9f7b54    1         0         0       25s
```

Investigate

The same pattern as with PSA in Chapter 10: no pod, so look at the events on the ReplicaSet.

MAC Step 8

```
kubectl --context rke2 events -n dbg-11 --for replicaset/api-6c8d9f7b54
```

Insert the name of your ReplicaSet, or use the `Shift-E` plugin on the ReplicaSet in k9s.

```
LAST SEEN   TYPE      REASON         OBJECT                       MESSAGE
8s          Warning   FailedCreate   ReplicaSet/api-6c8d9f7b54    Error creating:
admission webhook
"vpol.validate.kyverno.svc-fail" denied the request: Policy require-memory-limit
failed: Jeder
Container braucht ein Memory-Limit.
```

A policy's status tells you whether it also checks controllers. For `require-memory-limit`, `status.autogen` stays empty:

MAC Step 9

```
kubectl --context rke2 get vpol require-memory-limit -o jsonpath='{.status.autogen}{"\n"}'
```

Empty or `{}` means: this policy sees only pods. Compare it with `require-team-label` from the first scenario.

CAUSE

The policy applies only to pods and, because of the `objectSelector`, has no autogen rules. The Deployment and the ReplicaSet pass Kyverno, and only the pod creation by the ReplicaSet controller is rejected. That is why the message does not end up in your terminal, but as an event on the ReplicaSet. For users, a rejection during `apply` is much friendlier, so autogen should only be missing for a good reason.

Fix

FILE szenarien/11-kyverno/fix-limit.yaml (excerpt)

```
        resources:
          limits:
            memory: 128Mi
```

MAC Step 10

```
kubectl --context rke2 apply -f szenarien/11-kyverno/fix-limit.yaml
```

New pod template, so a new ReplicaSet. The old one stays around with `0` replicas.

WHAT YOU SHOULD SEE

`kubectl --context rke2 get pods -n dbg-11 -l app=api` shows `1/1 Running`. Then:

`kubectl --context rke2 delete -f policies/require-memory-limit.yaml`.

11.5 Mutation Changes the Pod Unexpectedly

The platform team rolls out a “secure default”: every pod in `dbg-11` gets `runAsNonRoot: true`. Well meant, but the application team knows nothing about it.

FILE policies/add-run-as-non-root.yaml (excerpt)

```
apiVersion: policies.kyverno.io/v1
kind: MutatingPolicy
metadata:
  name: add-run-as-non-root
spec:
  # ... matchConstraints as above, CREATE only
  mutations:
```

```
    - patchType: ApplyConfiguration
      applyConfiguration:
        expression: >-
          Object{
            spec: Object.spec{
              securityContext: Object.spec.securityContext{
                runAsNonRoot: true
              }
            }
          }
```

MAC Step 11

```
kubectl --context rke2 apply -f policies/add-run-as-non-root.yaml
kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-mutation.yaml
```

The Deployment `shop` uses the normal image `nginx:1.29-alpine`, which starts as root. The manifest contains no `securityContext`.

SYMPTOM

```
$ kubectl --context rke2 get pods -n dbg-11 -l app=shop
NAME                    READY   STATUS                       RESTARTS   AGE
shop-7b9c6d5f8-2kx9p    0/1     CreateContainerConfigError   0          20s
```

Investigate

MAC Step 12

```
kubectl --context rke2 describe pod -n dbg-11 -l app=shop | tail -3
```

The kubelet message at the end explains the status.

```
  Warning  Failed     4s (x3 over 20s)  kubelet  Error: container has runAsNonRoot and
image will run as
root (pod: "shop-7b9c6d5f8-2kx9p_dbg-11(5c1e9a7d-...)", container: shop)
```

The team swears it never set `runAsNonRoot`. So you compare the manifest and the live object:

MAC Step 13

```
grep -c runAsNonRoot szenarien/11-kyverno/kaputt-mutation.yaml
kubectl --context rke2 get deploy shop -n dbg-11 \
  -o jsonpath='{.spec.template.spec.securityContext}{"\n"}'
```

In the manifest: `0` matches. In the cluster: `{"runAsNonRoot":true}`. Something changed the object between `apply` and storage.

Even the Deployment is changed, not just the pod: autogen also applies to MutatingPolicies. Only mutating webhooks change objects when they are created. `kubectl --context rke2 get mpol` lists the Kyverno mutations, and `kubectl --context rke2 get mutatingwebhookconfigurations` lists all other candidates.

You confirm the suspicion offline with the Kyverno CLI. It shows the object as it would look after the mutation:

MAC — Step 14

```
kyverno apply policies/add-run-as-non-root.yaml \
  --resource szenarien/11-kyverno/kaputt-mutation.yaml \
  --values-file policies/values-dbg-11.yaml
```

Offline, the CLI knows no namespace labels. `values-dbg-11.yaml` tells it that `dbg-11` carries the label `kubernetes.io/metadata.name=dbg-11`.

```
policy add-run-as-non-root applied to dbg-11/Deployment/shop:
# ...
    template:
        spec:
            containers:
                - image: nginx:1.29-alpine
                  name: shop
            securityContext:
                runAsNonRoot: true
```

CAUSE

The MutatingPolicy sets `runAsNonRoot: true` in every new pod template. The image `nginx:1.29-alpine` starts as UID 0. Before the start, the kubelet checks whether the effective UID matches `runAsNonRoot`, and refuses the container with `CreateContainerConfigError`. In the default configuration, Kyverno does not write successful mutations as events, so the change is invisible.

Fix

The policy makes sense here. The image is what is wrong. nginx offers a variant without root:

FILE `szenarien/11-kyverno/fix-mutation.yaml (excerpt)`

```
        - name: shop
          image: nginxinc/nginx-unprivileged:1.29-alpine
          ports:
            - containerPort: 8080
```

MAC — Step 15

```
kubectl --context rke2 apply -f szenarien/11-kyverno/fix-mutation.yaml
```

The image runs as UID 101 and listens on 8080, because ports below 1024 need root.

WHAT YOU SHOULD SEE

The new pod is `Running`. Then: `kubectl --context rke2 delete -f policies/add-run-as-non-root.yaml`.

PITFALL

`kubectl diff` does not help here: it sends the manifest through admission as a server-side dry run, the mutation applies there too, and the difference disappears. If you want to see mutations as events, set `config.generateSuccessEvents: true` in the chart and remove `PolicyApplied` from `features.omitEvents.eventTypes`.

11.6 Webhook Unreachable

Now Kyverno itself fails. In real life this happens through an OOMKill, a failed node or a NetworkPolicy. In the lab you simulate it with `scale`.

MAC — Step 16

```
kubectl --context rke2 apply -f policies/require-team-label.yaml
kubectl --context rke2 scale deploy kyverno-admission-controller -n kyverno --replicas=0
kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-webhook.yaml
```

The Deployment `notfall` ("emergency") meets the policy. So it should be created.

SYMPTOM

```
namespace/dbg-11 unchanged
Error from server (InternalError): error when creating "szenarien/11-kyverno/kaputt-
webhook.yaml":
Internal error occurred: failed calling webhook "vpol.validate.kyverno.svc-fail": failed
to call
webhook: Post "https://kyverno-svc.kyverno.svc:443/vpol/require-team-label?timeout=10s":
no
endpoints available for service "kyverno-svc"
```

The same hits every pod that a ReplicaSet in `dbg-11` wants to create. A deleted pod does not come back.

Investigate

The message names the webhook, the Service and the reason. First check whether anybody is behind the Service at all:

MAC — Step 17

```
kubectl --context rke2 get deploy kyverno-admission-controller -n kyverno
kubectl --context rke2 get endpointslices -n kyverno -l kubernetes.io/service-name=kyverno-svc
```

`READY 0/0` and an EndpointSlice without endpoints. In a real incident you see `CrashLoopBackOff` or `Pending` here, and you continue with Chapters 4 and 6.

Then the question of why a missing webhook blocks instead of being skipped:

MAC Step 18

```
kubectl --context rke2 get validatingwebhookconfiguration kyverno-resource-validating-webhook-cfg \
  -o jsonpath='{range .webhooks[*]}{.name}{"\t"}{.failurePolicy}{"\n"}{end}' | grep vpol
```

Lists every webhook of the configuration with its `failurePolicy`.

```
vpol.validate.kyverno.svc-fail     Fail
```

In k9s you find the same under `:validatingwebhookconfigurations`, then `y` for the YAML view and `/failurePolicy`.

CAUSE

Without an explicit setting, CEL policies have `failurePolicy: Fail`. If the API server cannot reach the webhook, it rejects the request instead of letting it through. With only one replica of the admission controller, a single failed pod is enough for that. All resources that a policy matches are affected. `kube-system` and `kyverno` are excluded, which is why Kyverno itself can start again at all.

Fix

The order in an emergency:

1. Start the admission controller again and fix the cause of its failure.
2. If it does not come up, delete the webhook configuration. Kyverno creates it again by itself on the next start. Until then, no policies apply.
3. In the long run: two or three replicas, and `failurePolicy: Ignore` for non-critical policies.

MAC Step 19

```
kubectl --context rke2 scale deploy kyverno-admission-controller -n kyverno --replicas=1
kubectl --context rke2 rollout status deploy kyverno-admission-controller -n kyverno
```

Step 1. The emergency exit from step 2 would be `kubectl --context rke2 delete validatingwebhookconfiguration kyverno-resource-validating-webhook-cfg`.

WHAT YOU SHOULD SEE

`kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-webhook.yaml` reports `deployment.apps/notfall created`. Then: `kubectl --context rke2 delete -f policies/require-team-label.yaml`.

RKE2

The API server runs as the static pod `kube-apiserver-rke2-cp`. You see its messages about webhooks with `kubectl --context rke2 logs -n kube-system kube-apiserver-rke2-cp | grep 'failed calling webhook'`.

TALOS

Here too it is a static pod, `kube-apiserver-talos-cp`, with the same `kubectl logs`. If the API server itself fails, `talosctl -n 172.16.0.170 containers --namespace cri` and `talosctl logs --namespace cri` with the container ID help.

11.7 PolicyException Without Effect

A legacy application still runs without a memory limit. Instead of weakening the policy, it gets a `PolicyException`.

FILE `policies/exceptions/legacy-batch.yaml`

```yaml
apiVersion: policies.kyverno.io/v1
kind: PolicyException
metadata:
  name: legacy-batch
  namespace: dbg-11
spec:
  policyRefs:
    - name: require-memory-limit
      kind: ValidatingPolicy
  matchConditions:
    - name: nur-legacy-batch
      expression: >-
        object.metadata.?labels.app.orValue('') == 'legacy-batch'
```

MAC

Step 20

```
kubectl --context rke2 apply -f policies/require-memory-limit.yaml
kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-ausnahme.yaml
kubectl --context rke2 apply -f policies/exceptions/legacy-batch.yaml
```

The policy, the Deployment `legacy-batch` without a limit (`Ausnahme` means “exception”), and the exception. Watch the output of the last command.

SYMPTOM

```
Warning: PolicyException resources would not be processed until it is enabled.
policyexception.policies.kyverno.io/legacy-batch created
```

The Deployment stays at `0/1`, and the ReplicaSet shows the same `FailedCreate` message as in the second scenario.

Investigate

The warning comes from Kyverno itself. The startup arguments of the admission controller show which setting it means:

MAC Step 21

```
kubectl --context rke2 get deploy kyverno-admission-controller -n kyverno -o yaml \
  | grep -E 'enablePolicyException|exceptionNamespace'
```

The Helm values under `features.policyExceptions` turn into these arguments.

```
- --enablePolicyException=false
```

To list them, you spell out the group: `kubectl get policyexceptions.policies.kyverno.io -n dbg-11`. The old group `kyverno.io` also has a `PolicyException`, and without a group `kubectl` silently picks one of the two.

CAUSE

In Kyverno, PolicyExceptions are opt-in. Without `--enablePolicyException=true`, the API server accepts the exception, but Kyverno ignores it. In addition, `--exceptionNamespace` defines the namespace from which exceptions apply. This prevents anyone who may write in their own namespace from exempting themselves from policies.

Fix

FILE szenarien/11-kyverno/fix-ausnahme-values.yaml (excerpt)

```
features:
  policyExceptions:
    enabled: true
    namespace: dbg-11
```

In the lab, the exceptions live directly in `dbg-11`. In production you use a separate namespace that only the platform team may write to.

MAC Step 22

```
helm --kube-context rke2 upgrade kyverno kyverno/kyverno -n kyverno --version 3.9.2 \
  -f werkzeuge/kyverno-values.yaml -f szenarien/11-kyverno/fix-ausnahme-values.yaml --wait
```

The second file overrides only the `features.policyExceptions` block. The controllers restart with the new arguments.

WHAT YOU SHOULD SEE

After a short while, the ReplicaSet creates the pod, and `kubectl --context rke2 get pods -n dbg-11 -l app=legacy-batch` shows `Running`. The policy report lists the exception as `SKIP`:

```
$ kubectl --context rke2 get polr -n dbg-11
NAME                                   KIND   NAME                           PASS   FAIL
WARN   ERROR   SKIP   AGE
9b2f4c1e-7a3d-4e8b-b6f0-2d5c8a1e9f37   Pod    legacy-batch-7f6d9c8b5-wq4zr   0      0
0      0       1      40s
```

11.8 Test Policies Before They Get in the Way

You could have found almost all failures in this chapter without a cluster. For every policy under `policies/` there are test cases in `policies/tests/kyverno-test.yaml`: resources that must pass, fail or be skipped by an exception, and, for the mutation, the expected result.

FILE `policies/tests/kyverno-test.yaml (excerpt)`

```yaml
apiVersion: cli.kyverno.io/v1alpha1
kind: Test
metadata:
  name: kapitel-11
policies:
  - ../require-team-label.yaml
  - ../require-memory-limit.yaml
  - ../add-run-as-non-root.yaml
exceptions:
  - ../exceptions/legacy-batch.yaml
# ... resources, values
results:
  - isValidatingPolicy: true
    policy: require-team-label
    kind: Deployment
    resources: [web-ohne-team]
    result: fail
  # ... more cases
  - isMutatingPolicy: true
    policy: add-run-as-non-root
    kind: Pod
    resources: [root-web]
    patchedResources: patched/pod-root.yaml
    result: pass
```

MAC Step 23

```
kyverno test policies/tests
```

Runs locally with the Kyverno CLI 1.19 (`brew install kyverno`). In the lab, the file was checked with CLI 1.19.1.

WHAT YOU SHOULD SEE

```
Test Summary: 13 tests passed and 0 tests failed
```

The second scenario also shows up offline: `kyverno apply` with both policies on `kaputt-limit.yaml` reports `pass: 1, fail: 0`. The one `pass` comes from `require-team-label`. `require-memory-limit` does not check the Deployment at all. So a missing `fail` does not mean “all good”, but possibly “not responsible”.

The second safety belt runs in the cluster: introduce new policies with `validationActions: [Audit]`. They then block nothing, but Kyverno checks all existing resources in the background and writes the result to policy reports. `kubectl --context rke2 get polr -A` shows in the `FAIL` column whom a later `Deny` would hit.

TRY IT YOURSELF

Write another test case: a pod with the label `policy.nyrvex.com/skip: "true"` and without a memory limit. Which result do you expect for `require-memory-limit`, and why is this opt-out by label a security problem?

Hint: the `objectSelector` filters already in the API server. Anyone who may create pods may also set their labels. A PolicyException in a protected namespace is the cleaner way.

MAC

Step 24

```
kubectl --context rke2 delete -f policies/require-memory-limit.yaml -f policies/exceptions/legacy-
batch.yaml
kubectl --context rke2 delete ns dbg-11
```

Cleans up the chapter. If you want to repeat the scenarios on Talos, use the same commands with `--context talos`.

WHAT YOU LEARNED

- Kyverno rejections name the webhook, the policy and the message. `vpol` stands for ValidatingPolicy, `-fail` for `failurePolicy: Fail`.
- Autogen decides whether the Deployment is rejected right away or only the pod. Without autogen, the message shows up as `FailedCreate` on the ReplicaSet.
- Mutations are invisible by default. Compare the manifest and the live object, and check `kubectl get mpol` and the mutating webhooks.
- If the webhook is gone, `failurePolicy: Fail` blocks everything that matches. Emergency exit: start the controller or delete the webhook configuration.
- PolicyExceptions only work with `features.policyExceptions.enabled` and a matching namespace.
- `kyverno test` checks policies, including exceptions and mutations, without a cluster.

CHAPTER 12

Falco: Alerts and Disruptions

You get Falco running on both clusters, trace an alert back to the pod and container, silence a false positive in a targeted way and recognize when Falco stops delivering Kubernetes metadata.

Time: 60 minutes **Requires:** Chapters 2, 3 and 10

12.1 What Breaks in This Chapter

Falco uses eBPF to watch the system calls on every node and compares them with rules. It blocks nothing, it only reports. That is why failures with Falco are rarely application outages. They are questions for operations: Why does Falco not start? Where does this alert come from? Why does it report the same Job every night? And why is there no pod name in the alert?

Scenario	Symptom	Where the answer is
Falco does not start on Talos	DaemonSet `0/2`	`FailedCreate` on the DaemonSet, PSA `baseline`
Alert after `kubectl exec`	`Terminal shell in container`	`output_fields`, `hostname`, container ID
False positive from a Job	The same alert every ten minutes	Rule, condition, `exceptions`
No Kubernetes metadata	`k8s_pod=<NA>` in every alert	Socket list and mounts of the chart

The workloads run in the namespace `dbg-12`, Falco itself in the namespace `falco`.

12.2 Installing Falco

There is one values file each for RKE2 and Talos. They differ in only one line, and exactly this line is the topic of the last scenario.

FILE `werkzeuge/falco-values-rke2.yaml (excerpt)`

```
tty: true
driver:
  kind: modern_ebpf
falco:
  json_output: true
  json_include_output_property: true
  append_output:
    - suggested_output: true
    - match:
```

```
        source: syscall
      extra_output: "k8s_ns=%k8s.ns.name k8s_pod=%k8s.pod.name"
collectors:
  containerEngine:
    engines:
      docker:
        enabled: false
      # ... podman, containerd likewise off
      cri:
        enabled: true
        sockets: ["/run/k3s/containerd/containerd.sock"]
```

`modern_ebpf` needs no kernel module, which could not be loaded on Talos anyway. `suggested_output` appends the container ID and name to every message, and the second entry adds the namespace and pod. The `container` plugin gets the metadata through containerd's CRI interface. In `werkzeuge/falco-values-talos.yaml` that line says `/run/containerd/containerd.sock`.

MAC — Step 1

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm repo update
helm --kube-context rke2 upgrade --install falco falcosecurity/falco \
  -n falco --create-namespace --version 9.2.2 \
  -f werkzeuge/falco-values-rke2.yaml --wait --timeout 5m
```

Chart 9.2.2 ships Falco 0.45.2. On RKE2 this runs through without incident.

MAC — Step 2

```
kubectl --context rke2 logs -n falco -l app.kubernetes.io/name=falco -c falco --tail=20
```

With `-l`, `kubectl` reads from all Falco pods, one per node. With selectors, `kubectl` shows only ten lines per pod by default, hence `--tail`.

```
Mon Nov  9 14:02:11 2026: Falco version: 0.45.2 (x86_64)
Mon Nov  9 14:02:11 2026: Loading rules from:
Mon Nov  9 14:02:11 2026:    /etc/falco/falco_rules.yaml | schema validation: ok
Mon Nov  9 14:02:12 2026: Opening 'syscall' source with modern BPF probe.
```

What matters is `schema validation: ok` for every rules file and the line with `modern BPF probe`. If it is missing, the driver did not come up.

12.3 Falco Does Not Start on Talos

The same installation on Talos, but without `--wait`, so that you can watch:

MAC — Step 3

```
helm --kube-context talos upgrade --install falco falcosecurity/falco \
  -n falco --create-namespace --version 9.2.2 \
  -f werkzeuge/falco-values-talos.yaml
```

`--create-namespace` creates `falco` without labels. Helm reports `STATUS: deployed`, because without `--wait` it only checks whether the objects were created.

SYMPTOM

```
$ kubectl --context talos get ds -n falco
NAME    DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
falco   2         0         0       0            0           <none>          1m
```

Investigate

No pod, so look at the events on the DaemonSet. You know the pattern from Chapter 10:

MAC — Step 4

```
kubectl --context talos events -n falco --for daemonset/falco
```

Or in k9s: `:ds falco⏎`, then `d` and all the way down.

```
LAST SEEN   TYPE      REASON         OBJECT            MESSAGE
5s          Warning   FailedCreate   DaemonSet/falco   Error creating: pods
"falco-7xk2p" is forbidden:
violates PodSecurity "baseline:latest": hostPath volumes (volumes "container-engine-
socket-0",
"boot-fs", "lib-modules", "usr-fs", "etc-fs", "sys-fs", "proc-fs"), privileged
(container "falco"
must not set securityContext.privileged=true)
```

CAUSE

Talos enforces `baseline` cluster-wide. Falco needs a privileged container to load the eBPF driver, and `hostPath` mounts for `/proc`, `/etc` and the containerd socket. `baseline` forbids both. The DaemonSet itself is accepted, because `enforce` applies only to pods.

Fix

FILE `szenarien/12-falco/namespace-falco.yaml`

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: falco
  labels:
    pod-security.kubernetes.io/enforce: privileged
    pod-security.kubernetes.io/enforce-version: latest
```

MAC — Step 5

```
kubectl --context talos apply -f szenarien/12-falco/namespace-falco.yaml
kubectl --context talos rollout status ds/falco -n falco --timeout=5m
```

`kubectl apply` warns that the namespace created by Helm is missing the annotation `kubectl.kubernetes.io/last-applied-configuration`, and adds it. That is harmless. For a fresh installation, you create the namespace before running Helm.

WHAT YOU SHOULD SEE

`kubectl --context talos get ds -n falco` shows `2 2 2`, and the log contains `Opening 'syscall' source with modern BPF probe.`

RKE2

RKE2 without the CIS profile enforces `privileged`, so Falco starts right away. With `profile: cis`, `restricted` applies, and you need the same label as on Talos.

TALOS

Instead of the namespace label, you can also add `falco` to `exemptions.namespaces` in the machine config under `cluster.apiServer.admissionControl`. The label is more visible and does not need a patch of the nodes. `driver.kind: kmod` does not work on Talos, because you cannot load kernel modules there afterwards.

12.4 Alert After `kubectl exec`

A colleague opens a shell in a pod. Falco fires. Your job is to read from the alert where this happened.

FILE `szenarien/12-falco/kaputt-shell.yaml (excerpt)`

```yaml
    containers:
      - name: web
        image: busybox:1.37
        command: ["sh", "-c", "while true; do sleep 3600; done"]
```

MAC

Step 6

```
kubectl --context rke2 apply -f szenarien/12-falco/kaputt-shell.yaml
kubectl --context rke2 exec -it -n dbg-12 deploy/web -- sh -c 'id; exit'
```

`-it` gives the shell a terminal. That is exactly what the rule `Terminal shell in container` looks for.

SYMPTOM

A JSON line appears in the Falco logs. This excerpt shortens the text under `output`:

```
{"hostname":"rke2-w1","output":"14:21:37.482915063: Notice A shell was spawned in a
container
with an attached terminal | evt_type=execve user=root user_uid=0 user_loginuid=-1
process=sh
proc_exepath=/bin/sh ... command=sh -c id; exit terminal=34816 ...
container_id=8f2c41d9a7b3
container_name=web k8s_ns=dbg-12 k8s_pod=web-5b9d8c7f6-tx4lp","priority":"Notice",
"rule":"Terminal shell in container","source":"syscall", ...}
```

Investigate

With many messages, `jq` helps. The fields under `output_fields` carry the names of the Falco fields:

MAC — Step 7

```
kubectl --context rke2 logs -n falco -l app.kubernetes.io/name=falco -c falco --tail=500 \
  | jq -cR 'fromjson? | select(.rule=="Terminal shell in container")
      | {hostname, priority,
         ns: .output_fields["k8s.ns.name"], pod: .output_fields["k8s.pod.name"],
         id: .output_fields["container.id"], cmd: .output_fields["proc.cmdline"],
         user: .output_fields["user.name"]}'
```

`fromjson?` skips the text lines from the startup. What remains is one compact line per alert.

```
{"hostname":"rke2-w1","priority":"Notice","ns":"dbg-12","pod":"web-5b9d8c7f6-tx4lp",
 "id":"8f2c41d9a7b3","cmd":"sh -c id; exit","user":"root"}
```

Field	What it tells you
`hostname`	The node on which Falco saw the call. In the chart, this is the node name.
`k8s.ns.name`, `k8s.pod.name`	Namespace and pod. Both come through the container runtime, not from the Kubernetes API.
`container.id`	The first twelve characters of the container ID. Falco reads it directly from the cgroups, so it is there even without runtime access.
`proc.cmdline`	The command line that was run.
`user.name`, `user.loginuid`	The user **inside** the container. `-1` means: no login, typical for `kubectl exec`.

The container ID is the reliable anchor if pod names are missing or the pod is long gone:

MAC — Step 8

```
kubectl --context rke2 get pods -A \
  -o custom-
columns=NS:.metadata.namespace,POD:.metadata.name,ID:.status.containerStatuses[*].containerID \
  | grep 8f2c41d9a7b3
```

`containerID` has the form `containerd://8f2c41d9a7b3...`. The prefix from Falco is enough for `grep`.

RKE2-W1 — Step 9

```
crictl ps -a | grep 8f2c41d9a7b3
```

On the node from `hostname`, after the exports from Chapter 2 (`CRI_CONFIG_FILE`). For Talos:

`talosctl -n 172.16.0.171 containers --namespace cri | grep 8f2c41d9a7b3`.

K9S — Step 10

```
:ds falco⏎      Enter, select pod on the right node, l = logs
/Terminal⏎      filter logs by the rule
```

In the k9s log view, `/` filters the lines. For analysis across several pods, `kubectl` with `jq` is more convenient.

CAUSE

Not a failure in the strict sense: the rule `Terminal shell in container` reports every shell process with a terminal that enters a container as an entry point. `kubectl exec -it` does exactly that. Falco does not see **who** issued the command. For that you need the API server's audit log, matched by time, namespace and pod.

WHAT YOU SHOULD SEE

For every alert you can name the node, namespace, pod and container. A `kubectl exec` **without** `-it`, for example `kubectl exec -n dbg-12 deploy/web -- id`, does not trigger this rule.

12.5 False Positive From a Legitimate Job

A cleanup Job deletes finished pods every ten minutes. For that it talks to the Kubernetes API with `kubectl`. The name `aufraeumer` means "cleaner".

FILE `szenarien/12-falco/kaputt-cronjob.yaml (excerpt)`

```yaml
kind: CronJob
metadata:
  name: aufraeumer
  namespace: dbg-12
spec:
  schedule: "*/10 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: aufraeumer
          containers:
            - name: kubectl
              image: registry.k8s.io/kubectl:v1.36.6
              args: ["delete", "pods", "-n", "dbg-12", "--field-selector=status.phase==Succeeded"]
```

In the file, `args` is a list over several lines. Role and RoleBinding give the ServiceAccount exactly `get`, `list` and `delete` on pods.

MAC — Step 11

```
kubectl --context rke2 apply -f szenarien/12-falco/kaputt-cronjob.yaml
kubectl --context rke2 create job -n dbg-12 --from=cronjob/aufraeumer aufraeumer-jetzt
```

The second command starts the Job right away (`jetzt` means "now") instead of waiting for the next full tenth minute.

SYMPTOM

```
14:40:02.118204551: Notice Unexpected connection to K8s API Server from container |
connection=10.42.1.23:51734->10.43.0.1:443 lport=51734 rport=443 fd_type=ipv4
fd_proto=tcp
evt_type=connect user=root ... process=kubectl ... command=kubectl delete pods -n dbg-12
--field-selector=status.phase==Succeeded ... container_name=kubectl k8s_ns=dbg-12
k8s_pod=aufraeumer-jetzt-q8w2n
```

From now on six times per hour. Anyone who sees this for a week stops reading Falco alerts.

Investigate

Which condition fired? The rule is in the default rules inside the pod:

MAC — Step 12

```
kubectl --context rke2 exec -n falco ds/falco -c falco -- \
  grep -A 12 'rule: Contact K8S API Server From Container' /etc/falco/falco_rules.yaml
```

Shows the rule as it was loaded. The init container `falcoctl-artifact-install` installs the rules files.

```
- rule: Contact K8S API Server From Container
  desc: > ...
  condition: >
    evt.type=connect
    and (fd.typechar=4 or fd.typechar=6)
    and container
    and k8s_api_server
    and not k8s_containers
    and not user_known_contact_k8s_api_server_activities
```

`k8s_api_server` checks whether the target is `kubernetes.default.svc.cluster.local`. `k8s_containers` is a fixed list of known images, and our Job is not on it.

CAUSE

The rule is loud on purpose: a container that talks to the API could be an attacker with a stolen token. But our Job does this legitimately and regularly. Without an exception it produces noise that hides real hits of the same rule.

Fix

Do not turn off the rule. Add a narrow exception instead: this namespace, this image, this process.

FILE `szenarien/12-falco/fix-ausnahme-values.yaml`

```
customRules:
  dbg-ausnahmen.yaml: |-
    - rule: Contact K8S API Server From Container
      exceptions:
        - name: dbg12_aufraeumer
          fields: [k8s.ns.name, container.image.repository, proc.name]
```

```
      comps: [=, =, =]
      values:
        - [dbg-12, registry.k8s.io/kubectl, kubectl]
  override:
    exceptions: append
```

`override: exceptions: append` appends the exception to the existing rule without touching its condition or output. Each line under `values` is a tuple that matches `fields` and `comps`. The chart puts `customRules` into `/etc/falco/rules.d`, which Falco loads after the default rules. The old `append: true` is deprecated.

MAC

Step 13

```
helm --kube-context rke2 upgrade falco falcosecurity/falco -n falco --version 9.2.2 \
  -f werkzeuge/falco-values-rke2.yaml -f szenarien/12-falco/fix-ausnahme-values.yaml --wait
```

The Falco pods restart with the additional rules file. Do not leave out the second file in future upgrades, or the exception is gone again.

WHAT YOU SHOULD SEE

The startup log contains `/etc/falco/rules.d/dbg-ausnahmen.yaml | schema validation: ok`. A new run with `kubectl --context rke2 create job -n dbg-12 --from=cronjob/aufraeumer aufraeumer-2` stays quiet. The rule itself is still alive:

```
kubectl --context rke2 run apitest -n dbg-12 --image=busybox:1.37 --restart=Never -- wget -T 2 -q -O /dev/
null https://kubernetes.default.svc
```

still triggers it.

PITFALL

If a new exception is missing `fields` or `values`, Falco refuses to load the rules. The pod then does not start, and the log says `Rule exception must have fields property with a list of fields`. Check the startup log after every rule change. For simple cases there is a second way: fill the empty macro `user_known_contact_k8s_api_server_activities` with `override: condition: replace`.

12.6 No Kubernetes Metadata

Someone “unifies” the Falco values for both clusters and sets the socket to the default path. On Talos it is correct, on RKE2 it is not.

FILE `szenarien/12-falco/kaputt-values-sockets.yaml`

```
collectors:
  containerEngine:
    engines:
      cri:
        enabled: true
        sockets: ["/run/containerd/containerd.sock"]
```

MAC Step 14

```
helm --kube-context rke2 upgrade falco falcosecurity/falco -n falco --version 9.2.2 \
  -f werkzeuge/falco-values-rke2.yaml -f szenarien/12-falco/kaputt-values-sockets.yaml --wait
kubectl --context rke2 exec -it -n dbg-12 deploy/web -- sh -c 'exit'
```

The values are merged in the order of the `-f` flags, and the last file wins. Then you trigger the shell alert again.

SYMPTOM

```
14:58:10.903311277: Notice A shell was spawned in a container with an attached terminal
| ...
container_id=3c7e90b1f2a4 container_name=<NA> k8s_ns=<NA> k8s_pod=<NA>
```

Falco runs, all pods are `Running`, the alerts arrive. Only the pod is missing, and an exception with `k8s.ns.name` like the one in the last scenario no longer applies.

Investigate

`container_id` is filled, because it comes from the kernel. Everything that comes from the runtime is missing. So you ask: which socket does Falco query, and does it exist on the node?

MAC Step 15

```
kubectl --context rke2 get ds falco -n falco -o yaml \
  | yq '.spec.template.spec.volumes[] | select(.hostPath) | .hostPath.path'
```

The chart mounts exactly the directories of the configured sockets under `/host` in the container.

```
/run/containerd
/boot
/lib/modules
/usr
/etc
/sys/kernel
/proc
```

RKE2-W1 Step 16

```
grep runtime-endpoint /var/lib/rancher/rke2/agent/etc/crictl.yaml
ls -l /run/containerd/ /run/k3s/containerd/containerd.sock
```

The crictl configuration of RKE2 names the real socket. `/run/containerd/` is empty or missing.

```
runtime-endpoint: unix:///run/k3s/containerd/containerd.sock
/run/containerd/:
total 0
srw-rw---- 1 root root 0 Nov  9 08:11 /run/k3s/containerd/containerd.sock
```

CAUSE

RKE2 ships its own containerd, whose socket is under `/run/k3s/containerd/`. The `container` plugin queries only the configured path and reaches no runtime there. Without an answer from the runtime, all container and pod fields stay empty, and Falco shows `<NA>`. The kernel still delivers system calls and the container ID from the cgroups, which is why everything looks healthy.

Fix

MAC — Step 17

```
helm --kube-context rke2 upgrade falco falcosecurity/falco -n falco --version 9.2.2 \
  -f werkzeuge/falco-values-rke2.yaml -f szenarien/12-falco/fix-ausnahme-values.yaml --wait
```

Without the broken file, `/run/k3s/containerd/containerd.sock` from the RKE2 values file applies again. The exception stays in place.

WHAT YOU SHOULD SEE

A new `kubectl exec -it` again produces `container_name=web k8s_ns=dbg-12 k8s_pod=web-...`. The DaemonSet lists `/run/k3s/containerd` among the `hostPath` volumes.

RKE2

Socket of the kubelet runtime: `/run/k3s/containerd/containerd.sock`. You can look it up in `/var/lib/rancher/rke2/agent/etc/crictl.yaml`. The chart's default values already contain this path. The failure only happens when someone overrides the list.

TALOS

Talos runs two containerd instances: one for system services under `/system/run/containerd/` and one for the kubelet pods under `/run/containerd/containerd.sock`. Falco needs the second one. You can check it with `talosctl -n 172.16.0.171 containers --namespace cri`.

WHY IT MATTERS

In the lab, Falco writes to standard output. In production you send alerts to a central system, for example with Falcosidekick, which is included in the chart as an option. For troubleshooting this changes nothing: the fields are the same, and missing metadata stands out there just as much, only later.

TRY IT YOURSELF

On Talos, open a shell in the pod `web` (scenario from this chapter, `--context talos`). Read the node, pod and container ID from the alert, and find the container with `talosctl ... containers --namespace cri`. Which of the two Talos nodes was it?

Hint: `hostname` in the JSON is the node name, but `talosctl -n` needs the IP. `kubectl --context talos get nodes -o wide` shows the mapping.

MAC

Step 18

```
kubectl --context rke2 delete ns dbg-12
kubectl --context talos delete ns dbg-12
```

Cleans up the workloads. Falco can stay installed. You would remove it with `helm --kube-context rke2 uninstall falco -n falco`.

WHAT YOU LEARNED

- Falco needs `privileged` and `hostPath`. On Talos (default `baseline`), the namespace `falco` therefore has to be set to `privileged`, or the DaemonSet stays at `0`.
- `hostname`, `k8s.ns.name`, `k8s.pod.name` and `container.id` trace every alert back to the node, pod and container. Only the audit log knows the user who triggered it.
- You silence false positives with a narrow exception: `exceptions` with `override: exceptions: append` in `customRules`.
- `<NA>` for pod and namespace means: Falco cannot reach the container runtime. On RKE2 the socket is under `/run/k3s/containerd/`.

CHAPTER 13

Nodes

You recognize when a failure is not in the pod but in the node, investigate the kubelet and the container runtime via SSH on RKE2 and via `talosctl` on Talos, and understand why pods hang for minutes after a node failure.

Time: 75 minutes **Requires:** Chapters 2 and 3

13.1 What Breaks in This Chapter

So far the cause was always in a manifest. In this chapter it sits one layer deeper: in the kubelet, which starts and monitors the pods on every node and reports to the API server. If it fails, the cluster only hears silence. And Kubernetes interprets silence cautiously, with deadlines you need to know.

Scenario	Symptom	Where the answer is
Kubelet not running	Node `NotReady`	`describe node`, then the kubelet log on the node
Node failure	Pods hang in `Terminating`, replacement only after 5 minutes	Taints of the node, tolerations of the pod
Local storage full	Pods `Error`, reason `Evicted`, new pods keep coming	`.status.message` of the pod, events
Drain hangs	`kubectl drain` retries endlessly	`kubectl get pdb`, column `ALLOWED DISRUPTIONS`

All scenarios run in the namespace `dbg-13`. It has the label `pod-security.kubernetes.io/enforce: privileged`. Only the debug pod from the eviction scenario on Talos needs this. The applications themselves comply with `restricted`.

13.2 The Toolbox for Nodes

On an RKE2 node, you log in via SSH and work with the familiar Linux tools. On Talos there is no shell. Every question to the node goes out as an API call through `talosctl`. The table compares the commands you need in this chapter.

Question	RKE2 (on the node, as root)	Talos (on the Mac)
Is the kubelet running?	`systemctl status rke2-agent` (server: `rke2-server`)	`talosctl -n <IP> service kubelet`
Kubelet log	`/var/lib/rancher/rke2/agent/logs/kubelet.log`	`talosctl -n <IP> logs kubelet`
Service log	`journalctl -u rke2-agent`	`talosctl -n <IP> services`, `logs <dienst>`
Kernel messages	`dmesg -T`	`talosctl -n <IP> dmesg`
Containers on the node	`crictl ps -a`, `crictl logs <id>`	`talosctl -n <IP> containers --namespace cri`
Live overview	`top`, `df -h`	`talosctl -n <IP> dashboard`
Everything for support	Collect `journalctl` and logs by hand	`talosctl -n <IP> support`

For RKE2, the environment from Chapter 2 applies: `sudo -i`, then set `KUBECONFIG`, `PATH` and `CRI_CONFIG_FILE`. Without `CRI_CONFIG_FILE`, `crictl` does not find the RKE2 containerd socket.

PITFALL

Since Talos 1.14, the switch for Kubernetes containers in `talosctl containers` and `talosctl logs` is `--namespace cri`. The older `-k` still works but is marked as deprecated. Also, starting with 1.14, `talosctl support` encrypts the archive by default for the Sidero Labs support team. For your own analysis, you need `--no-encryption` or your own recipients via `--encryption-recipients`.

13.3 Node Becomes NotReady

The kubelet checks in with the API server regularly: through a Lease object in the namespace `kube-node-lease` and through the status of the Node object. If these check-ins stop, the node lifecycle controller in the controller manager waits for a grace period, the `node-monitor-grace-period`. In Kubernetes 1.36 it is 50 seconds by default (it used to be 40). After that, it sets all conditions of the node to `Unknown`.

On RKE2, you halt the kubelet by stopping the agent. On Talos that is not so easy, so you build a failure that happens often in practice: a typo in the kubelet arguments.

FILE `labor/talos/patch-kubelet-tippfehler.yaml`

```yaml
apiVersion: v1alpha1
kind: KubeletConfig
extraArgs:
  max-pod: "150"
```

`tippfehler` means “typo”: `max-pods` was intended. Since Talos 1.14, the kubelet configuration lives in its own document `KubeletConfig`. The old field `machine.kubelet` is deprecated.

MAC Step 1

```
kubectl --context rke2 apply -f szenarien/13-nodes/web.yaml
kubectl --context talos apply -f szenarien/13-nodes/web.yaml
ssh <user>@172.16.0.161
```

Creates the test application `web` on both clusters and opens a session on `rke2-w1`.

RKE2-W1 Step 2

```
sudo systemctl stop rke2-agent
```

Stops the RKE2 agent and with it the kubelet. The pods' containers usually keep running, but nobody reports them anymore.

MAC Step 3

```
talosctl -n 172.16.0.171 patch mc --patch @labor/talos/patch-kubelet-tippfehler.yaml
```

Talos applies the change without rebooting the node and restarts the kubelet with the new argument.

SYMPTOM

```
$ kubectl --context rke2 get nodes
NAME      STATUS     ROLES                       AGE   VERSION
rke2-cp   Ready      control-plane,etcd,master   9d    v1.36.6+rke2r1
rke2-w1   NotReady   <none>                      9d    v1.36.6+rke2r1
```

After about a minute, `rke2-w1` is `NotReady`, and so is `talos-w1` in the cluster `talos`. The pods on it still show `Running`.

Investigate

Your first look goes to the Node object. It tells you since when and why the node is no longer ready.

MAC Step 4

```
kubectl --context rke2 describe node rke2-w1 | sed -n '/^Taints/,/^Addresses/p'
```

Shows taints and conditions. The rest of the output (capacity, pods, events) is secondary here.

```
Taints:             node.kubernetes.io/unreachable:NoExecute
                    node.kubernetes.io/unreachable:NoSchedule
Unschedulable:      false
...
Conditions:
Type             Status    ... Reason              Message
----             ------    ... ------              -------
MemoryPressure   Unknown   ... NodeStatusUnknown   Kubelet stopped posting node status.
DiskPressure     Unknown   ... NodeStatusUnknown   Kubelet stopped posting node status.
PIDPressure      Unknown   ... NodeStatusUnknown   Kubelet stopped posting node status.
Ready            Unknown   ... NodeStatusUnknown   Kubelet stopped posting node status.
```

`Unknown` means the controller does not know, because the kubelet is silent. If it says `False` with a message from the kubelet instead, the kubelet is running and reports a problem itself, such as a missing CNI. This distinction saves a lot of time. The taint `unreachable` goes with the status `Unknown`. With `False`, the controller sets `node.kubernetes.io/not-ready`.

In k9s you see the same thing in the node view:

K9S — Step 5

```
:nodes⏎            Knoten auswählen, d = describe
```

Select the node (`Knoten auswählen`), then `d` for describe. The `STATUS` column shows `NotReady`. In the describe view, `/` and `Conditions` take you to the right place quickly.

Because the kubelet does not answer, `kubectl` does not help here. `kubectl debug node/rke2-w1` fails too: the debug pod would have to be started by exactly the kubelet that is down. You have to go to the node.

RKE2-W1 — Step 6

```
systemctl status rke2-agent --no-pager
journalctl -u rke2-agent --since "15 min ago" --no-pager | tail -n 20
tail -n 20 /var/lib/rancher/rke2/agent/logs/kubelet.log
```

The state of the service, its latest messages and the end of the kubelet log. `inactive (dead)` with a `Stopped` entry in the journal means someone halted the service. A crash would look different, with an exit code and restart attempts.

CAUSE

On `rke2-w1` the kubelet is not running, because the service `rke2-agent` was stopped. Without a kubelet, nobody renews the node's Lease. After 50 seconds, the node lifecycle controller sets the conditions to `Unknown` and adds the taint `node.kubernetes.io/unreachable`. New pods no longer land on the node, and the old ones stay where they are for now.

The Same on Talos

On Talos you do not ask `systemd`, but the Talos service manager. The kubelet is a service of its own there, just like `etcd`, `containerd` or `apid`.

MAC — Step 7

```
talosctl -n 172.16.0.171 services
talosctl -n 172.16.0.171 service kubelet
talosctl -n 172.16.0.171 logs kubelet --tail 20
```

A list of all services with state and health, then the kubelet details with its latest events, and finally its log.

In the service status, starts and failures alternate: Talos keeps restarting the kubelet. The log gives the reason, roughly:

```
Error: failed to parse kubelet flag: unknown flag: --max-pod
```

CAUSE

The kubelet aborts while parsing its arguments, because it does not know `--max-pod`. Talos passes `extraArgs` through unchecked. It only rejects arguments it manages itself, such as `kubeconfig` or `container-runtime-endpoint`. The same happens with flags that a new Kubernetes version has removed. After an upgrade, that is a typical cause of `NotReady`.

Fix

RKE2-W1 — Step 8

```
sudo systemctl start rke2-agent
```

Starts the agent again. The kubelet checks in within a few seconds.

MAC — Step 9

```
talosctl -n 172.16.0.171 edit mc
```

Opens the machine config in the editor. In the document `KubeletConfig`, delete the line `max-pod: "150"` (and `extraArgs:` if it becomes empty), save and close. Talos applies the change immediately.

WHAT YOU SHOULD SEE

`kubectl --context rke2 get nodes` and `kubectl --context talos get nodes` show `Ready` for both again. The taint `unreachable` is gone, and `talosctl -n 172.16.0.171 service kubelet` reports `STATE Running` and `HEALTH OK`.

RKE2

The kubelet, containerd and the supervisor run as child processes of `rke2-agent` (on the server: `rke2-server`). Logs are files under `/var/lib/rancher/rke2/agent/logs/` and in the journal. You set kubelet arguments with `kubelet-arg` in `/etc/rancher/rke2/config.yaml`, then restart the service.

TALOS

The kubelet is a Talos service, not a process you can touch. Status and log come only through `talosctl service` and `talosctl logs`. Arguments live in the document `KubeletConfig`. If even the Talos API is gone, only the VM console with the dashboard remains.

13.4 Pods Hang After a Node Failure

A `NotReady` node does not lose its pods right away. When a pod is created, the admission plugin `DefaultTolerationSeconds` gives it two tolerations: it tolerates the taints `not-ready` and `unreachable` for 300 seconds. Only after that does the taint eviction controller remove it. This time you leave the node down long enough.

MAC — Step 10

```
kubectl --context rke2 get pod -n dbg-13 -l app=web \
  -o jsonpath='{.items[0].spec.tolerations}' | jq
```

Shows the tolerations of a `web` pod. The manifest contains not a single one. Both come from the API server.

Two entries, for `node.kubernetes.io/not-ready` and `node.kubernetes.io/unreachable`, both with `"effect": "NoExecute"` and `"tolerationSeconds": 300`.

RKE2-W1 — Step 11

```
sudo systemctl stop rke2-agent
```

The same failure as before. This time you wait six minutes before you continue.

SYMPTOM

```
$ kubectl --context rke2 get pods -n dbg-13 -o wide
NAME                   READY   STATUS        RESTARTS   AGE   IP           NODE
web-7c9d8f6b5d-4kq2m   1/1     Running       0          31m   10.42.0.37   rke2-cp
web-7c9d8f6b5d-r8vtn   1/1     Terminating   0          31m   10.42.1.12   rke2-w1
web-7c9d8f6b5d-x7lpd   1/1     Running       0          47s   10.42.0.41   rke2-cp
```

For five minutes, the pod on `rke2-w1` appeared healthy and `Running`. Now it hangs in `Terminating`, and only now is there a replacement on `rke2-cp`.

Investigate

MAC — Step 12

```
kubectl --context rke2 events -n dbg-13 --for pod/web-7c9d8f6b5d-r8vtn
```

The events of the hanging pod. Take the name from your own output.

```
LAST SEEN   TYPE     REASON                  OBJECT                      MESSAGE
5m10s       Normal   TaintManagerEviction    pod/web-7c9d8f6b5d-r8vtn    Marking ...
```

The full message is `Marking for deletion Pod dbg-13/web-7c9d8f6b5d-r8vtn`. The controller has marked the pod for deletion. It is only deleted once the kubelet confirms that the containers are gone. But that kubelet is not running. So the pod stays in `Terminating` until the node comes back or someone removes the Node object.

K9S Step 13

```
:pods dbg-13⏎      Spalte NODE beachten, Pod wählen, d = describe
:nodes⏎            rke2-w1 wählen, d = describe, Taints prüfen
```

Watch the `NODE` column, select the pod and press `d`. Then select `rke2-w1` in the node view, press `d` and check the taints. In the pod list, the replacement pod with its small `AGE` stands out immediately.

CAUSE

The node was unreachable, so the controller set the taint `node.kubernetes.io/unreachable:NoExecute`. The pods tolerate it for 300 seconds. After that, the controller marks them for deletion. Only the kubelet of the failed node can confirm that, so they hang in `Terminating`. The replacement is created anyway, because the ReplicaSet controller no longer counts pods that have a `deletionTimestamp`.

WHY IT MATTERS

The waiting time is intentional. Kubernetes cannot tell a dead node from a network problem. Maybe the containers are still running and writing to a database. With a StatefulSet, a premature replacement would create two pods with the same identity. That is why no replacement appears there even after 300 seconds, as long as the old pod has not really been deleted.

Fix

The immediate fix is to bring the node back. As soon as the kubelet runs again, it terminates the marked pods and confirms the deletion.

RKE2-W1 Step 14

```
sudo systemctl start rke2-agent
```

The hanging pod disappears after a few seconds.

If the node is lost for good, you delete the Node object with `kubectl --context rke2 delete node rke2-w1`. Then the garbage collector removes the pods that were bound to the node that no longer exists. For stateless applications, you can also set the deadline yourself:

FILE szenarien/13-nodes/fix-toleranz.yaml (excerpt)

```yaml
    tolerations:
      - key: node.kubernetes.io/unreachable
        operator: Exists
        effect: NoExecute
        tolerationSeconds: 30
      # ... the same for node.kubernetes.io/not-ready
```

If the toleration is already in the manifest, the API server does not add its own.

MAC Step 15

```
kubectl --context rke2 apply -f szenarien/13-nodes/fix-toleranz.yaml
```

Rolls out `web` again with the shorter tolerations.

WHAT YOU SHOULD SEE

If you repeat the failure, the node shows `NotReady` after about 50 seconds, and 30 seconds later the replacement pod is on `rke2-cp`. Start the agent again afterwards.

RKE2

`systemctl stop rke2-agent` simulates a hard failure well: the kubelet is gone, the containers keep running. Only `rke2-killall.sh` would stop them too.

TALOS

`talosctl shutdown` and `talosctl reboot` cordon and drain the node first. The pods then move cleanly, and the scenario does not occur. Only `--force` skips cordon and drain. To simulate a real failure, hard power off the VM in the hypervisor.

13.5 Pods Get Evicted

The kubelet protects its node. If memory runs low, the disk fills up (inodes included) or the process IDs run out, it sets the conditions `MemoryPressure`, `DiskPressure` or `PIDPressure` and throws pods out. This throwing out is called eviction. But it also kicks in without any node shortage, when a pod exceeds its own limit for local storage (`ephemeral-storage`). That is the scenario here, because you do not really want to provoke a full disk in the lab.

FILE szenarien/13-nodes/kaputt-evicted.yaml (excerpt)

```
        command:
          - sh
          - -c
          - dd if=/dev/zero of=/daten/block bs=1M count=200 && sleep 3600
        resources:
          limits:
            ephemeral-storage: 100Mi
        volumeMounts:
          - name: daten
            mountPath: /daten
    volumes:
      - name: daten
        emptyDir: {}
```

200 MiB into an `emptyDir` (`daten` means “data”), with 100 MiB allowed. An `emptyDir` without its own medium lives on the node’s disk and counts toward the pod’s local storage.

MAC Step 16

```
kubectl --context rke2 apply -f szenarien/13-nodes/kaputt-evicted.yaml
```

Creates the Deployment `fueller` ("filler"). It behaves the same on Talos.

SYMPTOM

```
$ kubectl --context rke2 get pods -n dbg-13 -l app=fueller
NAME                       READY   STATUS              RESTARTS   AGE
fueller-6f8b7d9c4b-2xjzq   0/1     Error               0          2m4s
fueller-6f8b7d9c4b-9tqwe   0/1     Error               0          71s
fueller-6f8b7d9c4b-mz5rk   1/1     Running             0          9s
```

A new pod every few seconds, and the old ones stay behind with `Error`. Depending on the timing, you also see `ContainerStatusUnknown`.

Investigate

The `STATUS` column shows the state of the container. The reason the pod was terminated is one level higher, in the pod status.

MAC Step 17

```
kubectl --context rke2 get pods -n dbg-13 -l app=fueller \
  -o custom-columns=NAME:.metadata.name,PHASE:.status.phase,REASON:.status.reason
kubectl --context rke2 get pod -n dbg-13 <pod> -o jsonpath='{.status.message}{"\n"}'
```

The phase and reason of every pod, then the message of one of the pods with `Error`.

```
NAME                       PHASE     REASON
fueller-6f8b7d9c4b-2xjzq   Failed    Evicted
fueller-6f8b7d9c4b-9tqwe   Failed    Evicted
fueller-6f8b7d9c4b-mz5rk   Running   <none>
```

```
Pod ephemeral local storage usage exceeds the total limit of containers 100Mi.
```

The same message appears as an event with the reason `Evicted` on the pod. In k9s, open one of the pods with `d`. At the top of the describe view you see `Status: Failed`, `Reason: Evicted` and the message.

With a real node shortage, the message looks different, for example `The node was low on resource: ephemeral-storage.`, and the node shows `DiskPressure=True` with the taint `node.kubernetes.io/disk-pressure`. You can read the thresholds at which the kubelet steps in directly from its running configuration through the API server:

MAC Step 18

```
kubectl --context rke2 get --raw /api/v1/nodes/rke2-w1/proxy/configz \
  | jq .kubeletconfig.evictionHard
```

The `configz` endpoint returns the active kubelet configuration. This works the same on both distributions.

```
{
  "imagefs.available": "15%",
  "imagefs.inodesFree": "5%",
  "memory.available": "100Mi",
  "nodefs.available": "10%",
  "nodefs.inodesFree": "5%"
}
```

These are the Kubernetes defaults. If free space drops below 10 percent, the kubelet starts cleaning up: first unused images and dead containers, then pods. To see how full the disk really is on RKE2, run `df -h /var/lib` on the node. Without SSH, a debug pod works too:

MAC — Step 19

```
kubectl --context rke2 debug node/rke2-w1 -n dbg-13 -it \
  --image=busybox:1.37 --profile=sysadmin -- df -h /host/var/lib
```

Starts a privileged pod in the host network of `rke2-w1`. The node's file system is under `/host`. The pod is named `node-debugger-rke2-w1-` plus a random suffix and stays behind as `Completed` afterwards. Delete it with `kubectl delete pod` or in k9s with `Ctrl-d`.

CAUSE

With its `emptyDir`, the container uses more local storage than `limits.ephemeral-storage` allows. The kubelet checks this on every pass of its eviction manager and terminates the pod immediately, without a grace period. An evicted pod does not restart: it stays behind in phase `Failed`, and the Deployment replaces it with a new one that does the same thing.

Fix

The fix sets the limit to the real need and also limits the volume directly. With `sizeLimit` on the `emptyDir`, the message on overflow is `Usage of EmptyDir volume "daten" exceeds the limit "250Mi".`. That tells you right away which volume is to blame.

MAC — Step 20

```
kubectl --context rke2 apply -f szenarien/13-nodes/fix-evicted.yaml
kubectl --context rke2 delete pods -n dbg-13 --field-selector=status.phase=Failed
```

A new limit of 300 MiB and `sizeLimit: 250Mi` on the volume. The second line removes the evicted pods left behind. Otherwise Kubernetes only deletes them once the pod garbage collector reaches a very high threshold.

WHAT YOU SHOULD SEE

A single `fueller` pod stays `Running`, `RESTARTS` stays at 0, and no new pods appear.

RKE2

`kubectl debug node/...` works out of the box, and with `chroot /host` you even get the node's tools. On the node itself, `df -h` and `crictl stats` show the usage.

TALOS

`kubectl debug node/...` needs a namespace with `enforce: privileged`, otherwise the cluster-wide PSA `baseline` rejects the pod. In newly created 1.14 clusters, pods also run in their own sandbox (`workloadIsolation`) and do not see everything on the host. It is better to go directly through the Talos API: `talosctl mounts` shows the usage, and `talosctl debug <image>` opens a debug shell, without any kubelet.

13.6 Drain Hangs on a PodDisruptionBudget

Before maintenance, you empty a node with `kubectl drain`. The command cordons the node and then evicts every pod through the Eviction API. Before it does, this API asks all PodDisruptionBudgets whether the pod may go. A PDB that is too strict always says no.

FILE `szenarien/13-nodes/kaputt-pdb.yaml (excerpt)`

```yaml
kind: Deployment
metadata:
  name: api
spec:
  replicas: 2
# ...
---
kind: PodDisruptionBudget
metadata:
  name: api
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: api
```

Two replicas, and the PDB demands that both are always available. That looks like maximum safety and forbids every voluntary disruption.

MAC Step 21

```
kubectl --context rke2 apply -f szenarien/13-nodes/kaputt-pdb.yaml
kubectl --context rke2 drain rke2-w1 --ignore-daemonsets \
  --delete-emptydir-data --timeout=90s
```

`--ignore-daemonsets` is necessary because DaemonSet pods like `rke2-canal` run on every node. `--delete-emptydir-data` allows evicting pods with an `emptyDir`, such as `fueller`. The timeout makes sure the command gives up at some point.

SYMPTOM

```
node/rke2-w1 cordoned
evicting pod dbg-13/api-5b8f9c7d6-hx2lw
error when evicting pods/"api-5b8f9c7d6-hx2lw" -n "dbg-13" (will retry after 5s):
  Cannot evict pod as it would violate the pod's disruption budget.
evicting pod dbg-13/api-5b8f9c7d6-hx2lw
error when evicting pods/"api-5b8f9c7d6-hx2lw" -n "dbg-13" (will retry after 5s):
  Cannot evict pod as it would violate the pod's disruption budget.
...
```

The same attempt every five seconds, until `global timeout reached` appears after 90 seconds. If both `api` pods happen to be on `rke2-cp`, the drain goes through. In that case, check with `kubectl get pods -o wide` and drain the node that has an `api` pod.

Investigate

MAC Step 22

```
kubectl --context rke2 get pdb -n dbg-13
```

The `ALLOWED DISRUPTIONS` column is the only one that matters.

```
NAME   MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
api    2               N/A               0                     3m
```

Zero allowed disruptions. That does not change as long as both pods are running. In k9s, you reach the list with `:pdb` and the nodes with `:nodes`. There, `rke2-w1` now shows `Ready,SchedulingDisabled`.

CAUSE

`minAvailable: 2` with two replicas allows zero evictions. So the Eviction API answers with HTTP 429 (`Too Many Requests`), and `kubectl drain` keeps retrying endlessly. A PDB only protects against voluntary disruptions. It does not help against a node failure, and if it is too strict, it blocks maintenance.

Fix

MAC Step 23

```
kubectl --context rke2 apply -f szenarien/13-nodes/fix-pdb.yaml
kubectl --context rke2 drain rke2-w1 --ignore-daemonsets --delete-emptydir-data
kubectl --context rke2 uncordon rke2-w1
```

The new PDB allows one disruption with `maxUnavailable: 1`. The drain goes through, and afterwards you release the node again with `uncordon`. In k9s, cordon, uncordon and drain are `c`, `u` and `r` in the node view.

WHAT YOU SHOULD SEE

`kubectl --context rke2 get pdb -n dbg-13` shows `ALLOWED DISRUPTIONS 1`, and the drain ended with `node/rke2-w1 drained`. After `uncordon`, the node is back to `Ready` without `SchedulingDisabled`.

PITFALL

An aborted drain leaves the node cordoned. New pods then never land on it, and no error shows up anywhere. After every drain, even a failed one, take a look at `kubectl get nodes`.

RKE2

You drain nodes yourself before every maintenance, for example before an RKE2 upgrade. That is exactly where the PDB kicks in.

TALOS

`talosctl reboot`, `shutdown` and `upgrade` cordon and drain the node themselves, also through the Eviction API. So a PDB that is too strict delays these commands as well. Check the PDBs before a Talos upgrade.

13.7 Clean Up

MAC — Step 24

```
kubectl --context rke2 delete ns dbg-13
kubectl --context talos delete ns dbg-13
```

Deletes all objects of this chapter. First check that both workers are `Ready` and none is still cordoned.

TRY IT YOURSELF

A colleague reports: “Since yesterday, no new pod starts on `talos-w1`, but the node is `Ready`.” Reproduce this with `kubectl --context talos cordon talos-w1`, then create the Deployment `web` with four replicas. Find the cause using only `kubectl get nodes`, `describe node` and the pods’ events. Which line in `describe node` gives it away, and which taint goes with it?

Hint: look for `Unschedulable: true` and the taint `node.kubernetes.io/unschedulable:NoSchedule`. With `uncordon`, the problem is gone.

WHAT YOU LEARNED

- `Unknown` in the conditions means the kubelet is silent. `False` means the kubelet is running and reports a problem itself.
- On RKE2 you investigate the kubelet with `systemctl`, `journalctl` and `kubelet.log`. On Talos you use `talosctl services`, `service kubelet` and `logs kubelet`.
- After 50 seconds without a check-in, a node becomes `NotReady`. After another 300 seconds, its pods are marked for deletion and hang in `Terminating` until the kubelet is back.
- Evicted pods have phase `Failed` and reason `Evicted`. The message names the resource and the limit.
- A PDB with `minAvailable` equal to the replica count blocks every drain. `ALLOWED DISRUPTIONS` shows it at a glance.

CHAPTER 14

Control Plane, etcd and Webhooks

You check the API server and etcd on both distributions, back up etcd before every intervention, bring a crashed API server back, and recognize expired certificates, hanging webhooks and namespaces that refuse to die.

Time: 90 minutes **Requires:** Chapters 2, 3 and 13

14.1 What Breaks in This Chapter

Everything you have investigated so far went through the API server. In this chapter, the API server itself is the problem, or something it asks on every request: etcd, a webhook, an aggregated API. Failures at this layer do not hit one pod, but all users of the cluster at the same time.

Scenario	Symptom	Where the answer is
API server does not start	`connection refused` on every `kubectl`	Log of the `kube-apiserver` container on the node
Client certificate expired	`You must be logged in to the server`	`openssl x509 -enddate`, API server log
Webhook unreachable	`failed calling webhook` on create	`ValidatingWebhookConfiguration`, endpoints of the Service
Namespace hangs	`Terminating` without end	Conditions of the namespace, `kubectl get apiservice`

The scenarios run in the namespace `dbg-14`. Two of them touch the control plane itself. So read the section on etcd snapshots before you start.

14.2 Is the API Healthy?

The API server describes its own state through two endpoints. `/livez` answers whether the process is alive or should be restarted. `/readyz` says whether it can serve requests, which includes whether etcd answers. With `?verbose`, it lists every individual check.

MAC Step 1

```
kubectl --context rke2 get --raw='/readyz?verbose'
kubectl --context talos get --raw='/livez?verbose' | tail -n 3
```

Both calls use the same connection as every `kubectl get`, with your admin rights.

```
[+]ping ok
[+]log ok
[+]etcd ok
[+]etcd-readiness ok
[+]informer-sync ok
...
[+]shutdown ok
readyz check passed
```

A failed check is marked with `[-]`, for example `[-]etcd failed: reason withheld`. The API server holds back the reason on purpose. It is in its own log. You can query individual checks directly, for example `/readyz/etcd`.

WHY IT MATTERS

If you get no answer at all, but `The connection to the server 172.16.0.160:6443 was refused - did you specify the right host or port?`, no API server is running on this port. Then no `kubectl` helps anymore. You need a path that does not go through the Kubernetes API: SSH on RKE2, the Talos API on Talos.

14.3 Before Every Intervention: Check and Back Up etcd

etcd is the memory of the cluster. Every object and every Secret lives there. Before you tinker with the control plane, you check whether etcd is healthy and take a snapshot. Both lab clusters have exactly one etcd member, on `rke2-cp` and `talos-cp`.

RKE2

RKE2 runs etcd as a static pod. `etcdctl` is in the image. The client certificates are on the node under `/var/lib/rancher/rke2/server/tls/etcd/` and are mounted into the pod.

RKE2-CP — Step 2

```
etcdcontainer=$(crictl ps --name etcd --quiet)
TLS="--cacert /var/lib/rancher/rke2/server/tls/etcd/server-ca.crt
  --cert /var/lib/rancher/rke2/server/tls/etcd/server-client.crt
  --key /var/lib/rancher/rke2/server/tls/etcd/server-client.key"
crictl exec "$etcdcontainer" etcdctl $TLS endpoint health
crictl exec "$etcdcontainer" etcdctl $TLS endpoint status -w table
crictl exec "$etcdcontainer" etcdctl $TLS alarm list
```

As root in the RKE2 environment from Chapter 2. `$TLS` is deliberately unquoted, because the shell is supposed to split it into separate arguments.

```
127.0.0.1:2379 is healthy: successfully committed proposal: took = 2.4ms
```

`endpoint status` shows the version, database size, leader and Raft index as a table. `alarm list` prints nothing as long as no alarm is active. A `NOSPACE` alarm means the database has reached its quota, and etcd no longer accepts writes.

RKE2-CP Step 3

```
rke2 etcd-snapshot save --name vor-kapitel-14
rke2 etcd-snapshot list
```

A manual snapshot (`vor-kapitel-14` means “before chapter 14”). `list` (alias `ls`) shows the name, location, size and time of all snapshots.

WHAT YOU SHOULD SEE

A line `vor-kapitel-14-rke2-cp-...` with the location `file:///var/lib/rancher/rke2/server/db/snapshots/...`. On top of that come the automatic snapshots, which RKE2 takes by default at midnight and noon, keeping five of them. RKE2 does not clean up manual snapshots automatically.

Talos

On Talos, etcd is not a pod but a Talos service. Everything goes through the Talos API, even when Kubernetes no longer answers at all.

MAC Step 4

```
talosctl -n 172.16.0.170 service etcd
talosctl -n 172.16.0.170 etcd members
talosctl -n 172.16.0.170 etcd status
talosctl -n 172.16.0.170 etcd alarm list
```

Service status, member list, size and leader, and finally the alarms.

```
NODE           MEMBER             DB SIZE   IN USE            LEADER
172.16.0.170   3f1c2b9a7d4e6a10   28 MB     9.1 MB (32.50%)   3f1c2b9a7d4e6a10
```

MAC Step 5

```
mkdir -p ~/talos-backup
talosctl -n 172.16.0.170 etcd snapshot ~/talos-backup/etcd-$(date +%F).snapshot
```

Streams a consistent snapshot to the Mac. The file contains all Secrets of the cluster, so it does not belong in the repo.

RKE2

Restore: `systemctl stop rke2-server`, then `rke2 server --cluster-reset --cluster-reset-restore-path=<snapshot>`, then `systemctl start rke2-server`. RKE2 takes snapshots on a schedule by itself.

TALOS

Restore: wipe the node’s etcd data and bootstrap again with `talosctl bootstrap --recover-from=<snapshot>` (the procedure is in the “Disaster Recovery” guide). Talos has no built-in snapshot schedule. You build one yourself, for example as a cron job with `talosctl`.

14.4 The API Server Does Not Start

A common reason for a dead API server after an upgrade: an argument the new version no longer knows. Feature gates are especially prone to this. Once a feature has gone GA, its gate disappears after a few versions, and if you still set it, you get an API server that aborts at startup. You reproduce this with a made-up gate.

FILE szenarien/14-controlplane/rke2-config-kaputt.yaml (excerpt)

```
kube-apiserver-arg:
  - "feature-gates=DebugBuchGate=true"
```

A drop-in file for RKE2 (`kaputt` means "broken", `Buch` means "book"). It goes into `/etc/rancher/rke2/config.yaml.d/`, and RKE2 reads it after `config.yaml`. To undo it, you simply delete it.

FILE labor/talos/patch-apiserver-featuregate.yaml (excerpt)

```
apiVersion: v1alpha1
kind: KubeAPIServerConfig
extraArgs:
  feature-gates: DebugBuchGate=true
```

The same for Talos. Since 1.14, the document `KubeAPIServerConfig` configures the API server, and `cluster.apiServer` is deprecated.

MAC — Step 6

```
scp szenarien/14-controlplane/rke2-config-kaputt.yaml <user>@172.16.0.160:/tmp/
ssh <user>@172.16.0.160
```

Copies the file to `rke2-cp` and logs you in there.

RKE2-CP — Step 7

```
install -D -m 600 /tmp/rke2-config-kaputt.yaml \
  /etc/rancher/rke2/config.yaml.d/90-kaputt.yaml
systemctl restart rke2-server --no-block
```

As root. `--no-block` returns immediately instead of waiting for a start that never finishes. The workloads keep running, only the control plane is gone.

MAC — Step 8

```
talosctl -n 172.16.0.170 patch mc \
  --patch @labor/talos/patch-apiserver-featuregate.yaml
```

Talos renders the API server's static pod again without rebooting the node.

SYMPTOM

```
$ kubectl --context rke2 get nodes
The connection to the server 172.16.0.160:6443 was refused - did you specify
the right host or port?
```

On the Talos cluster, the same with `172.16.0.170:6443`. Depending on the timing, you may also see a timeout or `EOF`.

Investigate on RKE2

The API server is a container like any other. `crictl` asks containerd directly and does not need an API server.

RKE2-CP — Step 9

```
crictl ps -a --name kube-apiserver
crictl logs $(crictl ps -a --name kube-apiserver --quiet | head -n 1) 2>&1 \
  | tail -n 5
journalctl -u rke2-server --since "5 min ago" --no-pager | tail -n 10
```

The container list shows `Exited` entries with a rising number in the `ATTEMPT` column. The log of the latest attempt names the reason. The journal shows that RKE2 is waiting for the API server.

```
Error: invalid argument "DebugBuchGate=true" for "--feature-gates" flag:
unrecognized feature gate: DebugBuchGate
```

The same logs are available as files under `/var/log/pods/kube-system_kube-apiserver-rke2-cp_*/kube-apiserver/`. The manifest that RKE2 generated from the configuration is in `/var/lib/rancher/rke2/agent/pod-manifests/kube-apiserver.yaml`. A `grep feature-gates` in it confirms where the argument comes from.

Investigate on Talos

MAC — Step 10

```
talosctl -n 172.16.0.170 containers --namespace cri | grep kube-apiserver
talosctl -n 172.16.0.170 logs --namespace cri \
  kube-system/kube-apiserver-talos-cp:kube-apiserver:<id>
```

The first line shows the pod sandbox and the API server container with its ID. Put the ID into the second line. Because the container keeps restarting, the ID changes with every attempt.

The log contains the same message as on RKE2. To see which configuration Talos rendered, use `talosctl -n 172.16.0.170 get staticpods kube-apiserver -o yaml`. For a quick overall picture, run `talosctl -n 172.16.0.170 health`: it gets stuck at the check of the Kubernetes components.

CAUSE

kube-apiserver aborts while parsing its flags, because it does not know the feature gate `DebugBuchGate`. The kubelet keeps restarting the static pod, and every attempt ends immediately. Without an API server, no Kubernetes interface answers anymore. Controllers, the scheduler and `kubectl` stand still, but the running pods keep working.

Fix

RKE2-CP — Step 11

```
rm /etc/rancher/rke2/config.yaml.d/90-kaputt.yaml
systemctl restart rke2-server
```

RKE2 rewrites the API server manifest on every start, now without the gate.

MAC — Step 12

```
talosctl -n 172.16.0.170 edit mc
```

In the document `KubeAPIServerConfig`, delete the line `feature-gates: DebugBuchGate=true`. Talos renders the static pod again immediately.

WHAT YOU SHOULD SEE

`kubectl --context rke2 get --raw='/readyz'` and the same for `talos` answer with `ok`. `talosctl -n 172.16.0.170 health` runs through without errors.

RKE2

Without an API server, you go to the node via SSH. `crictl`, `journalctl` and the files under `/var/log/pods/` work independently of Kubernetes. You correct the configuration in `config.yaml` or `config.yaml.d/`.

TALOS

The Talos API (port 50000) keeps running when Kubernetes is dead. `containers`, `logs`, `edit mc` and `etcd status` all go through it. If the Talos API is gone too, only the VM console remains.

14.5 Expired Client Certificate

Certificates expire, and they do it exactly when nobody is thinking about it. So you create a certificate that expires after ten minutes. The script `kurzlebiges-zertifikat.sh` (“short-lived certificate”) sends a CertificateSigningRequest with `expirationSeconds: 600` to the cluster, approves it and builds a separate kubeconfig for the user `dbg14-user`.

FILE szenarien/14-controlplane/kurzlebiges-zertifikat.sh (excerpt)

```
kubectl --context "$CTX" apply -f - <<YAML
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: ${NAME}
spec:
  request: $(base64 < "${DIR}/${NAME}.csr" | tr -d '\n')
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 600
  usages:
    - client auth
YAML
kubectl --context "$CTX" certificate approve "$NAME"
```

600 seconds is the smallest allowed `expirationSeconds`. The key and kubeconfig end up in `szenarien/14-controlplane/tmp-<kontext>/` (`kontext` is the context name), and Git ignores this directory.

MAC — Step 13

```
kubectl --context rke2 apply -f szenarien/14-controlplane/kaputt-zertifikat.yaml
szenarien/14-controlplane/kurzlebiges-zertifikat.sh rke2
export KC=szenarien/14-controlplane/tmp-rke2/dbg14-user.kubeconfig
kubectl --kubeconfig $KC get configmaps
```

Creates the namespace, a ConfigMap and a RoleBinding, generates the certificate and uses it to read the ConfigMaps in `dbg-14`. That works. Now take a ten-minute break.

SYMPTOM

```
$ kubectl --kubeconfig $KC get configmaps
error: You must be logged in to the server (Unauthorized)
```

Depending on the kubectl version, the parentheses may also say `the server has asked for the client to provide credentials`. Nothing in the kubeconfig has changed.

Investigate

`Unauthorized` means the API server does not know who you are. With client certificates, the first suspect is the expiration date.

MAC — Step 14

```
kubectl config view --kubeconfig $KC --raw \
  -o jsonpath='{.users[0].user.client-certificate-data}' \
  | base64 -d | openssl x509 -noout -subject -enddate
```

Extracts the certificate from the kubeconfig and shows the subject and expiration time.

```
subject=CN=dbg14-user
notAfter=Nov  9 21:50:12 2026 GMT
```

That time is in the past. The cross-check is in the API server log, which you can read with your admin access:

MAC Step 15

```
kubectl --context rke2 logs -n kube-system kube-apiserver-rke2-cp --since=15m \
  | grep -m 1 "certificate has expired"
```

On Talos, the pod is called `kube-apiserver-talos-cp`.

It says something like `"Unable to authenticate the request" err="[x509: certificate has expired or is not yet valid: current time ... is after ...]"`.

CAUSE

The client certificate has expired. The API server checks the validity on every request and rejects expired certificates. For security reasons, it only tells the client `Unauthorized` and writes the exact reason into its own log.

Message on the client	What is expired or wrong
`You must be logged in to the server (Unauthorized)`	your client certificate (or token) from the kubeconfig
`x509: certificate has expired or is not yet valid`	the API server's serving certificate, or your computer's clock is wrong
`x509: certificate signed by unknown authority`	the CA in your kubeconfig does not match the cluster, for example after a reinstall

Fix

For the test user, you simply create a new certificate by running the script again. The more important question is when the real certificates of your clusters expire.

RKE2-CP Step 16

```
rke2 certificate check --output table
```

Lists all RKE2 certificates with expiration date, remaining time and status.

MAC Step 17

```
talosctl -n 172.16.0.170 get kubernetesdynamiccerts -o yaml | head -n 20
talosctl config info
kubectl config view --context talos --minify --raw \
  -o jsonpath='{.users[0].user.client-certificate-data}' \
  | base64 -d | openssl x509 -noout -enddate
```

The dynamic Kubernetes certificates of Talos in PEM format, then the lifetime of your `talosconfig` in the line `Certificate expires`, and finally the expiration date of your admin certificate for the context `talos`.

WHAT YOU SHOULD SEE

After running the script again, `kubectl --kubeconfig $KC get configmaps` returns the ConfigMap `hallo` ("hello") again. You no longer need `$KC` for the rest of the chapter: `unset KC`.

RKE2

Certificates are valid for 365 days. At startup, RKE2 renews every certificate that has expired or expires in less than 120 days. By hand: `systemctl stop rke2-server`, `rke2 certificate rotate`, `systemctl start rke2-server`. After that, you have to copy `/etc/rancher/rke2/rke2.yaml` to the Mac again, because the admin certificate is new too.

TALOS

Talos renews the server certificates itself, and the kubelet needs at least one restart a year for that. Client certificates are your job: every `talosctl kubeconfig` creates a kubeconfig valid for one year. You renew the `talosconfig` with `talosctl config new` while the old one is still valid.

14.6 A Webhook Blocks the Namespace

The API server asks admission webhooks on every matching write. If the webhook is unreachable, `failurePolicy` decides: `Ignore` lets the request through, `Fail` rejects it. Kyverno is one example (Chapter 11). This section is about the general case.

FILE `szenarien/14-controlplane/kaputt-webhook.yaml (excerpt)`

```
kind: ValidatingWebhookConfiguration
metadata:
  name: dbg14-pruefung
webhooks:
  - name: pruefung.dbg14.example.com
    failurePolicy: Fail
    timeoutSeconds: 5
    clientConfig:
      service:
        name: pruefer
        namespace: dbg-14
        path: /validate
    namespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: dbg-14
    rules:
      - operations: ["CREATE", "UPDATE"]
        resources: ["configmaps"]
# ...
```

The Service `pruefer` ("checker") does not have a single pod (`pruefung` means "check"). The `namespaceSelector` limits the damage to `dbg-14`.

MAC Step 18

```
kubectl --context rke2 apply -f szenarien/14-controlplane/kaputt-webhook.yaml
kubectl --context rke2 create configmap test -n dbg-14 --from-literal=a=b
```

Creates the webhook and the Service, then tries to create a ConfigMap.

SYMPTOM

```
error: failed to create configmap: Internal error occurred: failed calling webhook
"pruefung.dbg14.example.com": failed to call webhook: Post
"https://pruefer.dbg-14.svc:443/validate?timeout=5s": <Grund>
```

The actual reason (`<Grund>`) comes at the end. It depends on how the API server resolves the Service and what the network answers: `connect: connection refused`, `no endpoints available for service "pruefer"` or, with a backend that does not answer, `context deadline exceeded` after `timeoutSeconds`.

Investigate

The message names the webhook, not the configuration. You find both in the list:

MAC Step 19

```
kubectl --context rke2 get \
  validatingwebhookconfigurations,mutatingwebhookconfigurations
kubectl --context rke2 get validatingwebhookconfiguration dbg14-pruefung \
  -o jsonpath='{.webhooks[*].clientConfig.service}{"\n"}'
kubectl --context rke2 get endpointslices -n dbg-14
```

All webhook configurations, then the target of the webhook in question, and finally the endpoints of the target Service.

```
NAME            ADDRESSTYPE   PORTS     ENDPOINTS   AGE
pruefer-x8k2d   IPv4          <unset>   <unset>     2m
```

In k9s, open the list with `:validatingwebhookconfigurations` and the entry's YAML with `y`. No endpoints, so no pod behind the Service.

CAUSE

For every ConfigMap in `dbg-14`, the API server has to ask the webhook. The Service behind it has no pods, so the call fails. Because of `failurePolicy: Fail`, the API server rejects the request. Without a `namespaceSelector`, this would hit every namespace, and with pods as the target, the cluster could not even restart its own components.

Fix

The right fix is to get the backend running again. If it is gone for good, you remove the configuration. In an emergency, you temporarily switch to `Ignore`:

MAC Step 20

```
kubectl --context rke2 patch validatingwebhookconfiguration dbg14-pruefung \
  --type=json \
  -p='[{"op":"replace","path":"/webhooks/0/failurePolicy","value":"Ignore"}]'
kubectl --context rke2 delete validatingwebhookconfiguration dbg14-pruefung
```

The first command is the emergency exit, the second one is the actual solution. Webhook configurations themselves never go through webhooks, so you can always change or delete them.

WHAT YOU SHOULD SEE

`kubectl --context rke2 create configmap test -n dbg-14 --from-literal=a=b` reports `configmap/test created`.

14.7 The Namespace Stays in Terminating

A namespace only disappears once the namespace controller has deleted everything in it. For that, it needs to know which resource types exist at all, so it asks the API server's discovery. If an aggregated API does not answer, the list is incomplete, and the controller does not delete the namespace, to be on the safe side. The classic case: someone removes the metrics-server, but the APIService `v1beta1.metrics.k8s.io` stays behind.

FILE `szenarien/14-controlplane/kaputt-apiservice.yaml (excerpt)`

```yaml
kind: APIService
metadata:
  name: v1alpha1.debug.dbg14.example.com
spec:
  group: debug.dbg14.example.com
  version: v1alpha1
  service:
    name: debug-api
    namespace: dbg-14
# ...
---
kind: ConfigMap
metadata:
  name: festgehalten
  namespace: dbg-14
  finalizers:
    - dbg14.example.com/aufraeumen
```

An API group whose Service has no pods, and a ConfigMap `festgehalten` ("held") with a finalizer `aufraeumen` ("clean up") that no controller ever removes. Two typical reasons for hanging namespaces in one file.

MAC Step 21

```
kubectl --context rke2 apply -f szenarien/14-controlplane/kaputt-apiservice.yaml
kubectl --context rke2 delete ns dbg-14 --wait=false
```

`--wait=false`, because otherwise `kubectl` waits forever for the end.

SYMPTOM

```
$ kubectl --context rke2 get ns dbg-14
NAME     STATUS        AGE
dbg-14   Terminating   14m
```

No change even after minutes. Some `kubectl` commands report errors about the group `debug.dbg14.example.com` along the way.

Investigate

The namespace controller writes into the namespace's conditions what it is stuck on.

MAC — Step 22

```
kubectl --context rke2 get ns dbg-14 -o json | jq -r \
  '.status.conditions[] | select(.status=="True") | "\(.reason): \(.message)"'
```

Only the conditions with status `True`, that is, the actual problems.

```
DiscoveryFailed: Discovery failed for some groups, 1 failing: unable to retrieve
  the complete list of server APIs: debug.dbg14.example.com/v1alpha1: ...
SomeResourcesRemain: Some resources are remaining: configmaps. has 1 resource
  instances
SomeFinalizersRemain: Some content in the namespace has finalizers remaining:
  dbg14.example.com/aufraeumen in 1 resource instances
```

MAC — Step 23

```
kubectl --context rke2 get apiservice v1alpha1.debug.dbg14.example.com
kubectl --context rke2 get configmaps -n dbg-14
```

The APIService of the group and what is left in the namespace. Without a name, `get apiservice` lists all of them. Those served by the API server itself show `Local` in `SERVICE`.

```
NAME                                SERVICE            AVAILABLE                 AGE
v1alpha1.debug.dbg14.example.com    dbg-14/debug-api   False (ServiceNotFound)   15m
```

Before the deletion, it said `False (MissingEndpoints)`. Now the namespace controller has already removed the Service, and the APIService points into nothing. In k9s you find both lists under `:apiservices` and `:namespaces`.

CAUSE

Two things hold on to the namespace. First, the APIService is not available, discovery stays incomplete, and the controller refuses the final deletion. Because the APIService applies cluster-wide, this affects every namespace you delete during this time. Second, the ConfigMap has a finalizer. The API server only deletes it once someone removes the finalizer, and that someone does not exist.

Fix

MAC

Step 24

```
kubectl --context rke2 delete apiservice v1alpha1.debug.dbg14.example.com
kubectl --context rke2 patch configmap festgehalten -n dbg-14 \
  --type=merge -p '{"metadata":{"finalizers":null}}'
```

First remove the orphaned APIService, then remove the finalizer. With real finalizers, first check which controller sets it and why it does not remove it.

WHAT YOU SHOULD SEE

A few seconds later, `kubectl --context rke2 get ns dbg-14` reports `NotFound`. `kubectl --context rke2 get apiservice` shows only `True` in the `AVAILABLE` column.

PITFALL

A tip that circulates online is to remove the finalizer `kubernetes` directly from the namespace via `/finalize`. The namespace is then gone, but its objects are not necessarily: they stay orphaned in etcd and reappear as soon as someone creates a namespace with the same name. Always fix the cause.

TRY IT YOURSELF

Reproduce the webhook scenario on `talos`, but first change `failurePolicy` to `Ignore` and `timeoutSeconds` to `10` in a copy of the file. Then, instead of no pod at all, start a pod in the Service `pruefer` that accepts connections but does not speak TLS (for example `nicolaka/netshoot:v0.15` with `nc -lk 8443`, adjust selector and label). How long does `kubectl create configmap` take now, and what is in the API server log?

Hint: with `Ignore`, the ConfigMap goes through, but only after the timeout. A slow webhook with `Ignore` makes the cluster sluggish instead of broken. Clean up afterwards with `kubectl --context talos delete validatingwebhookconfiguration dbg14-pruefung` and `kubectl --context talos delete ns dbg-14`.

WHAT YOU LEARNED

- `/readyz?verbose` and `/livez?verbose` show the API server's individual checks. If there is no answer at all, you need SSH or the Talos API.
- Before every intervention in the control plane: check etcd (`etcdctl` in the etcd container or `talosctl etcd status`) and take a snapshot (`rke2 etcd-snapshot save` or `talosctl etcd snapshot`).
- An API server that does not start writes the reason into its container log: on RKE2 through `crictl logs`, on Talos through `talosctl logs --namespace cri`.
- `Unauthorized` means your client certificate. `x509` on the client means the server's certificate or CA.
- A webhook with `failurePolicy: Fail` and no backend blocks every matching request. You can still always delete webhook configurations.
- Hanging namespaces explain themselves in their conditions. The most common causes: an unavailable APIService and finalizers without a controller.

CHAPTER 15

Rollouts, Autoscaling and Helm

You recognize hanging rollouts and roll them back, get an HPA with `<unknown>` to measure, install the metrics-server on Talos for it, free a stuck Helm release, and read why Jobs give up and ConfigMap changes do not arrive.

Time: 75 minutes **Requires:** Chapters 2, 3 and 6

15.1 What Breaks in This Chapter

The failures in this chapter do not happen on the first deployment, but with the second version, when scaling and when automating. What they have in common: nothing visibly crashes. The old version keeps running, the HPA does nothing, Helm refuses. If you do not check, you only notice days later.

Scenario	Symptom	Where the answer is
Rollout hangs	`rollout status` ends with `exceeded its progress deadline`	Condition `Progressing`, events of the new pod
HPA does not measure	`TARGETS` shows `cpu: <unknown>/50%`	Events of the HPA, `kubectl top`
Helm release hangs	`another operation ... is in progress`	`helm history`, status `pending-upgrade`
Job gives up	Job `Failed`, several pods `Error`	Condition and event `BackoffLimitExceeded`
ConfigMap has no effect	Pod shows the old value	How the ConfigMap is mounted

All scenarios run in the namespace `dbg-15`. The manifests comply with `restricted`, so on Talos there are no warnings from Pod Security Admission.

15.2 The Rollout Hangs

A Deployment replaces pods step by step. With `maxSurge: 1` and `maxUnavailable: 0`, it first starts a new pod and removes an old one only once the new one is ready. If it never becomes ready, the rollout stops. After `progressDeadlineSeconds`, the Deployment controller declares it failed, but it does not roll anything back.

MAC Step 1

```
kubectl --context rke2 apply -f szenarien/15-rollouts/web-v1.yaml
kubectl --context rke2 rollout status deploy/web -n dbg-15
```

Version 1 with three replicas. `rollout status` ends with `deployment "web" successfully rolled out`.

FILE `szenarien/15-rollouts/kaputt-rollout.yaml (excerpt)`

```
  annotations:
    kubernetes.io/change-cause: "v2: httpd auf 8081"
spec:
  progressDeadlineSeconds: 60
# ...
          command: ["httpd", "-f", "-p", "8081", "-h", "/tmp"]
          readinessProbe:
            tcpSocket:
              port: 8080
```

Version 2 listens on a new port (“httpd auf 8081” means “httpd on 8081”), but the readiness probe still checks the old one. The deadline is short at 60 seconds. The default is 600.

MAC Step 2

```
kubectl --context rke2 apply -f szenarien/15-rollouts/kaputt-rollout.yaml
kubectl --context rke2 rollout status deploy/web -n dbg-15
```

Starts the rollout to version 2 and watches it.

SYMPTOM

```
Waiting for deployment "web" rollout to finish: 1 out of 3 new replicas have been
updated...
error: deployment "web" exceeded its progress deadline
```

The application still answers: `kubectl get deploy -n dbg-15` shows `READY 3/3`, `UP-TO-DATE 1`. The three old pods keep running.

Investigate

MAC Step 3

```
kubectl --context rke2 describe deploy web -n dbg-15 \
  | sed -n '/^Conditions/,/^Events/p'
kubectl --context rke2 get rs,pods -n dbg-15 -l app=web
```

The conditions of the Deployment, then the ReplicaSets and pods. The new ReplicaSet has one pod that is `0/1` ready.

```
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    False   ProgressDeadlineExceeded
```

`Available True` together with `Progressing False` means: the application runs, but the new version does not arrive. The new pod gives the reason:

MAC — Step 4

```
kubectl --context rke2 events -n dbg-15 --types=Warning
```

Only warnings in the namespace.

```
LAST SEEN   TYPE      REASON      OBJECT                    MESSAGE
4s          Warning   Unhealthy   pod/web-6d5f8c9b7-tq4lz   Readiness probe ...
```

The full message is `Readiness probe failed: dial tcp 10.42.1.23:8080: connect: connection refused`. In k9s, `:deploy dbg-15` opens the Deployments, and `z` the ReplicaSets of the selected entry. `Enter` takes you on to the pods, and `d` to the events.

CAUSE

The new pod listens on 8081, the readiness probe knocks on 8080 and gets `connection refused`. The pod never becomes ready, so the Deployment removes no old pod and starts no further new one. After 60 seconds without progress, the controller sets `Progressing=False` with `ProgressDeadlineExceeded`. That is only a message. The rollout stays stuck in this state.

Fix

MAC — Step 5

```
kubectl --context rke2 rollout history deploy/web -n dbg-15
kubectl --context rke2 rollout undo deploy/web -n dbg-15
```

`history` lists the revisions with the `change-cause` annotation. `undo` goes back to the previous revision, and with `--to-revision=<n>` to a specific one.

```
REVISION  CHANGE-CAUSE
1         v1: httpd auf 8080
2         v2: httpd auf 8081
```

Then you roll out the corrected version, in which probe and port match:

MAC — Step 6

```
kubectl --context rke2 apply -f szenarien/15-rollouts/fix-rollout.yaml
kubectl --context rke2 rollout status deploy/web -n dbg-15
```

Version 2 with `readinessProbe` on port 8081.

WHAT YOU SHOULD SEE

`deployment "web" successfully rolled out`, and `kubectl --context rke2 get deploy web -n dbg-15` shows `3/3` with `UP-TO-DATE 3`. `rollout history` now shows the new `change-cause`.

PITFALL

With a broken rollout, `kubectl apply` reports `configured` and ends successfully. Pipelines that only run `apply` notice nothing. A `kubectl rollout status --timeout=...` afterwards returns a non-zero exit code when the deadline is exceeded and makes the pipeline fail.

15.3 The HPA Does Not Measure

A HorizontalPodAutoscaler with the target `averageUtilization: 50` calculates like this: current CPU usage divided by the CPU request, averaged over all pods. For that, it needs two things: measurements from the Resource Metrics API (`metrics.k8s.io`, provided by the metrics-server) and a request in every container.

FILE `szenarien/15-rollouts/kaputt-hpa.yaml (excerpt)`

```
      containers:
        - name: rechner
          image: busybox:1.37
          resources:
            limits:
              memory: 32Mi
---
kind: HorizontalPodAutoscaler
spec:
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

No CPU request in the container `rechner` (“calculator”). You cannot calculate 50 percent of nothing.

MAC Step 7

```
kubectl --context rke2 apply -f szenarien/15-rollouts/kaputt-hpa.yaml
kubectl --context talos apply -f szenarien/15-rollouts/kaputt-hpa.yaml
```

The same manifest on both clusters.

SYMPTOM

```
$ kubectl --context rke2 get hpa -n dbg-15
NAME      REFERENCE            TARGETS              MINPODS   MAXPODS   REPLICAS   AGE
rechner   Deployment/rechner   cpu: <unknown>/50%   1         4         1          2m
```

The same picture on both clusters. But the causes are different.

Investigate

MAC Step 8

```
kubectl --context rke2 describe hpa rechner -n dbg-15 | sed -n '/^Conditions/,$p'
kubectl --context talos describe hpa rechner -n dbg-15 | sed -n '/^Events/,$p'
```

Conditions and events of the HPA on both clusters.

On RKE2, the condition says `ScalingActive False` with the reason `FailedGetResourceMetric`, and the event reads:

```
failed to get cpu utilization: missing request for cpu in container rechner
of Pod rechner-7b9c6d8f5-l2kxn
```

On Talos it fails one step earlier:

```
failed to get cpu utilization: unable to get metrics for resource cpu: unable
to fetch metrics from resource metrics API: the server could not find the
requested resource (get pods.metrics.k8s.io)
```

MAC Step 9

```
kubectl --context talos top pods -n dbg-15
kubectl --context talos get apiservice v1beta1.metrics.k8s.io
```

On Talos, `top` reports `error: Metrics API not available`, and the APIService does not exist (`NotFound`).

On RKE2 both return values, because `rke2-metrics-server` runs there.

In k9s, `:hpa dbg-15` shows the HPAs, and `d` the describe view with the events. If the Metrics API is missing, the CPU and memory columns in the k9s pod view also stay empty.

CAUSE

On RKE2 the CPU request is missing: the HPA gets measurements but cannot convert them into percent. On Talos the source of measurements is missing as well, because Talos does not ship a metrics-server. In both cases the HPA sets `ScalingActive=False` and does not change the replica count, neither up nor down.

Fix on Talos: Install the metrics-server

The metrics-server fetches its values over HTTPS from the kubelet of every node. By default, the Talos kubelet signs its serving certificate itself, and the metrics-server would reject it. So the Talos documentation describes three steps: the kubelet requests its certificate from the cluster CA, an approver approves these requests, and then the metrics-server comes in.

FILE `labor/talos/patch-kubelet-serving-certs.yaml (excerpt)`

```
apiVersion: v1alpha1
kind: KubeletConfig
```

```
extraArgs:
  rotate-server-certificates: "true"
```

The kubelet then submits a CertificateSigningRequest with the signer `kubernetes.io/kubelet-serving`.

MAC — Step 10

```
talosctl -n 172.16.0.170,172.16.0.171 patch mc \
  --patch @labor/talos/patch-kubelet-serving-certs.yaml
kubectl --context talos apply -f https://raw.githubusercontent.com/\
alex1989hu/kubelet-serving-cert-approver/v0.12.1/deploy/standalone-install.yaml
kubectl --context talos get csr
```

The patch on both nodes, then the approver at the pinned version 0.12.1. After a short while, the CSRs with the signer `kubernetes.io/kubelet-serving` show `Approved,Issued`.

PITFALL

As long as the serving CSRs are `Pending`, the kubelet has no certificate. Then `kubectl logs` and `kubectl exec` on this node also fail with a TLS error. So install the approver right after the patch. Without the patch, on the other hand, the metrics-server reports something like `x509: cannot validate certificate for 172.16.0.171 because it doesn't contain any IP SANs`. The shortcut `--kubelet-insecure-tls` turns off the check. The Talos documentation deliberately takes the secure route.

FILE `werkzeuge/metrics-server-values-talos.yaml (excerpt)`

```
replicas: 1
args: []
resources:
  requests:
    cpu: 50m
    memory: 64Mi
```

No `--kubelet-insecure-tls`. The chart's default arguments query the kubelet through the `InternalIP` and the port from the node status.

MAC — Step 11

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm --kube-context talos upgrade --install metrics-server \
  metrics-server/metrics-server --version 3.14.0 \
  -n kube-system -f werkzeuge/metrics-server-values-talos.yaml --wait
kubectl --context talos top nodes
```

Installs metrics-server 0.9.0 into `kube-system`. After about a minute, `top nodes` returns CPU and memory for both nodes.

Now the Talos HPA shows the same message as RKE2: `missing request for cpu`. The fix sets the request for both:

MAC Step 12

```
kubectl --context rke2 apply -f szenarien/15-rollouts/fix-hpa.yaml
kubectl --context talos apply -f szenarien/15-rollouts/fix-hpa.yaml
```

The container now requests `cpu: 50m`.

WHAT YOU SHOULD SEE

After one to two minutes, `kubectl get hpa -n dbg-15` shows `cpu: 0%/50%` or a small percentage on both clusters. `describe hpa` reports `ScalingActive True`.

RKE2

The metrics-server ships as `rke2-metrics-server` and is managed by the RKE2 Helm controller. You customize it with a `HelmChartConfig` in `/var/lib/rancher/rke2/server/manifests/`.

TALOS

No metrics-server, no Metrics API. You install it yourself, following the pattern from the Talos documentation with serving certificates from the cluster CA. That also means you have to update it yourself.

15.4 The Helm Release Hangs

Helm stores every revision of a release as a Secret in the namespace. Before an upgrade, it creates the new revision with the status `pending-upgrade`, and at the end it sets `deployed` or `failed`. If Helm is killed hard in between, `pending-upgrade` stays behind. That status is also the lock against parallel upgrades. A typical trigger in practice: a CI runner aborts a job for exceeding its time limit while `helm upgrade --wait` is still waiting.

FILE `szenarien/15-rollouts/haenger.sh (excerpt)`

```
helm --kube-context "$CTX" upgrade demo "$CHART" \
  -n dbg-15 --set readinessPort=9999 --wait --timeout 5m &
HELM_PID=$!
sleep 15
kill -9 "$HELM_PID"
```

The script `haenger.sh` (“hanger”): revision 2 never becomes ready, and `helm` waits. After 15 seconds comes `SIGKILL`. Helm would still react to `Ctrl-C` or `SIGTERM` and mark the release as `failed`, but not to `SIGKILL`.

MAC Step 13

```
szenarien/15-rollouts/haenger.sh rke2
helm --kube-context rke2 upgrade demo \
  szenarien/15-rollouts/demo-chart -n dbg-15
```

The script installs revision 1 from the mini chart `demo-chart` and aborts the upgrade to revision 2. Then you try to carry on as usual.

SYMPTOM

```
Error: UPGRADE FAILED: another operation (install/upgrade/rollback) is in progress
```

Yet no other operation is running at all. Every further attempt ends the same way.

Investigate

MAC — Step 14

```
helm --kube-context rke2 history demo -n dbg-15
kubectl --context rke2 get secrets -n dbg-15 -l owner=helm --show-labels
```

The revisions of the release, then the Secrets in which Helm stores them. The `status` label of each Secret matches the `STATUS` column.

```
REVISION  UPDATED                   STATUS           CHART       ... DESCRIPTION
1         Mon Nov  9 22:41:03 2026  deployed         demo-0.1.0  ... Install complete
2         Mon Nov  9 22:41:20 2026  pending-upgrade  demo-0.1.0  ... Preparing upgrade
```

In k9s, `:helm` opens the releases, and `r` shows the history of the selected release. `helm list -n dbg-15` also shows the release, because Helm 4 lists all states by default. With `--pending` you filter specifically.

CAUSE

The latest revision is `pending-upgrade`. Before every upgrade, Helm checks whether the most recent revision is in a `pending` state, and if so aborts with `another operation ... is in progress`. The process that was supposed to finish the revision was terminated with `SIGKILL` and could not clean anything up.

Fix

MAC — Step 15

```
helm --kube-context rke2 rollback demo 1 -n dbg-15
helm --kube-context rke2 upgrade demo szenarien/15-rollouts/demo-chart \
  -n dbg-15 --wait
```

The rollback does not check the lock and creates revision 3 as a copy of revision 1. After that, a normal upgrade works again. In the k9s history, you roll back with `r` on the revision you want.

WHAT YOU SHOULD SEE

`helm history demo -n dbg-15` ends with a revision in status `deployed`. Revision 3 has the description `Rollback to 1`.

If the rollback does not help, for example because the very first installation hangs (`pending-install`), you are left with deleting the Secret of the hanging revision: `kubectl delete secret sh.helm.release.v1.demo.v2 -n dbg-15`. After that, the previous revision applies again. Only do this if you are sure that no Helm process is really running anymore.

RKE2

RKE2 installs its bundled charts (`rke2-traefik`, `rke2-coredns` …) through the built-in Helm controller, visible as the Jobs `helm-install-rke2-...` in `kube-system`. You do not change these releases with `helm upgrade`, but with `HelmChartConfig`.

TALOS

Talos has no Helm controller. Everything you install with Helm, including the metrics-server and Traefik, you manage yourself with the Helm CLI.

15.5 The Job Gives Up

On failure, a Job starts new pods until `backoffLimit` is reached. The waiting time between attempts grows: 10 seconds, 20, 40 and so on. `activeDeadlineSeconds` also limits the total runtime.

MAC — Step 16

```
kubectl --context rke2 apply -f szenarien/15-rollouts/kaputt-job.yaml
```

The Job `import` expects `/daten/import.csv`, and the file does not exist. `backoffLimit: 2` allows two retries.

SYMPTOM

```
$ kubectl --context rke2 get jobs,pods -n dbg-15 -l app=import
NAME               STATUS   COMPLETIONS   DURATION   AGE
job.batch/import   Failed   0/1           52s        52s

NAME               READY   STATUS   RESTARTS   AGE
pod/import-7xq9d   0/1     Error    0          52s
pod/import-bk2mw   0/1     Error    0          41s
pod/import-zr4pt   0/1     Error    0          20s
```

Investigate

MAC — Step 17

```
kubectl --context rke2 describe job import -n dbg-15 | sed -n '/^Events/,$p'
kubectl --context rke2 logs -n dbg-15 job/import
```

The events of the Job, then the log of one of its pods. `logs job/...` picks a pod by itself.

```
  Warning  BackoffLimitExceeded  2s   job-controller  Job has reached the specified
                                                      backoff limit
FEHLER: /daten/import.csv fehlt
```

The last line means “ERROR: /daten/import.csv is missing”. In k9s, `:jobs dbg-15` shows the Jobs. `Enter` leads to the pods, `l` to the logs and `p` to the logs of the previous container, if one was restarted.

CAUSE

Every attempt ends with exit code 1. After three pods (one attempt plus two retries), the Job controller sets the condition `Failed` with the reason `BackoffLimitExceeded` and starts nothing more. If `activeDeadlineSeconds` had expired instead, it would say `DeadlineExceeded` with `Job was active longer than specified deadline`.

Fix

The pod template of a Job is immutable. An `apply` with a changed command fails with `field is immutable`. You delete the Job and create it again.

MAC — Step 18

```
kubectl --context rke2 delete job import -n dbg-15
kubectl --context rke2 apply -f szenarien/15-rollouts/fix-job.yaml
kubectl --context rke2 wait --for=condition=Complete job/import \
  -n dbg-15 --timeout=60s
```

The corrected Job creates its input file itself. `wait` returns as soon as the Job is done.

WHAT YOU SHOULD SEE

`job.batch/import condition met`, and `kubectl get jobs -n dbg-15` shows `Complete` and `1/1`.

PITFALL

`ttlSecondsAfterFinished: 3600` deletes the Job one hour after it ends, together with its pods and logs. For CronJobs, `successfulJobsHistoryLimit` and `failedJobsHistoryLimit` clean up as well. Investigate failed Jobs promptly. To test a CronJob on purpose, use `kubectl create job --from=cronjob/<name> test-1`.

15.6 The ConfigMap Change Does Not Arrive

MAC — Step 19

```
kubectl --context rke2 apply -f szenarien/15-rollouts/kaputt-configmap.yaml
kubectl --context rke2 patch configmap begruessung -n dbg-15 \
  --type=merge -p '{"data":{"GRUSS":"Hallo v2"}}'
```

The Deployment `gruss` (“greeting”) reads `GRUSS` via `envFrom` from the ConfigMap `begruessung` (“welcome”) and prints the value every ten seconds. Then you change the ConfigMap.

SYMPTOM

`kubectl --context rke2 logs -n dbg-15 deploy/gruss --tail 3` still shows only `Hallo v1` minutes later.

CAUSE

The kubelet sets environment variables once, when the container starts. A later change to the ConfigMap never reaches the running process. The kubelet does update ConfigMaps mounted as volumes, with a delay (sync period plus cache, together often one to two minutes), except for `subPath` mounts. Whether the application then rereads the changed file is up to the application.

MAC

Step 20

```
kubectl --context rke2 rollout restart deploy/gruss -n dbg-15
```

Restarts the pods, which read the ConfigMap at startup. In k9s, use `r` in the Deployment view.

WHAT YOU SHOULD SEE

After the restart, `kubectl --context rke2 logs -n dbg-15 deploy/gruss --tail 1` shows the text `Hallo v2`.

In Helm charts, you solve this permanently with an annotation on the pod template that contains a checksum of the ConfigMap, for example `checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}`. If the ConfigMap changes, the template changes, and Kubernetes rolls out again.

15.7 Clean Up

MAC

Step 21

```
helm --kube-context rke2 uninstall demo -n dbg-15
kubectl --context rke2 delete ns dbg-15
kubectl --context talos delete ns dbg-15
```

Removes the Helm release and both namespaces. The metrics-server on Talos stays, because you need it in the remaining exercises and for k9s.

TRY IT YOURSELF

In a copy of `fix-rollout.yaml`, set the image to `busybox:1.37-gibtesnicht` ("does not exist"), `progressDeadlineSeconds` to `30` and the `change-cause` to `v3: neues Image` ("new image"). Apply it. Which condition does the Deployment have after a minute, what status does the new pod have, and which revision do you restore with `rollout undo --to-revision` so that `v2` with the corrected probe runs?

Hint: again `Progressing False` with `ProgressDeadlineExceeded`, but this time the new pod is in `ImagePullBackOff` (Chapter 5). `rollout history` shows you which revision has the `change-cause` "Probe korrigiert" ("probe fixed").

WHAT YOU LEARNED

- A hanging rollout shows `Progressing=False` with `ProgressDeadlineExceeded`. Kubernetes does not roll back by itself. You do that with `rollout undo`.
- `<unknown>` on an HPA has two common reasons: no Metrics API (Talos without metrics-server) or no request in the container. The HPA's events tell the two apart.
- The metrics-server on Talos needs kubelet serving certificates from the cluster CA: `rotate-server-certificates`, an approver for the CSRs, then the chart.
- `another operation (install/upgrade/rollback) is in progress` means the latest revision is `pending-*`. `helm history` shows it, and `helm rollback` releases the lock.
- Jobs give up after `backoffLimit` with `BackoffLimitExceeded`. Their template is immutable, so delete and recreate them.
- ConfigMaps used as environment variables only take effect after a restart: `rollout restart` or a checksum annotation.

CHAPTER 16

Appendix

Error messages, commands and keys to look up quickly when things are on fire.

16.1 Cheat Sheet: From the Message to the First Command

The left column is what you see. The middle one names the most common cause, not the only one. The right one tells you where to look first.

Message or status	Most common cause	First look	Ch.
`Pending`, `Insufficient cpu`	Requests larger than the free capacity on every node	`describe pod`, events	4
`didn't match Pod's node affinity/ selector`	Label missing on the node or typo in the selector	`get nodes --show-labels`	4
`had untolerated taint(s)`	Taint without a matching toleration	`describe node`, `Taints:`	4
`unbound immediate PersistentVolumeClaims`	PVC without a StorageClass or with a wrong name	`describe pvc`	4, 9
Deployment without pods, `FailedCreate`	ResourceQuota, LimitRange or Pod Security rejects	Events on the ReplicaSet	4, 10
`ImagePullBackOff`, `not found`	Typo in the tag or repository	`describe pod`, events	5
`ImagePullBackOff`, `pull access denied`	Private repository without `imagePullSecrets`	Secret in the same namespace	5
`toomanyrequests`	Docker Hub rate limit	Registry mirror on the node	5
`CreateContainerConfigError`	Secret, ConfigMap or key missing	`describe pod`	5
`exec format error`	Script without a shebang or wrong architecture	Check image and entrypoint	5
`CrashLoopBackOff`, exit 127	Command not found in the container	`logs --previous`	6
`CrashLoopBackOff`, exit 1	Application exits with an error	`logs --previous`	6

Message or status	Most common cause	First look	Ch.
`OOMKilled`, exit 137	Memory limit too small or memory leak	`lastState`, limits	6
Restarts, `failed liveness probe`	Probe too strict, startup takes longer	Events, `startupProbe`	6
`Running`, but `0/1` ready	Readiness probe fails	Events, EndpointSlice	6, 7
`Read-only file system`	`readOnlyRootFilesystem` without a writable volume	`securityContext`	6
Service without endpoints	Selector does not match the pod labels	`get endpointslices`	7
`connection refused`	Wrong `targetPort` or the process listens elsewhere	`kubectl debug`, `ss -ltn`	7
`NXDOMAIN`, DNS timeout	Wrong name or namespace, or egress on port 53 blocked	`nslookup` from a pod	7
NetworkPolicy has no effect	CNI does not enforce policies (Flannel on Talos)	Check the CNI	7
HTTP 404 from the ingress	Wrong `ingressClassName`, host or Service port	Controller logs	8
HTTP 503 from the ingress	No ready pod behind the Service	Endpoints, readiness	8
`TRAEFIK DEFAULT CERT`	TLS Secret malformed or not found	`describe secret`, Traefik log	8
`Permission denied` on a volume	The process UID may not write	`ls -ln` in the pod	9
`Multi-Attach`, pod on another node stuck	RWO volume, replicas on several nodes	`get pods -o wide`	9
`Forbidden`, `cannot list resource`	RBAC: binding missing, wrong namespace or Role instead of ClusterRole	`auth can-i --as`	10
`violates PodSecurity`	Pod Security Admission of the namespace	Namespace labels	10
`admission webhook ... denied the request`	A Kyverno policy applies	Policy name from the message	11

Message or status	Most common cause	First look	Ch.
`failed calling webhook`	Webhook backend unreachable, `failurePolicy: Fail`	`get validatingwebhookconfigurations`	11, 14
Falco alert	A process does something a rule considers suspicious	Alert fields, pod, container ID	12
Node `NotReady`	Kubelet not running or cannot reach the API server	Kubelet log on the node	13
Pod `Terminating`, `Unknown`	Node down, eviction after 300 s	`get nodes`, tolerations	13
`Evicted`	Node or pod exceeded a limit for memory or disk	`status.message`	13
Drain hangs	PodDisruptionBudget allows no disruption	`get pdb`	13
`x509: certificate has expired`	Client certificate in the kubeconfig expired	Decode the certificate	14
Namespace stays `Terminating`	Unreachable APIService or finalizer	`get apiservice`, conditions	14
`ProgressDeadlineExceeded`	New pods never become ready	`rollout status`, new pods	15
HPA shows `<unknown>`	metrics-server missing or pods without CPU requests	`top pods`, `describe hpa`	15
`another operation ... is in progress`	Interrupted Helm release in a `pending-*` state	`helm history`	15
`BackoffLimitExceeded`	Job pods fail more often than allowed	Logs of the job pods	15

16.2 RKE2 and Talos: Same Question, Two Commands

On RKE2, the commands run on the node, after `sudo -i` and the exports from chapter 2. On Talos, they run on the Mac, with `-n` and the node's IP.

Question	RKE2 (on the node)	Talos (on the Mac)
Is the node service running?	`systemctl status rke2-server` or `rke2-agent`	`talosctl -n <IP> services`
Log of the node service	`journalctl -u rke2-server`	`talosctl -n <IP> logs kubelet`

Question	RKE2 (on the node)	Talos (on the Mac)
Kubelet log	`/var/lib/rancher/rke2/agent/logs/kubelet.log`	`talosctl -n <IP> logs kubelet`
Kernel messages, OOM killer	`journalctl -k`	`talosctl -n <IP> dmesg`
Containers on the node	`crictl ps -a`	`talosctl -n <IP> containers --namespace cri`
Log of a container	`crictl logs <id>`	`talosctl -n <IP> logs --namespace cri <id>`
Check or pull an image	`crictl images`, `crictl pull`	`talosctl -n <IP> image list`, `image pull`
Live overview	`top`, `df -h`	`talosctl -n <IP> dashboard`
Configuration	`/etc/rancher/rke2/config.yaml`	`talosctl -n <IP> get machineconfig -o yaml`
Change the configuration	Edit the file, restart the service	`talosctl -n <IP> patch mc --patch @file`
Registry mirror	`/etc/rancher/rke2/registries.yaml`	`RegistryMirrorConfig` document
etcd health	`etcdctl` in the etcd pod	`talosctl -n <IP> etcd status`, `etcd members`
etcd snapshot	`rke2 etcd-snapshot save`	`talosctl -n <IP> etcd snapshot <file>`
Check certificates	`rke2 certificate check`	`talosctl config info`, `get kubernetesdynamiccerts`
Collect everything	Logs and `crictl` output by hand	`talosctl -n <IP> support --no-encryption`

16.3 k9s: The Most Important Keys

Key	Effect
`:pods`, `:deploy`, `:events`	Switch the resource view, with a namespace after it: `:pods dbg-04`
`:ctx`	Switch context
`0`	Show all namespaces
`/text`, `/!text`, `/-l app=web`	Filter, inverse filter, filter by label
`d`, `y`, `e`	Describe, YAML, edit
`l`, `p`	Logs, logs of the previous run
`s`	Shell in the container
`Enter`	Open a pod's container view
`o`, `Shift-j`	Jump to the node, jump to the owner

Key	Effect
`Shift-f`	Set up a port forward
`Ctrl-w`	Show more columns
`Ctrl-d`, `Ctrl-k`	Delete with confirmation, delete immediately
`:xray deploy`, `:pulses`	Dependency tree, cluster overview
`Shift-D`	Plugin from chapter 3: netshoot debug container on the pod
`Shift-E`	Plugin from chapter 3: events of the selected resource
`Esc`, `?`	Back, help with all keys of the current view

Start k9s with `--readonly` when you only want to look and not change anything.

16.4 Versions

All details apply to the state of this book, November 9, 2026.

Component	Version
RKE2	`v1.36.6+rke2r1`
Talos Linux and `talosctl`	`v1.14.5`
Kubernetes on both clusters	`1.36.6`
k9s	`v0.51.0`
Kyverno (chart `kyverno/kyverno`)	`v1.19.2` (chart `3.9.2`)
Falco (chart `falcosecurity/falco`)	`0.45.2` (chart `9.2.2`)
Traefik for Talos (chart `traefik/traefik`)	`v3.7.17` (chart `42.3.1`)
metrics-server for Talos (chart `metrics-server/metrics-server`)	chart `3.14.0`
local-path-provisioner	`v0.0.37`
netshoot	`nicolaka/netshoot:v0.15`

16.5 The Repo at a Glance

The repo is shared with the German edition, so folder and file names are German: `labor` is the lab, `werkzeuge` the tools, `szenarien` the scenarios, `kaputt` means broken.

```
kubernetes-debuggen/
├── buch/            source of the German edition (Typst), book-en/ this one
├── labor/
│   ├── rke2/        config-cp.yaml, config-w1.yaml, registries-mirror.yaml
│   └── talos/       patch-cp.yaml, patch-w1.yaml, patch-local-path.yaml,
```

```
│                   patch-registry.yaml and the patches from ch. 13 and 14
├── k9s/            plugins.yaml                                  (ch. 2, 3)
├── werkzeuge/      Helm values for Kyverno, Falco, Traefik, metrics-server,
│                   local-path for Talos
├── policies/       Kyverno policies with tests under tests/      (ch. 11)
└── szenarien/      one folder per chapter, with kaputt-*.yaml and fix-*.yaml
```

Every `kaputt-*.yaml` file creates its own namespace `dbg-<chapter>`. That makes cleanup always the same: `kubectl --context <cluster> delete namespace dbg-<chapter>`.

16.6 Official Documentation

- Kubernetes, debugging: kubernetes.io/docs/tasks/debug
- RKE2: docs.rke2.io
- Talos Linux 1.14: docs.siderolabs.com/talos/v1.14
- k9s: k9scli.io
- Kyverno: kyverno.io/docs
- Falco: falco.org/docs
- Traefik: doc.traefik.io/traefik

WHAT YOU LEARNED

- Troubleshooting is a method, not a talent: symptom, layer, cause, fix, verification.
- Most answers are in the events, in `describe` and in the logs of the previous run.
- On RKE2 you SSH into the node, on Talos you ask the API with `talosctl`.
- Policies, admission and runtime security are failure sources of their own. Read the message to the end, it almost always names the culprit.

DEBUGGING KUBERNETES

One pod is stuck in `Pending`, another has been restarting for an hour, and the webhook rejects every change. Where do you start?

This book turns troubleshooting into a routine. You build two clusters, one with RKE2 and one with Talos Linux, and break them on purpose. For every failure you see the symptom, the investigation with `kubectl` and k9s, the cause and the fix, on both distributions.

INSIDE

Method	From symptom to layer to cause
Tools	kubectl, k9s, crictl, talosctl and kubectl debug
Workloads	Pending, ImagePullBackOff, CrashLoopBackOff, OOMKilled
Network	Services, DNS, NetworkPolicies, Ingress and TLS
Security	RBAC, Pod Security, Kyverno and Falco as a source of failures
Platform	Nodes, etcd, certificates, webhooks and rollouts
Distributions	RKE2 and Talos side by side

Thomas Zachmann
Author on Leanpub



Hands-on guide · As of November 9, 2026